COMP4211: Machine Learning

November 11, 2026

Contents

1 Foundations of Supervised Learning **4**

1.1 Course Introduction . . . 4
1.1.1 Machine Learning vs Traditional Programming . . . 4
1.1.2 What is Machine Learning? . . . 4
1.1.3 Course Organization . . . 4
1.1.4 Course Outcomes . . . 4
1.2 Linear Regression . . . 5
1.2.1 Linear Regression: Problem Statement . . . 5
1.2.2 Mean Squared Error . . . 5
1.2.3 Matrix Representation . . . 5
1.2.4 Gradient Derivation and the Normal Equation . . . 6
1.2.5 Beyond Linear Regression: Feature Mapping . . . 6
1.2.6 Model Capacity, Overfitting and Underfitting . . . 6
1.2.7 Model Selection . . . 7
1.2.8 Regularization . . . 8
1.2.9 Performance Metrics for Regression . . . 9
1.2.10 Probabilistic Formulation of Linear Regression . . . 9
1.3 Logistic Regression and Gradient Descent . . . 10
1.3.1 Logistic Regression: Problem Statement . . . 10
1.3.2 Entropy, Cross Entropy, and KL Divergence . . . 11
1.3.3 Cross Entropy for Supervised Learning . . . 12
1.3.4 Likelihood and Maximum Likelihood Estimation . . . 12
1.3.5 Cross Entropy Loss for Logistic Regression . . . 13
1.3.6 Gradient Descent . . . 13
1.3.7 Stochastic Gradient Descent and Variants . . . 14
1.3.8 Softmax Regression (Multi-class) . . . 14
1.3.9 Performance Metrics for Classification . . . 16
1.4 Other Classification Approaches . . . 16
1.4.1 Optimization Approach: Empirical Risk Minimization . . . 16
1.4.2 Generative Approach: Naive Bayes and LDA . . . 17
1.4.3 Discriminative vs Generative: Summary . . . 19
1.5 The Bias–Variance Decomposition . . . 19
1.5.1 Setup and Notation . . . 19
1.5.2 Full Derivation . . . 19
1.5.3 Trade-off Illustration . . . 20
1.5.4 Methods to Control Variance . . . 20
1.5.5 Bias–Variance for Classification . . . 21

2 Deep Neural Networks **21**
2.1 Deep Feedforward Networks . . . 21
2.1.1 Motivation: From Probabilistic Models to Deep Learning . . . 21
2.1.2 Feedforward Neural Networks as Function Approximators . . . 21
2.1.3 Weight Initialization . . . 21
2.1.4 Activation Functions . . . 22
2.1.5 Forward Propagation . . . 24
2.1.6 Universal Approximation Theorem . . . 24
2.1.7 FNN as a Probabilistic Model . . . 25
2.1.8 Backpropagation . . . 26
2.2 Regularization and Optimization . . . 27
2.2.1 Regularization . . . 27
2.2.2 Optimization Algorithms . . . 29
2.2.3 Learning Rate Schedules . . . 31
2.2.4 Batch Normalization . . . 31
2.3 Convolutional Neural Networks . . . 33
2.3.1 Introduction and Inductive Biases . . . 33
2.3.2 The Convolutional Layer . . . 33
2.3.3 Technical Issues: Padding, Stride, Output Size . . . 34
2.3.4 Pooling Layer . . . 34
2.3.5 Example CNN Architectures . . . 35
2.3.6 Data Augmentation . . . 36
2.4 Recurrent Neural Networks . . . 37
2.4.1 Why Sequential Models? . . . 37
2.4.2 Vanilla Recurrent Neural Network . . . 37
2.4.3 Vanishing and Exploding Gradients . . . 38
2.4.4 Long Short-Term Memory (LSTM) . . . 38
2.4.5 Gated Recurrent Unit (GRU) . . . 40
2.4.6 RNN Architectures and Attention . . . 40

3 Attention-Based Architectures **41**
3.1 Self-Attention and the Transformer . . . 41
3.1.1 Self-Attention Layer . . . 41
3.1.2 Multi-Head Attention . . . 43
3.1.3 Positional Encoding . . . 43
3.1.4 Encoder . . . 43
3.1.5 Decoder . . . 44
3.1.6 BERT: Transformer Encoder for Representation Learning . . . 44
3.2 Decoder-Only Transformers and Large Language Models . . . 45
3.2.1 Decoder-Only Transformer Models . . . 45
3.2.2 Three-Stage LLM Training Pipeline . . . 45
3.2.3 Using LLMs . . . 46
3.2.4 Scaling LLMs . . . 47
3.3 Vision Transformer (ViT) and CLIP . . . 47
3.3.1 Vision Transformer (ViT) . . . 47
3.3.2 CLIP: Contrastive Language–Image Pre-training . . . 48

4 Classical Machine Learning Methods **50**

4.1 Support Vector Machines . . . 50
4.1.1 Problem Statement . . . 50
4.1.2 Hard-Margin Support Vector Machine . . . 50
4.1.3 Soft-Margin Support Vector Machine . . . 54
4.1.4 Relationship with the Hinge Loss . . . 55
4.1.5 Kernel Extension for Nonlinear Models . . . 55
4.2 Decision Trees and Random Forests . . . 58
4.2.1 Decision Trees . . . 58
4.2.2 Decision Tree Learning Algorithm . . . 58
4.2.3 Decision Tree Ensembles and Random Forests . . . 61
4.2.4 Continuous-Valued Attributes . . . 63

5 Generative Models **64**
5.1 Variational Autoencoder . . . 64
5.1.1 Motivation: Unsupervised Generative Modeling . . . 64
5.1.2 Encoder and Decoder . . . 64
5.1.3 Objective Function: ELBO . . . 64
5.1.4 The Reparameterization Trick . . . 65
5.1.5 Closed-Form KL Divergence for Two Gaussians . . . 66
5.1.6 Final VAE Objective . . . 66
5.1.7 Generating Examples with a Trained VAE . . . 66
5.1.8 VAEs in Modern Deep Generative Modeling . . . 67
5.2 Generative Adversarial Networks . . . 67
5.2.1 The Minimax Objective . . . 67
5.2.2 Training Procedure . . . 69
5.2.3 Theoretical Analysis . . . 69
5.2.4 Two Generator Cost Functions and Vanishing Gradients . . . 70
5.2.5 Mode Collapse and Training Instability . . . 70
5.2.6 GAN vs. VAE . . . 70
5.2.7 Fréchet Inception Distance (FID) . . . 71
5.2.8 Legacy and Influence . . . 71
5.3 Diffusion Models . . . 71
5.3.1 Denoising Diffusion Probabilistic Models (DDPM): The Forward Process . . . 72
5.3.2 The Reverse Process: Learning to Denoise . . . 74
5.3.3 Training Objective . . . 75
5.3.4 Theory: Variational Derivation . . . 76
5.3.5 Making Diffusion Models More Efficient . . . 77
5.3.6 Three Representations of the Reverse Process . . . 78
5.3.7 Guided Generation . . . 79

1 Foundations of Supervised Learning

1.1 Course Introduction

1.1.1 Machine Learning vs Traditional Programming

In **traditional programming**, we provide:

- Input data and explicit instructions (program/logic)
- The computer then processes this to generate an output

In **machine learning**, we provide:

- Input data and expected output (labels)
- The computer then analyzes the data to learn patterns and automatically generate a model or program that can make predictions or decisions on new, unseen data

The key distinction is that traditional programs are written by humans, while machine learning programs are learned from data.

1.1.2 What is Machine Learning?

Arthur Samuel (1959)

Machine learning is a "field of study that gives computers the ability to learn without being explicitly programmed".

Machine learning is the science of getting machines to learn and act in a similar way to humans while also autonomously learning from real-world interactions and sets of training data that we feed them.

Formal Definition

Machine Learning is the science of making computer artifacts improve their performance with respect to a certain performance criterion using example data or past experience, without requiring humans to program their behavior explicitly. Equivalently: a set of methods that automatically detect patterns in data, use the uncovered patterns for prediction or decision making.

1.1.3 Course Organization

The course is organized into five parts:

1. **Part 1: Foundation of Machine Learning** — Linear regression, Logistic regression, Bias-Variance Decomposition
2. **Part 2: Foundation of Deep Learning** — Feedforward neural networks, Fundamental Issues in Deep Learning, Convolutional neural networks, Recurrent neural networks
3. **Part 3: Selected Advanced Topics** — Transformers, BERT, LLM, ViT, VLM, Variational Autoencoders, Generative Adversarial Networks
4. **Part 4: Classical Methods** — Support vector machines, Decision trees and random forests
5. **Part 5: Diffusion Models** — Introduction to DDPM, excluded from the final exam

1.1.4 Course Outcomes

By the end of this course, students should be able to:

- Understand issues involved in learning from data and the major types of machine learning tasks
- Explain the principles underlying a variety of machine learning algorithms

- Apply a variety of machine learning algorithms to data
- Evaluate and compare the performance of different machine learning algorithms according to common performance criteria

1.2 Linear Regression

1.2.1 Linear Regression: Problem Statement

Setup

Given a training set $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$, where each $\mathbf{x}_i \in \mathbb{R}^D$ is a column vector and each $y_i \in \mathbb{R}$.
We want to learn a linear model:

$$y = f(\mathbf{x}, \mathbf{w}) = \mathbf{w}^\top \mathbf{x} = \sum_{j=0}^{D} w_j x_j$$

where $\mathbf{w} = (w_0, w_1, \ldots, w_D)^\top$ are the weights. We always set $x_0 = 1$ so that w_0 acts as the bias term (often denoted b). Each weight w_j determines how important the feature x_j is in predicting the response y.

1.2.2 Mean Squared Error

Objective: Minimize Prediction Error

We want the predicted values $f(\mathbf{x}_i)$ to be close to the observed values y_i. We minimize the **Mean Squared Error (MSE)**:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \mathbf{w}^\top \mathbf{x}_i\right)^2$$

MSE is the average of the squared differences between predictions and true values. Squaring ensures the loss is always non-negative and penalizes large errors more.

MSE is a quadratic "bowl" function of $\mathbf{w}$ with a unique global minimum. We find it by setting the gradient to zero.

1.2.3 Matrix Representation

Represent the full training set using the **design matrix** X and a column vector $\mathbf{y}$:

$$X = \begin{pmatrix} x_{1,0} & x_{1,1} & \cdots & x_{1,D} \\ x_{2,0} & x_{2,1} & \cdots & x_{2,D} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N,0} & x_{N,1} & \cdots & x_{N,D} \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_N^\top \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

The MSE can then be written compactly as:

$$L(\mathbf{w}) = \frac{1}{N} \|\mathbf{y} - X\mathbf{w}\|_2^2 = \frac{1}{N} (\mathbf{y} - X\mathbf{w})^\top (\mathbf{y} - X\mathbf{w})$$

Expanding:

$$L(\mathbf{w}) = \frac{1}{N} \left(\mathbf{w}^\top X^\top X \mathbf{w} - 2\mathbf{w}^\top X^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}\right)$$

1.2.4 Gradient Derivation and the Normal Equation

To find the minimum, compute the gradient of $L(\mathbf{w})$ w.r.t. $\mathbf{w}$ and set it to zero. Using the identities $\frac{\partial \mathbf{w}^\top A\mathbf{w}}{\partial \mathbf{w}} = (A + A^\top)\mathbf{w}$ and $\frac{\partial \mathbf{b}^\top \mathbf{w}}{\partial \mathbf{w}} = \mathbf{b}$:

$$\nabla L(\mathbf{w}) = \frac{1}{N}\left((X^\top X + (X^\top X)^\top)\mathbf{w} - 2X^\top \mathbf{y}\right) = \frac{2}{N}\left(X^\top X\mathbf{w} - X^\top \mathbf{y}\right)$$

Normal Equation

Setting $\nabla L(\mathbf{w}) = \mathbf{0}$ gives the **normal equation**:

$$X^\top X\mathbf{w} = X^\top \mathbf{y}$$

The minimizer is:

$$\boxed{\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}}$$

This is called the **Ordinary Least Squares (OLS)** solution. Note that $X^\top X$ is $(D+1)\times(D+1)$, so its inverse exists only when the features are linearly independent.

This is one of the rare cases in machine learning where the optimal solution can be written in closed form.

1.2.5 Beyond Linear Regression: Feature Mapping

Linear regression can model non-linear relationships by replacing $\mathbf{x}$ with a non-linear function $\phi(\mathbf{x})$:

$$y = f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x})$$

This is known as **basis function expansion**, and ϕ is called a **feature mapping**. The model is still linear in $\mathbf{w}$ after the mapping, so the normal equation still applies.

Polynomial Regression For $D = 1$ input and degree-d polynomial:

$$\phi(x) = [1, x, x^2, \ldots, x^d]^\top$$

For $D = 2$ input with degree $d = 2$:

$$\phi(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]^\top$$

In general, $\phi(\mathbf{x})$ includes all monomials up to degree d.

1.2.6 Model Capacity, Overfitting and Underfitting

Hypothesis Space and Capacity

The **hypothesis space** $\mathcal{H}$ of a learning algorithm is the set of functions it is allowed to select as the solution. The "size" or expressiveness of $\mathcal{H}$ is called the **capacity** of the model. For polynomial regression, the larger the degree d, the higher the model capacity.

Generalization We care about performance on new, unseen examples — this is called **generalization**. We use the error on a held-out **test set** to measure it:

$$L_{\text{test}}(\hat{\mathbf{w}}) = \frac{1}{N_{\text{test}}}\|\mathbf{y}_{\text{test}} - X_{\text{test}}\hat{\mathbf{w}}\|_2^2$$

The **training error** is $L_{\text{train}} = \frac{1}{N_{\text{train}}}\|\mathbf{y}_{\text{train}} - X_{\text{train}}\hat{\mathbf{w}}\|_2^2$.

Key insight: small training error does **not** imply small generalization error. The generalization gap $L_{\text{test}} - L_{\text{train}}$ arises because parameters are selected to minimize training error, which may also fit the noise in the training data.

Overfitting and Underfitting

Bias–Variance Trade-off

As model capacity increases:
- At low capacity: both training and generalization error are high — **underfitting** (model is too simple, high bias)
- At some optimal capacity: generalization error reaches its minimum
- At high capacity: training error continues to decrease (can reach zero), but generalization error increases — **overfitting** (model is too flexible, high variance)

Choosing appropriate capacity is achieved by **validation** or **regularization**.

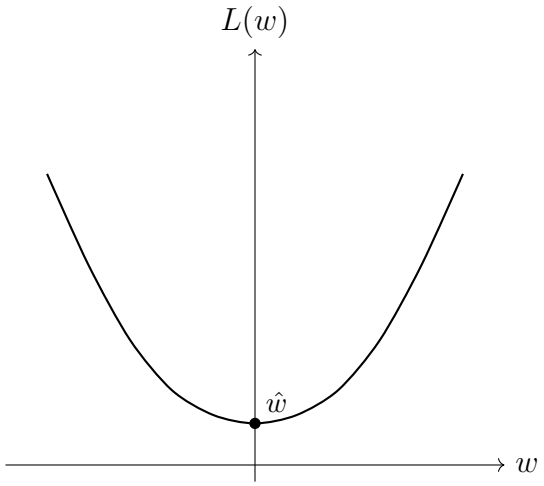


Figure 1: MSE loss bowl: quadratic in $\mathbf{w}$, unique global minimum at $\hat{\mathbf{w}}$.

1.2.7 Model Selection

Validation Hyperparameters (e.g., polynomial degree d, regularization strength λ) are determined by **validation**:

1. Randomly divide the labeled training set into two disjoint subsets: a smaller training set and a validation set (held-out set), typically withholding 20% of examples
2. For each candidate hyperparameter value, train on the training set and measure error on the validation set (**validation error**)
3. Pick the hyperparameter with minimum validation error

Cross-Validation When labeled data is limited, **k-fold cross-validation** makes better use of all data:

1. Partition the N examples into k disjoint subsets (folds) of approximately equal size
2. Run learning k times, each time using one fold as validation and the remaining $k-1$ folds as training
3. Average validation performance over the k runs; this is the cross-validation estimate of generalization error
4. Typical choice: $k = 10$

1.2.8 Regularization

Instead of picking a small hypothesis space, start with a large one and use regularization to pick an appropriate solution. Regularization penalizes complex models by adding a term to the loss function.

L2 Regularization (Ridge Regression)

The penalized error function:

$$L(\mathbf{w}, w_0) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - (w_0 + \mathbf{w}^\top \phi(\mathbf{x}_i))\right)^2 + \lambda\|\mathbf{w}\|_2^2$$

where $\lambda \geq 0$ controls regularization strength, and w_0 is **not** regularized (it does not influence model complexity; regularizing it would depend on the choice of origin of $\mathbf{x}$). The **ridge regression** solution has a closed form:

$$\boxed{\hat{\mathbf{w}}_{\text{ridge}} = (\lambda I_K + X^\top X)^{-1} X^\top \mathbf{y}}$$

Compare with OLS: $\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}$. L2 regularization shrinks weights toward zero, reducing variance at the cost of potentially slightly increased bias.

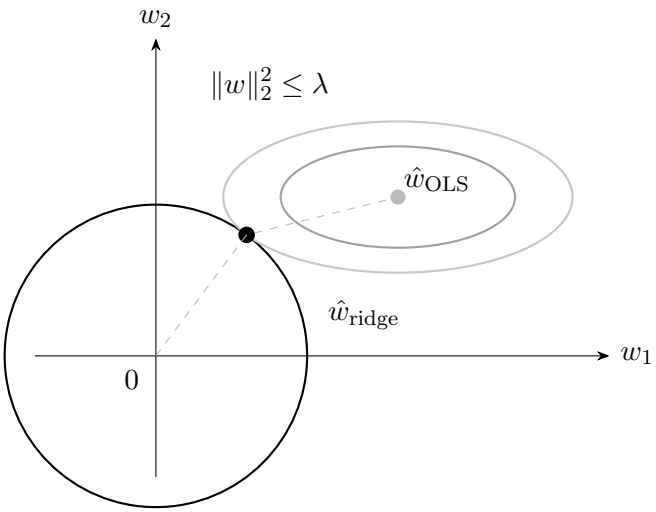


Figure 2: Ridge regression: the L2 constraint is a circle centered at the origin; the quadratic loss has elliptical level sets centered at the OLS solution $\hat{w}_{\text{OLS}}$. The ridge solution $\hat{w}_{\text{ridge}}$ is their tangent point — it shrinks OLS toward the origin while keeping both features non-zero.

L1 Regularization (LASSO)

LASSO (Least Absolute Shrinkage and Selection Operator) uses L1 regularization:

$$L(\mathbf{w}, w_0) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - (w_0 + \mathbf{w}^\top \phi(\mathbf{x}_i))\right)^2 + \lambda\|\mathbf{w}\|_1$$

L1 regularization forces certain weights to exactly zero, yielding a sparse model that uses only a subset of features. Geometrically, the level sets of L1 are diamonds while those of L2 are circles; when intersecting with the ellipses of the quadratic loss, L1 minima often land on the axes where weights = 0. Unlike ridge regression, LASSO does not have a closed-form solution; it is solved with coordinate descent or LARS.

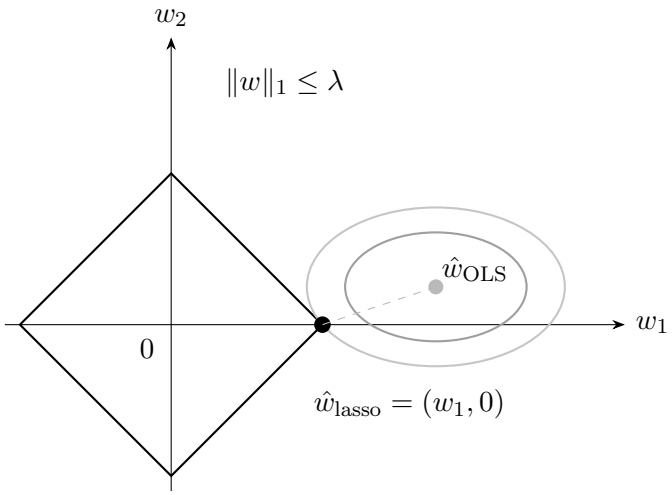


Figure 3: LASSO: the L1 constraint is a diamond centered at the origin, with vertices on the axes. The loss ellipse is tangent to the diamond exactly at a vertex on an axis, so one weight becomes exactly zero — this is how L1 regularization performs automatic feature selection.

1.2.9 Performance Metrics for Regression

- **Mean Squared Error (MSE)**: $\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$. Applicable to train/validation/test sets.
- **Root Mean Squared Error (RMSE)**: $\sqrt{\text{MSE}}$. Brings error back to the original units of y (e.g., meters instead of m^2).
- **Mean Absolute Error (MAE)**: $\frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$. Less sensitive to outliers than MSE.
- **Coefficient of Determination (R^2)**:

$$R^2 = 1 - \frac{\text{MSE}}{\text{variance of } y} = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}$$

  Best possible $R^2 = 1$ (perfect prediction). A model always predicting $\bar{y}$ gives $R^2 = 0$. Negative values are possible if the model is worse than the constant predictor.

1.2.10 Probabilistic Formulation of Linear Regression

We can derive least squares from a probabilistic model. Assume:

$$y = \mathbf{w}^\top \mathbf{x} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

where ϵ is Gaussian noise. Then the conditional distribution is:

$$p(y \mid \mathbf{x}, \theta) = \mathcal{N}\big(y \mid \mu(\mathbf{x}), \sigma^2\big), \quad \mu(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$$

Parameter estimation by minimizing **cross entropy** (equivalent to maximizing **likelihood**):

$$-\frac{1}{N}\sum_{i=1}^{N}\log p(y_i \mid \mathbf{x}_i, \theta) = \frac{1}{2}\log(2\pi\sigma^2) + \frac{1}{2N\sigma^2}\sum_{i=1}^{N}(y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

Assuming σ fixed, minimizing cross entropy reduces exactly to minimizing MSE.

Summary

Under additive Gaussian noise assumptions, the method of least squares is equivalent to maximum likelihood estimation. This probabilistic viewpoint also allows natural extensions (e.g., Bayesian linear regression, heteroscedastic noise).

1.3 Logistic Regression and Gradient Descent

1.3.1 Logistic Regression: Problem Statement

Setup

Given training set $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$, where $y_i \in \{0, 1\}$. We want to model the **conditional probability distribution** of the label y given the input $\mathbf{x}$:

$$p(y \mid \mathbf{x}, \mathbf{w})$$

This is fundamentally different from linear regression where we directly predicted y. Here we model the full probabilistic relationship.

The Sigmoid (Logistic) Function

$$\boxed{\sigma(z) = \frac{1}{1 + \exp(-z)} = \frac{e^z}{e^z + 1}}$$

This function maps the entire real line $\mathbb{R}$ into $[0, 1]$, making it ideal for representing probabilities. Important property:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

The Logistic Regression Model

Probabilistic Model

$$p(y \mid \mathbf{x}, \mathbf{w}) = \mathrm{Ber}\big(y \mid \sigma(\mathbf{w}^\top \mathbf{x})\big) \quad \text{or explicitly} \quad p(y = 1 \mid \mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x})$$

$$p(y = 1 \mid \mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}$$

$$p(y = 0 \mid \mathbf{x}, \mathbf{w}) = 1 - \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{\exp(-\mathbf{w}^\top \mathbf{x})}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}$$

The **logit** (log-odds) of $p(y = 1 \mid \mathbf{x}, \mathbf{w})$ is linear in $\mathbf{x}$:

$$\log \frac{p(y = 1 \mid \mathbf{x}, \mathbf{w})}{p(y = 0 \mid \mathbf{x}, \mathbf{w})} = \log \exp(\mathbf{w}^\top \mathbf{x}) = \mathbf{w}^\top \mathbf{x}$$

This explains the name "logistic regression" — it is a linear regression on the logit transform of the probability.

Decision Rule To classify, we use the **optimal Bayes classifier**:

$$\hat{y} = \arg\max_y p(y \mid \mathbf{x}, \mathbf{w})$$

which simplifies to:

$$\hat{y} = 1 \iff p(y = 1 \mid \mathbf{x}, \mathbf{w}) > 0.5 \iff \mathbf{w}^\top \mathbf{x} > 0$$

Why is it called a "Bayes" classifier?

The decision rule $p(y = 1 \mid \mathbf{x}) > 0.5$ minimizes the probability of mistakes when the costs of false positive and false negative are equal. This is the decision rule that a hypothetical "perfectly informed" Bayesian would use. In practice, we estimate $p(y \mid \mathbf{x})$ from training data, but the decision rule form is the same.

Logistic regression is a **linear classifier**: its decision boundary $\mathbf{w}^\top \mathbf{x} = 0$ is a hyperplane in the feature space.

1.3.2 Entropy, Cross Entropy, and KL Divergence

Entropy

The entropy of a discrete random variable X with distribution P:

$$H(X) = \sum_X P(X) \log \frac{1}{P(X)} = -\mathbb{E}_P[\log P(X)]$$

Convention: $0 \cdot \log(1/0) = 0$. Base-2 logarithm gives units of bits. Entropy measures uncertainty: the more uniform the distribution, the higher the entropy. Maximum entropy for K outcomes is $\log K$ (uniform distribution).
For continuous variables, replace the sum with an integral: $H(X) = -\int p(x) \log p(x) dx$.

Kullback–Leibler Divergence

The KL divergence from Q to P (also called relative entropy):

$$D_{\mathrm{KL}}(P \| Q) = \sum_X P(X) \log \frac{P(X)}{Q(X)}$$

It measures how much distribution Q "diverges" from the true distribution P. By Gibbs' inequality, $D_{\mathrm{KL}}(P \| Q) \geq 0$, with equality iff $P = Q$. Note that KL divergence is not symmetric: $D_{\mathrm{KL}}(P \| Q) \neq D_{\mathrm{KL}}(Q \| P)$.

Gibbs' Inequality Proof via Jensen's Inequality

The function $f(x) = -\log x$ is convex on $(0, \infty)$ (its second derivative $f''(x) = 1/x^2 > 0$ for all $x > 0$). Jensen's inequality states that for a convex function f,

$$f(\mathbb{E}_P[X]) \leq \mathbb{E}_P[f(X)]$$

Applying this with $X = Q(x)/P(x)$ (a random variable distributed according to P):

$$f\left(\sum_x P(x) \cdot \frac{Q(x)}{P(x)}\right) \leq \sum_x P(x) \cdot f\left(\frac{Q(x)}{P(x)}\right)$$

The left side evaluates to $f(1) = -\log 1 = 0$. Substituting $f(t) = -\log t$ on the right:

$$0 \leq \sum_x P(x) \cdot \left(-\log \frac{Q(x)}{P(x)}\right) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = D_{\mathrm{KL}}(P \| Q)$$

So $D_{\mathrm{KL}}(P\|Q) \geq 0$, with equality iff $Q = P$ (since Jensen's inequality is tight iff the random variable is constant, i.e., $Q(x)/P(x) = 1$ for all x).

Cross Entropy

Using the identity $\log \frac{P}{Q} = \log \frac{1}{Q} - \log \frac{1}{P}$, we get:

$$D_{\mathrm{KL}}(P\|Q) = \underbrace{-\sum_X P(X) \log Q(X)}_{H(P,Q) \text{ (cross entropy)}} - \underbrace{\left(-\sum_X P(X) \log P(X)\right)}_{H(P) \text{ (true entropy)}}$$

The **cross entropy** is:

$$H(P, Q) = -\sum_X P(X) \log Q(X) = D_{\mathrm{KL}}(P\|Q) + H(P)$$

Since $H(P)$ is constant (determined by the true distribution), minimizing cross entropy $H(P, Q)$ over Q is equivalent to minimizing the KL divergence. Cross entropy is minimized when $Q = P$, at which point $H(P, Q) = H(P)$.

1.3.3 Cross Entropy for Supervised Learning

In supervised learning, the true distribution is $P(\mathbf{x}, y)$ and our model gives $Q(y \mid \mathbf{x})$. The **conditional cross entropy** between $P(\mathbf{x}, y)$ and $Q(y \mid \mathbf{x})$ is:

$$H(P, Q) = -\int P(\mathbf{x}, y) \log Q(y \mid \mathbf{x}) \, d\mathbf{x} \, dy$$

Approximating with the empirical distribution from the training set:

$$H(P, Q) \approx -\frac{1}{N} \sum_{i=1}^{N} \log Q(y_i \mid \mathbf{x}_i)$$

This is exactly the **negative log-likelihood (NLL)** of the model on the training data.

1.3.4 Likelihood and Maximum Likelihood Estimation

Likelihood

The **likelihood** of parameters θ given data $\mathcal{D}$ is:

$$L(\theta \mid \mathcal{D}) = P(\mathcal{D} \mid \theta)$$

Under the i.i.d. assumption, the likelihood factors as:

$$L(\theta \mid \mathcal{D}) = \prod_{i=1}^{N} P(\mathbf{x}_i, y_i \mid \theta)$$

Maximum Likelihood Estimation (MLE) chooses the θ that maximizes the likelihood:

$$\theta^* = \arg\max_\theta L(\theta \mid \mathcal{D})$$

Maximizing L is equivalent to maximizing the **log-likelihood** $\log L$ (which is easier since products become sums).

MLE is intuitive: it finds the parameters that "best explain" the observed data. It is also statistically consistent: the MLE converges in probability to the true parameter value as the sample size goes to infinity.

1.3.5 Cross Entropy Loss for Logistic Regression

For a single example $(\mathbf{x}, y)$ with model prediction $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x})$:

$$\log p(y \mid \mathbf{x}, \mathbf{w}) = y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

This is the log probability under the Bernoulli model. The cross-entropy loss for the full dataset is:

$$\boxed{\begin{aligned} L(\mathbf{w}) &= -\frac{1}{N} \sum_{i=1}^{N} \log p(y_i \mid \mathbf{x}_i, \mathbf{w}) \\ &= -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right], \quad \hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i) \end{aligned}}$$

Why No Closed Form?

Unlike linear regression, cross-entropy loss for logistic regression does not have a closed-form minimizer. The sigmoid function makes the optimization problem non-linear, so we must resort to gradient-based optimization.

1.3.6 Gradient Descent

Update Rule

To minimize a loss function $L(\mathbf{w})$, **gradient descent** iterates:

$$\boxed{\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla L(\mathbf{w}^{(t)})}$$

where $\eta > 0$ is the **learning rate** (also called step size). The gradient points in the direction of steepest ascent; we subtract it to move downhill.

The learning rate is a critical hyperparameter: too small $\Rightarrow$ slow convergence; too large $\Rightarrow$ may diverge or oscillate. Gradient descent can get stuck at local minima or saddle points, but in practice usually works well for well-behaved losses.

Gradient of the Sigmoid

$$\sigma'(z) = \frac{d}{dz} \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z))$$

Cross-Entropy Gradient Derivation

Let $z = \mathbf{w}^\top \mathbf{x} + b$, $p = \sigma(z) = 1/(1 + e^{-z})$, loss $L = -[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))]$. Using chain rule:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w_j} = \frac{\partial L}{\partial z} \cdot x_j$$

Now compute $\partial L/\partial z$. Note $\partial\sigma/\partial z = \sigma(1-\sigma) = p(1-p)$:

$$\frac{\partial L}{\partial z} = -\left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)}\right) \cdot \sigma(z)(1-\sigma(z))$$

$$= -\left(y(1-\sigma(z)) - (1-y)\sigma(z)\right) = -\left(y - y\sigma(z) - \sigma(z) + y\sigma(z)\right) = \sigma(z) - y$$

So the final gradient is:

$$\boxed{\frac{\partial L}{\partial w_j} = \left(\sigma(\mathbf{w}^\top \mathbf{x} + b) - y\right) x_j}$$

Gradient of Cross Entropy for Logistic Regression Let $z_i = \mathbf{w}^\top \mathbf{x}_i$, $\hat{y}_i = \sigma(z_i)$. For a single example $(\mathbf{x}, y)$:

$$\frac{\partial \log p(y \mid \mathbf{x}, \mathbf{w})}{\partial w_j} = [y - \sigma(\mathbf{w}^\top \mathbf{x})] x_j$$

Summing over all examples and accounting for the negative sign and $1/N$ factor:

Gradient for Logistic Regression

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \left(\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i\right) \mathbf{x}_i$$

Elegant interpretation: the gradient is **prediction error times input**.

1.3.7 Stochastic Gradient Descent and Variants

- **Batch Gradient Descent**: use all N examples per update. Accurate gradient but slow when N is large.
- **Stochastic Gradient Descent (SGD)**: use one randomly picked example per update. Very fast per step; noisy gradient but often works surprisingly well for large datasets.
- **Mini-batch Gradient Descent**: use a random subset B of examples (the "mini-batch", with $|B| \ll N$). Compromise: less noisy than pure SGD, faster than batch. This is the default choice in modern deep learning.

Update rule with L2 regularization (repeats weight decay):

$$\mathbf{w} \leftarrow (1 - \alpha\lambda)\mathbf{w} + \frac{\alpha}{|B|} \sum_{i \in B} \left(\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i\right) \mathbf{x}_i$$

1.3.8 Softmax Regression (Multi-class)

For K classes ($C \geq 2$), replace sigmoid with the **softmax** function:

$$\text{softmax}(\mathbf{z})_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}$$

The model for each class has its own weight vector $\mathbf{w}_c$:

Softmax Regression Model

$$p(y = c \mid \mathbf{x}, W) = \frac{\exp(\mathbf{w}_c^\top \mathbf{x})}{\sum_{c'=1}^{K} \exp(\mathbf{w}_{c'}^\top \mathbf{x})}$$

where $W = (\mathbf{w}_1, \ldots, \mathbf{w}_K)$ is a $(D+1) \times K$ weight matrix. Decision rule: $\hat{y} = \arg\max_c p(y = c \mid \mathbf{x}, W)$.

Relation to Logistic Regression

When $K = 2$, softmax regression reduces exactly to logistic regression. Divide numerator and denominator by $\exp(\mathbf{w}_1^\top \mathbf{x})$, let $\mathbf{w} = \mathbf{w}_2 - \mathbf{w}_1$, and you recover the sigmoid form.

Cross Entropy Loss for Softmax Regression

$$L(W) = -\frac{1}{N}\sum_{i=1}^{N}\left[\mathbf{1}(y_i = c)\log p(y_i = c \mid \mathbf{x}_i, W)\right] = -\frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{w}_{y_i}^\top \mathbf{x}_i - \log\sum_{c=1}^{K}\exp(\mathbf{w}_c^\top \mathbf{x}_i)\right)$$

The quantity $\log\sum_c \exp(\mathbf{w}_c^\top \mathbf{x})$ is the **log-sum-exp** (LSE), a smooth approximation to the maximum function. The loss is small when the true class's logit is the (soft) maximum.

Gradient of Softmax Loss

Gradient for Softmax Regression

$$\boxed{\nabla_{\mathbf{w}_c} L(W) = \frac{1}{N}\sum_{i=1}^{N}\left(p(y = c \mid \mathbf{x}_i, W) - \mathbf{1}(y_i = c)\right)\mathbf{x}_i}$$

Compare with logistic regression gradient: $\left(\sigma(\mathbf{w}^\top \mathbf{x}_i) - y_i\right)\mathbf{x}_i$. Same elegant form: prediction minus actual indicator times input.

Softmax Gradient Derivation

Let $z_c = \mathbf{w}_c^\top \mathbf{x}$ and $p_c = \exp(z_c)/\sum_{c'}\exp(z_{c'})$. We first derive the Jacobian of the softmax function, i.e., $\partial p_c/\partial z_{c'}$. Write $p_c = e^{z_c}/Z$ where $Z = \sum_{c'} e^{z_{c'}}$.

For $c' = c$:

$$\frac{\partial p_c}{\partial z_c} = \frac{e^{z_c}\cdot Z - e^{z_c}\cdot e^{z_c}}{Z^2} = \frac{e^{z_c}(Z - e^{z_c})}{Z^2} = \frac{e^{z_c}}{Z}\cdot\frac{Z - e^{z_c}}{Z} = p_c(1 - p_c)$$

For $c' \neq c$:

$$\frac{\partial p_c}{\partial z_{c'}} = \frac{0\cdot Z - e^{z_c}\cdot e^{z_{c'}}}{Z^2} = -\frac{e^{z_c}e^{z_{c'}}}{Z^2} = -p_c p_{c'}$$

So the full Jacobian is:

$$\frac{\partial p_c}{\partial z_{c'}} = p_c(\delta_{c,c'} - p_{c'})$$

where $\delta_{c,c'} = \mathbf{1}(c = c')$.

Now the cross-entropy loss for a single example is $L = -\mathbf{1}(y = c)\log p_c$. By chain rule:

$$\frac{\partial L}{\partial z_{c'}} = \frac{\partial L}{\partial p_{c'}}\cdot\frac{\partial p_{c'}}{\partial z_{c'}} = -\frac{\mathbf{1}(y = c')}{p_{c'}}\cdot p_{c'}(1 - p_{c'}) = -(1 - p_{c'})\cdot\mathbf{1}(y = c')$$

Wait — we need the gradient with respect to one specific $\mathbf{w}_c$. Use chain rule through

$z_c = \mathbf{w}_c^\top \mathbf{x}$:

$$\frac{\partial L}{\partial \mathbf{w}_c} = \frac{\partial L}{\partial z_c} \cdot \frac{\partial z_c}{\partial \mathbf{w}_c} = \left(\frac{\partial L}{\partial p_c} \cdot \frac{\partial p_c}{\partial z_c} + \sum_{c' \neq c} \frac{\partial L}{\partial p_{c'}} \cdot \frac{\partial p_{c'}}{\partial z_c} \right) \cdot \mathbf{x}$$

Since the loss only involves the true class: $L = -\log p_y$ (where y is the true class), so $\partial L / \partial p_{c'} = -\mathbf{1}(y = c')/p_{c'}$. Substituting:

$$\frac{\partial L}{\partial \mathbf{w}_c} = \left(-\frac{\mathbf{1}(y = c)}{p_c} \cdot p_c(1 - p_c) + \sum_{c' \neq c} -\frac{\mathbf{1}(y = c')}{p_{c'}} \cdot (-p_{c'} p_c) \right) \mathbf{x}$$

If $c = y$: $\mathbf{1}(y = c) = 1$, all other terms have $\mathbf{1}(y = c') = 0$, giving $-(1 - p_c)\mathbf{x} = (p_c - 1)\mathbf{x} = (p_c - \mathbf{1}(y = c))\mathbf{x}$. If $c \neq y$: the first term vanishes ($\mathbf{1}(y = c) = 0$), and the only non-zero summand is when $c' = y$, giving $(-1/p_y)(-p_y p_c)\mathbf{x} = p_c \mathbf{x} = (p_c - 0)\mathbf{x} = (p_c - \mathbf{1}(y = c))\mathbf{x}$. In both cases:

$$\boxed{\frac{\partial L}{\partial \mathbf{w}_c} = (p_c - \mathbf{1}(y = c))\mathbf{x}}$$

1.3.9 Performance Metrics for Classification

1. **Confusion Matrix**: for binary classification, count TP, FN, FP, TN. For multi-class, a $C \times C$ matrix with rows = actual, columns = predicted.
2. **Accuracy**: $\dfrac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = \dfrac{\text{correct}}{\text{all}}$. Misleadingly high for imbalanced datasets.
3. **Precision**: $\dfrac{\text{TP}}{\text{TP} + \text{FP}}$. "Of all positives predicted, how many are truly positive?"
4. **Recall** (Sensitivity, True Positive Rate): $\dfrac{\text{TP}}{\text{TP} + \text{FN}}$. "Of all actual positives, how many did we find?"
5. **F1-score**: harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} = \frac{1}{\frac{1}{2}\left(\frac{1}{P} + \frac{1}{R}\right)}$$

6. **PR Curve**: precision vs. recall at varying thresholds; area under it is F_1 averaged over thresholds.
7. **ROC Curve**: TPR (recall) vs. FPR at varying thresholds; area under ROC = **AUC**, measuring overall ranking quality.

1.4 Other Classification Approaches

Overview

Classification methods fall broadly into two categories: **optimization-based** (directly minimize a loss on predictions) and **generative** (model the full data distribution). We cover both.

1.4.1 Optimization Approach: Empirical Risk Minimization

The **optimization approach** directly minimizes a loss function on the training set. For binary classification with $y_i \in \{-1, +1\}$, the model is:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

The most natural loss is the **zero-one loss**:

$$L_{0/1}(y, z) = \mathbf{1}[yz \leq 0], \quad z = \mathbf{w}^\top \mathbf{x} + b$$

where $\mathbf{1}[\cdot]$ is the indicator function. The zero-one loss is 1 when the prediction is wrong, 0 when right. The empirical risk minimization (ERM) objective is:

$$\min_{\mathbf{w},b} \frac{1}{N} \sum_{i=1}^{N} L_{0/1}(y_i, \mathbf{w}^\top \mathbf{x}_i + b)$$

Problem with Zero-One Loss

The zero-one loss is not differentiable (it has zero gradient almost everywhere, with no well-defined gradient at the discontinuity where $yz = 0$). This prevents direct optimization via gradient descent. We need a **surrogate loss**: a smooth, convex function that upper bounds the zero-one loss. This guarantee ensures that minimizing the surrogate also pushes the zero-one loss down.

Convex Function A function f is **convex** if for any two points $x_1 < x_2$ and any $t \in [0, 1]$:

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$$

Convex functions have the important property that any local minimum is a global minimum — making them easy to optimize.

Common Surrogate Losses

Zero-one:	$L_{0/1}(y, z) = \mathbf{1}[yz \leq 0]$
Logistic:	$L_{\log}(y, z) = \log_2\left(1 + e^{-yz}\right)$
Hinge (SVM):	$L_{\text{hinge}}(y, z) = \max\{0, 1 - yz\}$
Exponential (Boosting):	$L_{\exp}(y, z) = e^{-yz}$

Connection to Logistic Regression

The logistic surrogate loss is exactly what we minimized in logistic regression (up to a constant). So logistic regression can be viewed as: linear classifier trained with logistic surrogate loss. This unifies the probabilistic and optimization viewpoints.

Loss vs. Error When a surrogate loss is used, we distinguish between:

- **Loss**: the value of the surrogate loss function (e.g., cross-entropy) on a dataset
- **Error**: the zero-one classification error (fraction misclassified)

Loss is what we optimize during training; error is what we ultimately care about for model quality.

1.4.2 Generative Approach: Naive Bayes and LDA

Generative models take a different route: instead of directly modeling $p(y \mid \mathbf{x})$, they model $p(\mathbf{x} \mid y)$ and $p(y)$, then use Bayes rule:

$$p(y = k \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid y = k)p(y = k)}{\sum_{c=1}^{K} p(\mathbf{x} \mid y = c)p(y = c)}$$

Linear Discriminant Analysis (LDA) LDA assumes each class density is Gaussian with **shared covariance**:

$$p(\mathbf{x} \mid y = k) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \Sigma)$$

Under this assumption, the decision boundary becomes linear, just like logistic regression. The parameters $\boldsymbol{\mu}_k$ and Σ are estimated from the sample means and pooled covariance of the training data.

Why LDA Produces Linear Decision Boundaries

By Bayes' rule, the posterior is:

$$p(y = k \mid \mathbf{x}) \propto p(\mathbf{x} \mid y = k)\, p(y = k)$$

Taking the logarithm (which is monotone, so preserves argmax):

$$\log p(y = k \mid \mathbf{x}) = \log p(\mathbf{x} \mid y = k) + \log p(y = k) + C$$

Substitute the shared-covariance Gaussian density $p(\mathbf{x} \mid y = k) = (2\pi)^{-D/2}|\Sigma|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right)$:

$$\log p(\mathbf{x} \mid y = k) = -\frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\Sigma| - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)$$

Expanding the quadratic term:

$$(\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_k) = \mathbf{x}^\top \Sigma^{-1}\mathbf{x} - \boldsymbol{\mu}_k^\top \Sigma^{-1}\mathbf{x} - \mathbf{x}^\top \Sigma^{-1}\boldsymbol{\mu}_k + \boldsymbol{\mu}_k^\top \Sigma^{-1}\boldsymbol{\mu}_k$$

Since Σ^{-1} is symmetric, $\boldsymbol{\mu}_k^\top \Sigma^{-1}\mathbf{x} = \mathbf{x}^\top \Sigma^{-1}\boldsymbol{\mu}_k = \mathbf{x}^\top \Sigma^{-1}\boldsymbol{\mu}_k$. So substituting back into the log posterior:

$$\log p(y = k \mid \mathbf{x}) = \underbrace{-\frac{1}{2}\mathbf{x}^\top \Sigma^{-1}\mathbf{x}}_{\text{same for all } k} + \boldsymbol{\mu}_k^\top \Sigma^{-1}\mathbf{x} - \frac{1}{2}\boldsymbol{\mu}_k^\top \Sigma^{-1}\boldsymbol{\mu}_k + \log p(y = k) + C'$$

The term $-\frac{1}{2}\mathbf{x}^\top \Sigma^{-1}\mathbf{x}$ and the constant C' are identical for every class k, so they drop out when comparing posteriors across classes. What remains:

$$\boxed{\delta_k(\mathbf{x}) = \boldsymbol{\mu}_k^\top \Sigma^{-1}\mathbf{x} - \frac{1}{2}\boldsymbol{\mu}_k^\top \Sigma^{-1}\boldsymbol{\mu}_k + \log p(y = k)}$$

This is the linear discriminant function, which is linear in $\mathbf{x}$. The decision boundary between any two classes k and c is where $\delta_k(\mathbf{x}) = \delta_c(\mathbf{x})$, which is a hyperplane in $\mathbf{x}$ — hence the name Linear Discriminant Analysis.

Comparison of Logistic Regression and LDA

Both models produce linear decision boundaries. The difference:

- **Logistic regression**: a discriminative model, directly learns $p(y \mid \mathbf{x})$. Often more accurate for predictions.
- **LDA**: a generative model, makes stronger distributional assumptions. Can be more robust with limited data.

Naive Bayes The "naive" assumption: **features are conditionally independent** given the class:

$$p(\mathbf{x} \mid y = k) = \prod_{j=1}^{D} p(x_j \mid y = k)$$

This drastically simplifies the model while still performing surprisingly well, especially for text classification with bag-of-words features (where multinomial Naive Bayes is a baseline).

1.4.3 Discriminative vs Generative: Summary

- **Discriminative**: directly estimate $p(y \mid \mathbf{x})$ (logistic regression, softmax, SVM, perceptron). Often more accurate for predictions.
- **Generative**: estimate $p(\mathbf{x} \mid y)$ and $p(y)$; derive $p(y \mid \mathbf{x})$ via Bayes (LDA, Naive Bayes). Better with limited labeled data, handles missing features naturally.

1.5 The Bias–Variance Decomposition

Motivation

Earlier we observed that training error always decreases with model capacity, but generalization error first decreases then increases. The **bias–variance decomposition** gives a quantitative explanation by breaking the expected generalization error into three fundamentally different sources. It is derived here for regression with squared loss, but the bias–variance trade-off intuition extends to classification as well.

1.5.1 Setup and Notation

Let $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$ be a training set sampled i.i.d. from the true population distribution $(\mathbf{x}, y) \sim \mathcal{D}$. A learning algorithm produces a hypothesis h_S that depends on the random training set S.

- **Training (empirical) error**: $\hat{\epsilon}(h_S) = \dfrac{1}{m}\sum_{i=1}^{m}(y_i - h_S(\mathbf{x}_i))^2$
- **Generalization error**: $\epsilon(h_S) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}[(y - h_S(\mathbf{x}))^2]$
- **Expected generalization error (over random training sets)**: $\epsilon = \mathbb{E}_S[\epsilon(h_S)]$

  The **generalization gap** is $\epsilon(h_S) - \hat{\epsilon}(h_S)$.

1.5.2 Full Derivation

We derive the decomposition for a fixed input point $\mathbf{x}$, then average over all $\mathbf{x}$:

$$\begin{aligned}
\mathbb{E}_S\left[(h_S(\mathbf{x}) - y)^2\right] &= \mathbb{E}_S\left[(h_S(\mathbf{x}) - \mathbb{E}_S[h_S(\mathbf{x})] + \mathbb{E}_S[h_S(\mathbf{x})] - y)^2\right] \\
&= \underbrace{\mathbb{E}_S\left[(h_S(\mathbf{x}) - \mathbb{E}_S[h_S(\mathbf{x})])^2\right]}_{\text{Variance}} \\
&\quad + \underbrace{(\mathbb{E}_S[h_S(\mathbf{x})] - \mathbb{E}[y \mid \mathbf{x}])^2}_{\text{Bias}^2} \\
&\quad + \underbrace{\mathbb{E}\left[(y - \mathbb{E}[y \mid \mathbf{x}])^2\right]}_{\text{Noise}}
\end{aligned}$$

The cross term vanishes because $\mathbb{E}_S[(h_S - \mathbb{E}_S[h_S])] = 0$ and the noise term is independent of S. Averaging over $\mathbf{x}$:

Bias–Variance Decomposition

$$\boxed{\mathbb{E}_S[(h_S(\mathbf{x}) - y)^2] = \text{Bias}^2 + \text{Variance} + \text{Noise}}$$

- **Bias** (squared): error from the model being too simple. The hypothesis class $\mathcal{H}$ does not contain functions that match the true relationship $\mathbb{E}[y \mid \mathbf{x}]$. Underfitting.
- **Variance**: error from the model being too sensitive to training data fluctuations. High-capacity models "memorize" the noise in different training sets and produce very different solutions. Overfitting.
- **Noise**: irreducible error from the data-generation process itself. Nothing we do can remove this.

1.5.3 Trade-off Illustration

Think of learning a d-degree polynomial from noisy data. Each training set gives a slightly different fit:

- $d = 0$ (constant): high bias, low variance. Every training set gives roughly the same average value, but it's a terrible fit to the true function.
- $d = 1$ (linear): still high bias, slightly lower variance.
- $d = 3$ (cubic): low bias, moderate variance. Fits the data just right.
- $d = 9$ (high-degree): low bias, high variance. Perfectly fits every training point, but wildly fluctuates between training sets.

Increasing model capacity always reduces bias but increases variance. The optimal capacity balances the two.

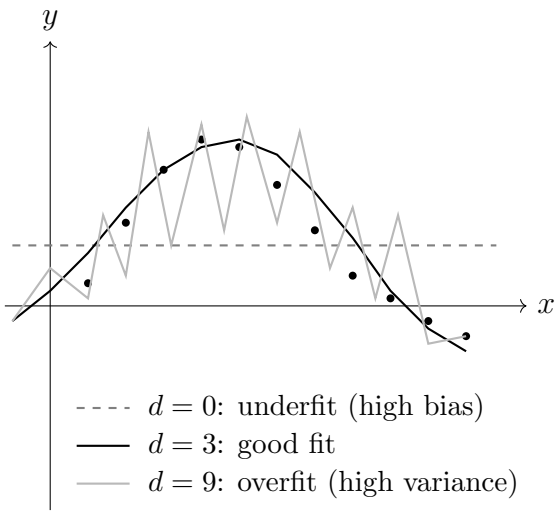


Figure 4: Bias–variance trade-off with polynomial fits. Gray dashed ($d = 0$): too simple, high bias, underfits. Black solid ($d = 3$): well-balanced, low bias and moderate variance. Gray ($d = 9$): too complex, memorizes noise, high variance, overfits. Filled circles are noisy training data.

1.5.4 Methods to Control Variance

1. **Regularization** (L1, L2): reduces variance by penalizing complex models; may slightly increase bias.
2. **Early stopping**: halt gradient descent before overfitting begins; an implicit regularization.

3. **Ensemble methods**: averaging or combining multiple models reduces variance dramatically.
4. **Bootstrap / Bagging**: train multiple models on bootstrap samples (data sampled with replacement), then average. Reduces variance.
5. **Boosting**: sequentially train models that focus on examples previous models misclassified; reduces bias.

1.5.5 Bias–Variance for Classification

The decomposition was originally derived for squared-loss regression. For 0-1 classification error, a similar decomposition exists under certain conditions, but the variance term is more complex. Nevertheless, the trade-off intuition — simpler models have high bias, complex models have high variance — carries over completely to classification.

2 Deep Neural Networks

2.1 Deep Feedforward Networks

2.1.1 Motivation: From Probabilistic Models to Deep Learning

So far we have studied probabilistic models for supervised learning:

$$\{\mathbf{x}_i, y_i\}_{i=1}^N \longrightarrow P(y \mid \mathbf{x})$$

In deep learning, we introduce a feature transformation learned by a neural network:

$$\{\mathbf{x}_i, y_i\}_{i=1}^N \longrightarrow \mathbf{h} = f(\mathbf{x}), \;\; P(y \mid \mathbf{h})$$

Here $\mathbf{h} = f(\mathbf{x})$ is a feature transformation represented by a neural network, and $P(y \mid \mathbf{h})$ is a probabilistic model on the transformed features. The whole model is $P(y \mid \mathbf{x}, \theta)$, where θ consists of both the weights in f and parameters in $P(y \mid \mathbf{h})$.

2.1.2 Feedforward Neural Networks as Function Approximators

Deep Feedforward Networks (FNN / MLP)

Deep feedforward networks — also called **feedforward neural networks (FNNs)** or **multilayer perceptrons (MLPs)** — define a function $\mathbf{h} = f(\mathbf{x}, \theta)$. During learning, the parameters θ are optimized so that $f(\mathbf{x}, \theta)$ approximates some target function $f^*(\mathbf{x})$. The network consists of simple computing elements called units (or neurons), organized into input, hidden, and output layers. Information flows from input to output without cycles (hence "feedforward").

The Unit A unit accepts a vector of inputs $\mathbf{x}$, computes an **affine transformation** $z = \mathbf{w}^\top \mathbf{x} + b$, where $\mathbf{w}$ is the weight vector and b is the bias, and then applies a nonlinear **activation function** $g(z)$:

$$\text{output} = g(\mathbf{w}^\top \mathbf{x} + b)$$

The quantity z is sometimes called the net input of the unit. Different unit types use different activation functions.

2.1.3 Weight Initialization

Why Good Initialization Matters

If weights start too small, signals shrink as they pass through each layer and become too tiny to be useful. If weights start too large, signals grow until they are too massive to be useful. Good initialization keeps the signal variance roughly constant across layers.

Xavier Initialization (Glorot) Used with tanh and sigmoid activations. For a neuron with n inputs:

$$w \sim U\left[-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}\right]$$

This keeps the variance of activations roughly constant across layers when using saturating activations.

He / Kaiming Initialization Used with ReLU activations. For a neuron with n inputs:

$$w \sim \mathcal{N}\left(0, \frac{2}{n}\right)$$

The factor of 2 compensates for the fact that ReLU zeros out half the signal on average.

Bias Initialization Biases are typically initialized to zero. For ReLU networks, a small positive bias (e.g., 0.1) is sometimes used to reduce the chance of starting with "dead" neurons (neurons whose pre-activation is always negative, producing zero gradient).

2.1.4 Activation Functions

General Form

All activation functions follow the pattern $g(z)$ where $z = \mathbf{w}^\top \mathbf{x} + b$. Below we list the most important ones.

Sigmoid (Logistic)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Output range: $(0, 1)$. Smooth and interpretable as a probability.
- **Problem**: Small gradients across most of its domain (vanishing gradient). The derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ has a maximum of 0.25 at $z = 0$. Multiplying many such gradients across deep layers causes the gradient to vanish exponentially.
- Saturates easily (outputs near 0 or 1), implying slow learning.
- **Recommendation**: Not recommended for hidden units. Acceptable as an output unit for binary classification (see later).

Rectified Linear Unit (ReLU)

$$g(z) = \max\{0, z\}$$

- Constant gradient ($= 1$) when $z > 0$, zero gradient when $z < 0$.
- Much faster learning than sigmoid/tanh due to non-vanishing gradient in the positive region.

- **Dead neuron problem**: If $z < 0$ for all training examples, the gradient is always zero and the neuron can never update. Mitigated by initializing biases to small positive values.
- The most commonly used activation function in modern deep learning.

Variations of ReLU General form: $g(z, \alpha) = \max\{0, z\} + \alpha \min\{0, z\}$

- **Absolute value rectification**: $\alpha = -1$ (i.e., $g(z) = |z|$)
- **Leaky ReLU**: α is a small fixed value like 0.01
- **Parametric ReLU (PReLU)**: α is learned from data
- All have non-zero gradient when $z < 0$, which mitigates the dead neuron problem.

Hyperbolic Tangent (Tanh)

$$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1$$

- Output range: $(-1, 1)$. Zero-centered, which is preferable to sigmoid.
- Larger gradients than sigmoid (maximum derivative $= 1$ at $z = 0$).
- Still saturates for large $|z|$, so the vanishing gradient problem persists.
- A popular choice in practice, especially in recurrent networks.

Softplus

$$g(z) = \zeta(z) = \log(1 + e^z)$$

- A smooth (infinitely differentiable) approximation to ReLU. Note that $\zeta'(z) = \sigma(z)$.
- Empirically not as good as ReLU for training deep networks.

Exponential Linear Unit (ELU)

$$g(z) = \begin{cases} z & \text{if } z \geq 0 \\ \alpha(e^z - 1) & \text{otherwise} \end{cases}$$

where $\alpha > 0$ is a hyperparameter (often initialized to 1). Its derivative is:

$$g'(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ \alpha e^z = g(z) + \alpha & \text{otherwise} \end{cases}$$

ELU has a non-zero gradient for negative inputs and tends to push mean activations toward zero, which helps with learning speed.

SwiGLU Family (Modern Activations) **SILU / SiLU (Sigmoid Linear Unit)**:

$$g(z) = z \cdot \sigma(z)$$

Swish:

$$g(z) = z \cdot \sigma(\beta z)$$

where β is a hyperparameter (fixed or learned). Discovered via Neural Architecture Search (NAS). Non-monotonic, unbounded above (avoids saturation), and has non-zero gradient for $z < 0$. Often outperforms ReLU in deep networks.

Mish:

$$g(z) = z \cdot \tanh\big(\log(1 + e^z)\big) = z \cdot \tanh(\zeta(z))$$

GELU (Gaussian Error Linear Unit):

$$g(z) = z \cdot \Phi(z)$$

where $\Phi(z)$ is the standard Gaussian CDF. Note the approximation $\Phi(z) \approx \sigma(1.7z)$. Used extensively in Transformers (BERT, GPT).

SwiGLU — used in modern Transformers (LLaMA, PaLM):

$$\text{SwiGLU}(\mathbf{z}) = \text{Swish}(W\mathbf{z} + \mathbf{b}) \odot (V\mathbf{z} + \mathbf{c})$$

where $W, V, \mathbf{b}, \mathbf{c}$ are learnable parameters, $\odot$ is elementwise product, and $\text{Swish}(z) = z \cdot \sigma(\beta z)$. More expressive than pure ReLU or GELU feed-forward blocks.

2.1.5 Forward Propagation

Let $\mathbf{h}^{(l)}$ denote the column vector of activations at layer l, $W^{(l)}$ the weight matrix at layer l, $\mathbf{b}^{(l)}$ the bias vector, and $g^{(l)}$ the activation function. Weight matrix $W^{(l)}$ has rows corresponding to output units and columns corresponding to input units (i.e., the weights feeding into each output unit are in each row).

The forward propagation rule for a general layer l is:

$$\boxed{\mathbf{h}^{(l)} = g^{(l)}\big(W^{(l)\top}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\big)}$$

Layer-by-Layer Forward Propagation

For a network with L layers, with input $\mathbf{x}$:

$$\mathbf{h}^{(0)} = \mathbf{x} \quad \text{(input layer)} \tag{1}$$

$$\mathbf{h}^{(1)} = g^{(1)}\big(W^{(1)}\mathbf{h}^{(0)} + \mathbf{b}^{(1)}\big) \tag{2}$$

$$\mathbf{h}^{(2)} = g^{(2)}\big(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\big) \tag{3}$$

$$\vdots$$

$$\mathbf{h}^{(L)} = g^{(L)}\big(W^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}\big) \quad \text{(output layer)} \tag{4}$$

Each layer applies an affine transformation followed by a nonlinear activation. Without the nonlinear g, stacking layers would be equivalent to a single linear layer (since the composition of linear functions is linear).

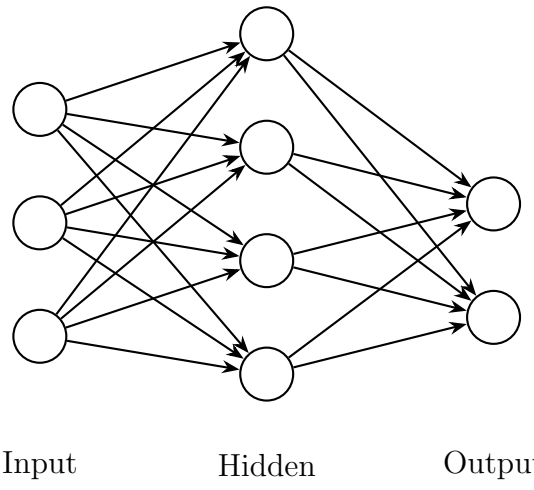


Figure 5: Feedforward neural network: information flows left to right through fully connected layers.

2.1.6 Universal Approximation Theorem

Universal Approximation

Theorem. Only one hidden layer of sigmoid units suffices to approximate any well-behaved function (e.g., bounded continuous functions on a compact domain) to arbitrary precision.

This theorem is often misinterpreted. It guarantees existence but says nothing about learnability or parameter efficiency. In practice:

- Deeper networks can approximate the same function with exponentially fewer parameters due to the hierarchical nonlinearity they can induce.
- Deeper networks also better mimic the hierarchical organization of data in many real-world applications (e.g., vision: edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects).
- Deep learning is most useful when you need a complex function and have abundant data.

2.1.7 FNN as a Probabilistic Model

An FNN can be combined with a probabilistic model on top of it:

$$\mathbf{h} = f(\mathbf{x}) \quad \text{(feature transform from hidden layers)}, \qquad P(\mathbf{y} \mid \mathbf{h}) \quad \text{(output layer)}$$

Together, the whole network defines $P(\mathbf{y} \mid \mathbf{x}, \theta)$. First define the **logit vector**:

$$\mathbf{z} = W_{\text{out}}^{\top}\mathbf{h} + \mathbf{b}_{\text{out}}$$

Linear-Gaussian Output (Regression) When y is a real-valued scalar, assume:

$$p(y \mid \mathbf{x}) = \mathcal{N}(y \mid z,\, \sigma^2)$$

where z is a scalar logit. The per-sample negative log-likelihood loss is:

$$L(\mathbf{x}, y, \theta) = -\log p(y \mid \mathbf{x}) \propto \frac{1}{2}(y - z)^2$$

This is the **Mean Squared Error (MSE)** loss. The derivative of the loss with respect to the logit is:

$$\frac{\partial L}{\partial z} = z - y$$

which is simply the prediction error.

Sigmoid Output (Binary Classification) When $y \in \{0, 1\}$:

$$P(y \mid \mathbf{x}) = \text{Ber}(y \mid \sigma(z))$$

The per-sample loss is the **binary cross-entropy**:

$$L(\mathbf{x}, y, \theta) = -\big[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))\big]$$

The derivative with respect to the logit is:

$$\frac{\partial L}{\partial z} = \sigma(z) - y$$

Again, the prediction error. Note: sigmoid units are fine as output units (even though not recommended for hidden units) because saturation occurs only when the model is already correct ($z \gg 0$ with $y = 1$, or $z \ll 0$ with $y = 0$).

Softmax Output (Multi-Class Classification) When $y \in \{1, 2, \ldots, C\}$:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^{C} \exp(z_j)}$$

Numerically stable implementation uses the **log-sum-exp** trick:

$$\log P(y = k \mid \mathbf{x}) = z_k - \log\left(\sum_{j=1}^{C} \exp(z_j)\right)$$

The per-sample **cross-entropy loss** and its derivative are:

$$L = -\log P(y \mid \mathbf{x}), \qquad \frac{\partial L}{\partial z_k} = P(y = k \mid \mathbf{x}) - \mathbf{1}[y = k]$$

The derivative is again the difference between the predicted probability and the one-hot target.

2.1.8 Backpropagation

Given data $\{\mathbf{x}_i, y_i\}_{i=1}^{N}$, we minimize the average loss using stochastic gradient descent (SGD):

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} L(\mathbf{x}_i, y_i, \theta) \qquad \text{and} \qquad \theta \leftarrow \theta - \eta \nabla_\theta L(\mathbf{x}_i, y_i, \theta)$$

The key challenge is computing the gradient $\nabla_\theta L$. The **backpropagation** algorithm does this efficiently using the chain rule.

Chain Rule Review For composite functions, the chain rule states:

$$\frac{dy}{dt} = \frac{dy}{dx}\frac{dx}{dt}$$

Backpropagation applies this systematically through the network graph.

Output Layer Gradient Consider a small slice of the network: unit k in the output layer, unit j in the previous hidden layer. Ignoring biases for simplicity:

$$z_k = \sum_j u_j W_{kj}, \qquad u_j = g(z_j)$$

where z_k is the logit of unit k, u_j is the output of unit j, and W_{kj} is the weight from unit j to unit k. The gradient of the loss with respect to W_{kj} is:

$$\frac{\partial L}{\partial W_{kj}} = \frac{\partial L}{\partial z_k}\frac{\partial z_k}{\partial W_{kj}} = u_j \cdot \delta_k$$

where we define the **error term** of the output unit:

$$\boxed{\delta_k \;=\; \frac{\partial L}{\partial z_k}}$$

This is the prediction error we computed earlier: e.g., $\delta_k = z_k - y_k$ for regression, or $\delta_k = \sigma(z_k) - y$ for binary classification.

Hidden Layer Gradient Now consider weight W_{ji} from unit i to unit j (where unit j is in some hidden layer and unit k is in the next layer):

$$z_j = \sum_i u_i W_{ji}, \qquad u_j = g(z_j), \qquad z_k = \sum_j u_j W_{kj}$$

Applying the chain rule:

$$\begin{aligned}\frac{\partial L}{\partial W_{ji}} &= \sum_k \delta_k \frac{\partial z_k}{\partial u_j}\frac{\partial u_j}{\partial z_j}\frac{\partial z_j}{\partial W_{ji}} \\ &= \sum_k \delta_k W_{kj} \cdot g'(z_j) \cdot u_i \;=\; u_i \cdot \delta_j\end{aligned}$$

where the hidden unit error is defined as:

$$\boxed{\delta_j \;=\; g'(z_j) \sum_k W_{kj}\delta_k}$$

This is the key recurrence: the error at a hidden unit is the local derivative of the activation times the weighted sum of errors from the next layer.

Backpropagation Algorithm

Objective: Compute $\partial L/\partial W_{ji}$ for all weights (and biases) for one training example.

1. **Forward pass**: Propagate forward to compute the net input z_j and output $u_j = g(z_j)$ of each unit.
2. **Backward pass** (error propagation):
   - For each output unit k: $\delta_k \leftarrow \partial L/\partial z_k$ (the prediction error).
   - For each hidden unit j, working backwards from the output layer:

$$\delta_j \;\leftarrow\; g'(z_j) \sum_k W_{kj}\delta_k$$

3. **Compute gradients**:

$$\frac{\partial L}{\partial W_{ji}} \;=\; u_i\,\delta_j, \qquad \frac{\partial L}{\partial b_j} \;=\; \delta_j$$

The weights W are typically stored as matrices where each row corresponds to a receiving unit, so the gradient $\partial L/\partial W$ has the same shape as W. Biases receive the same update as the error term of their unit.

Summary of Key Identities

$$\boxed{\frac{\partial L}{\partial W_{kj}} = u_j\delta_k, \qquad \delta_k = \frac{\partial L}{\partial z_k}} \qquad \text{(output layer)}$$

$$\boxed{\frac{\partial L}{\partial W_{ji}} = u_i\delta_j, \qquad \delta_j = g'(z_j)\sum_k W_{kj}\delta_k} \qquad \text{(hidden layer)}$$

Backpropagation and SGD are implemented automatically in deep learning frameworks such as PyTorch, TensorFlow, and JAX via automatic differentiation. Weights are packed into tensors for efficient GPU computation.

2.2 Regularization and Optimization

Deep learning models are powerful but prone to overfitting and difficult to train. This lecture covers three critical topics: regularization, optimization algorithms, and batch normalization.

2.2.1 Regularization

The Overfitting Problem

Deep learning models have very large numbers of parameters (millions to billions). With limited training data, they can memorize the training set including its noise, leading to poor generalization on unseen data. Regularization techniques aim to reduce generalization error while preserving model expressiveness.

L2 Regularization (Weight Decay) The most common regularization method augments the loss function with a penalty on parameter magnitude:

$$\boxed{\tilde{L}(\theta) = L(\theta) + \lambda \|\theta\|_2^2}$$

where $\lambda \geq 0$ is the regularization strength hyperparameter. The penalty encourages smaller weights, which reduces model complexity and prevents sharp minima that overfit noise. This is called weight decay because the corresponding SGD update is:

$$\theta \leftarrow \theta - \eta \nabla_\theta L - 2\eta\lambda\theta$$

At each step, weights are explicitly "decayed" toward zero by a factor $2\eta\lambda$. Almost always do this in deep learning.

Bagging (Bootstrap Aggregating) Bagging reduces variance by combining several models trained on different random subsets of data:

1. Draw B bootstrap samples from the training set (each of size N, drawn with replacement).
2. Train a separate model on each bootstrap sample.
3. For prediction, average the outputs (regression) or take a majority vote (classification).

Bagging works because different bootstrap samples contain different randomness; the models are unlikely to all make the same errors on the test set, so averaging cancels out variance.

Dropout Training a separate model for each bootstrap sample is too expensive for deep networks. **Dropout** (Hinton et al., 2012) is an inexpensive approximation of bagging specifically designed for deep learning.

Dropout Algorithm

For each minibatch during training:

1. For each hidden unit (and optionally input unit), sample a binary mask variable $m \sim \text{Bernoulli}(p)$, where p is the keep probability (typically 0.5 for hidden layers, higher like 0.8 for input layers).
2. Multiply the unit's activation by its mask before passing it to the next layer. Units with $m = 0$ are temporarily removed from the network.
3. Perform one step of gradient descent. Only parameters of "active" units ($m = 1$) receive non-zero gradients.

Dropout is only applied during training, never during inference.

Dropout prevents complex co-adaptations of parameters: no single unit can rely on specific other units being present, because during training they are randomly removed. This forces the network to learn more robust, distributed representations.

Inverted Dropout Since dropout is not applied at test time, a hidden unit's expected input is larger at test time (it receives connections from $1/p$ times as many units on average). Two equivalent corrections:

- **Post-hoc scaling**: after training, multiply each layer's input weights by the keep probability p.
- **Inverted dropout** (preferred): divide each unit's output by p during training. Then no change is needed at test time — inference is not slowed down.

2.2.2 Optimization Algorithms

Our task is: given a loss $L(\theta)$, find the parameters θ that minimize it. The standard approach is gradient descent with variations.

Stochastic Gradient Descent (SGD) SGD estimates the true gradient from a random minibatch $\mathcal{B}$ of examples:

$$\theta \leftarrow \theta - \eta \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta L(\mathbf{x}_i, y_i, \theta)$$

where η is the learning rate. SGD introduces noise into the gradient estimate, which can help escape local minima.

Momentum SGD with **momentum** (Polyak, 1964) accelerates convergence by accumulating a velocity vector:

$$\mathbf{v}_t = \alpha \mathbf{v}_{t-1} - \eta \nabla_\theta L(\theta_t), \qquad \theta_{t+1} = \theta_t + \mathbf{v}_t$$

- $\mathbf{v}_t$ is an exponentially decaying average of past negative gradients.
- $\alpha \in [0, 1)$ is the momentum coefficient, typically 0.5, 0.9, or 0.99.
- View θ as a "particle" rolling through parameter space: momentum lets it build up speed in consistent directions and dampens oscillations in high-curvature directions.

AdaGrad (Adaptive Gradient Algorithm) AdaGrad (Duchi et al., 2011) uses per-parameter adaptive learning rates. Parameters that have changed a lot get smaller effective learning rates:

$$G_{t,i} = G_{t-1,i} + \left(\frac{\partial L}{\partial \theta_i}\right)^2, \qquad \theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i}} + \epsilon} \frac{\partial L}{\partial \theta_i}$$

- Each parameter θ_i gets its own accumulated squared gradient sum $G_{t,i}$.
- Good for sparse data (e.g., NLP embeddings) where some parameters receive gradients much less frequently than others.
- **Problem**: $G_{t,i}$ only grows, so the effective learning rate shrinks to zero over time — can stall in deep networks.

RMSProp (Root Mean Square Propagation) RMSProp (Hinton, unpublished) fixes AdaGrad's monotonic accumulation by using an exponential moving average to dis-

card ancient history:

$$G_{t,i} = \beta G_{t-1,i} + (1-\beta)\left(\frac{\partial L}{\partial \theta_i}\right)^2, \qquad \theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i}} + \epsilon}\frac{\partial L}{\partial \theta_i}$$

where β is the decay rate (typically 0.9). This adapts quickly to changing gradient statistics.

Adam (Adaptive Moments) Adam (Kingma & Ba, 2015) combines RMSProp with momentum, plus bias correction for the initial estimates:

1. Compute biased first-moment estimate (momentum):

$$\boxed{m_{t,i} = \beta_1 m_{t-1,i} + (1-\beta_1)\frac{\partial L}{\partial \theta_i}}$$

2. Compute biased second-moment estimate (RMSProp):

$$\boxed{v_{t,i} = \beta_2 v_{t-1,i} + (1-\beta_2)\left(\frac{\partial L}{\partial \theta_i}\right)^2}$$

3. Bias correction (important because $m_0 = v_0 = 0$ causes initial underestimation):

$$\boxed{\hat{m}_{t,i} = \frac{m_{t,i}}{1-\beta_1^t}, \qquad \hat{v}_{t,i} = \frac{v_{t,i}}{1-\beta_2^t}}$$

4. Parameter update:

$$\boxed{\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{\hat{v}_{t,i}} + \epsilon}\hat{m}_{t,i}}$$

Typical hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Adam is often the best default choice.

Adam Bias Correction Derivation

The running averages are initialized at zero: $m_0 = v_0 = 0$. At step t, the EMA updates are $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$ and $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$, where $g_t = \partial L/\partial \theta_i$ is the gradient at step t.

Unrolling the recursion. For the first moment:

$$m_t = (1-\beta_1)g_t + \beta_1 m_{t-1}$$

Substitute $m_{t-1} = (1-\beta_1)g_{t-1} + \beta_1 m_{t-2}$:

$$m_t = (1-\beta_1)g_t + \beta_1(1-\beta_1)g_{t-1} + \beta_1^2 m_{t-2}$$

Continuing to unroll all the way to $m_0 = 0$:

$$m_t = (1-\beta_1)\sum_{k=0}^{t-1} \beta_1^k \, g_{t-k}$$

Expected value under stationarity. Suppose gradients are drawn from a stationary distribution $\mathbb{E}[g_t] = \mu$. Then:

$$\mathbb{E}[m_t] = (1-\beta_1)\sum_{k=0}^{t-1} \beta_1^k \, \mathbb{E}[g_{t-k}] = (1-\beta_1)\mu \sum_{k=0}^{t-1} \beta_1^k$$

The geometric sum evaluates to $\sum_{k=0}^{t-1} \beta_1^k = \frac{1-\beta_1^t}{1-\beta_1}$. Therefore:

$$\mathbb{E}[m_t] = (1-\beta_1)\mu \cdot \frac{1-\beta_1^t}{1-\beta_1} = (1-\beta_1^t)\mu$$

So m_t is a biased estimate of μ, with multiplicative bias factor $1-\beta_1^t$. The bias vanishes as $t \to \infty$, but is substantial early in training (e.g., for $\beta_1 = 0.9$ and $t = 1$, the bias is 90%). Correcting by dividing:

$$\boxed{\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}}$$

gives unbiased estimates $\mathbb{E}[\hat{m}_t] = \mu$ and $\mathbb{E}[\hat{v}_t] = \mathbb{E}[g^2]$. The same derivation applies verbatim to v_t with β_2 instead of β_1.

AdamW (Decoupled Weight Decay)

L2 Regularization vs. Weight Decay

For SGD, L2 regularization $\tilde{L} = L + \lambda\|\theta\|^2$ and weight decay $\theta \leftarrow \theta - \eta\nabla L - \eta\lambda\theta$ are mathematically equivalent. But for Adam (and other adaptive optimizers), they are not equivalent. L2 regularization adds the gradient of $\lambda\|\theta\|^2$ to the gradient used by Adam, which then gets divided by the per-parameter adaptive step size $\sqrt{\hat{v}_i}$. This means parameters with large historic gradients get regularized less than intended.

AdamW (Loshchilov & Hutter, 2019) decouples weight decay from the adaptive gradient update:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{\hat{v}_{t,i}} + \epsilon}\hat{m}_{t,i} - \eta\lambda\theta_{t,i}$$

The weight decay term $\eta\lambda\theta$ is applied directly to the parameter, independent of the adaptive optimizer. This has become the standard way to apply L2 regularization with Adam.

2.2.3 Learning Rate Schedules

The learning rate η should typically decrease during training. Common schedules:

- **Step decay**: reduce by a factor (e.g., 0.1) at fixed epoch intervals. Used in AlexNet.
- **Exponential decay**: $\eta_t = \eta_0 \cdot \gamma^t$ for $\gamma \in (0, 1)$.
- **Cosine annealing**: $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\frac{t}{T}\pi))$. Smoothly decreases then "restarts" — the restart acts like a warm restart because the model keeps good weights from the previous cycle.
- **1/t decay**: $\eta_t = \eta_0/(1 + kt)$.

2.2.4 Batch Normalization

Different features in the input may have very different scales (e.g., $x_1 \in [0, 1]$ vs. $x_2 \in [10, 1000]$). This causes two problems:

1. Regularization affects features with small magnitude more than those with large magnitude.
2. Contours of the loss function become elongated, slowing gradient descent.

Data Normalization Standard practice normalizes each input feature to zero mean and unit variance before training:

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \qquad \mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{ij}, \qquad \sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{ij} - \mu_j)^2$$

Normalization at Each Layer

Covariate Shift

In a deep model, each layer can be seen as taking the output of the previous layer as input. As earlier layers' parameters change during training, the distribution of inputs to later layers shifts — this is called **internal covariate shift**. It makes training difficult because later layers must constantly adapt to changing input distributions.

Batch Normalization (BatchNorm) (Ioffe & Szegedy, 2015) normalizes the inputs of each layer for each mini-batch, making layers more independent and accelerating training.

BatchNorm Computation

For a layer with pre-activations $\{x_i\}_{i\in\mathcal{B}}$ within a minibatch $\mathcal{B}$:

1. Compute minibatch mean: $\mu_{\mathcal{B}} = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} x_i$
2. Compute minibatch variance: $\sigma_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}(x_i - \mu_{\mathcal{B}})^2$
3. Normalize: $\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$
4. **Affine transform** (learnable parameters γ and β):

$$\boxed{y_i = \gamma\hat{x}_i + \beta}$$

The affine transform lets the layer learn to undo normalization if needed. This is crucial because it preserves the layer's representational power — BatchNorm does not reduce to the identity transformation.

Training vs. Inference

- **Training**: normalize using the current minibatch statistics $\mu_{\mathcal{B}}, \sigma_{\mathcal{B}}^2$.
- **Inference (test time)**: use running estimates of μ and σ^2 computed via exponential moving average over the entire training set. At test time there is no meaningful "minibatch" statistic for a single example.

Why BatchNorm Works

- **Reduces vanishing gradients**: by keeping activations in a reasonable range, BatchNorm makes gradient propagation much easier, even when using saturating activations like sigmoid or tanh.
- **Less sensitive to initialization**: the network works well with a wider range of weight initializations.
- **Regularizing effect**: the noise in minibatch statistics acts like a regularizer, somewhat similar to dropout.
- **Allows higher learning rates**: training converges faster because the loss landscape becomes more isotropic (less elongated).

LayerNorm vs. BatchNorm

Normalization Variants

LayerNorm normalizes across the feature dimension for a single example (instead of across the batch dimension for a single feature):

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad \mu = \frac{1}{D}\sum_{j=1}^{D} x_j, \quad \sigma^2 = \frac{1}{D}\sum_{j=1}^{D}(x_j - \mu)^2$$

- BatchNorm: normalize each feature across the batch. Depends on minibatch size; problematic for small batch sizes. Used in CNNs.
- LayerNorm: normalize all features within a single example. Independent of batch size. Used in RNNs and Transformers.
- Both include learnable γ and β parameters.

2.3 Convolutional Neural Networks

2.3.1 Introduction and Inductive Biases

Convolutional Neural Networks (CNNs, ConvNets) are specialized feedforward neural networks designed for processing data with a known grid-like topology — most notably images, where neighboring pixels are spatially related. Unlike FNNs, CNN inputs are 3D tensors (width $\times$ height $\times$ channels), where spatial structure matters.

CNNs were inspired by observations in the visual cortex by Hubel and Wiesel (1959, Nobel Prize 1981), who found that neurons have small receptive fields (reacting only to stimuli from a small visual region), and that higher-level neurons aggregate outputs of lower-level ones to detect complex patterns. Fukushima's neocognitron (1980) was the precursor to modern CNNs.

Two Key Inductive Biases in CNNs

CNNs incorporate two strong inductive biases for visual data:

1. **Locality (Local Receptive Field)**: Each unit in a convolutional layer connects only to a small patch of the previous layer (its receptive field), rather than the full volume. This reflects the observation that visual features (edges, textures) are local in space.
2. **Translation Equivariance (Weight Sharing)**: All units on the same depth slice share the same filter weights. This lets the network detect the same feature at any location in the image, dramatically reducing the number of parameters.

An FNN with the same number of hidden units but no weight sharing would have vastly more parameters and would fail to exploit spatial structure.

2.3.2 The Convolutional Layer

Cross-Correlation (aka "Convolution" in Deep Learning)

Let I be a 2D input (one channel of an image) and K be a 2D filter (kernel) of size $(2r+1)\times(2r+1)$, so $K(m,n) = 0$ when $|m| > r$ or $|n| > r$. The output at coordinates (i,j) is:

$$\boxed{S(i,j) = \sum_{m,n} I(i+m,\ j+n)\, K(m,n)}$$

This formula is technically **cross-correlation**, not convolution. True convolution

flips the kernel: $\sum_{m,n} I(i-m, j-n)K(m,n)$. However, since the kernel K is learned from scratch, flipping it would just learn a flipped version of the same kernel. The distinction is irrelevant in deep learning, and the terms are used interchangeably.

The result of cross-correlation is passed through a nonlinear activation function (e.g., ReLU), giving the output of each convolutional unit.

Filters, Feature Maps, and Depth

- A single filter (kernel) slides across the input volume, producing one **feature map**.
- A convolutional layer uses multiple filters to detect different features (e.g., horizontal edges, vertical edges, corners). Each filter produces one depth slice.
- The output of a convolutional layer is a 3D tensor consisting of multiple stacked feature maps.
- The receptive field is local in width and height but always spans the full depth of the input volume (i.e., all input channels). So a filter of spatial size $F \times F$ applied to a D_1-channel input has total weight count $F \cdot F \cdot D_1 + 1$ (the $+1$ is for the bias).

2.3.3 Technical Issues: Padding, Stride, Output Size

Reduction in Spatial Size Without padding, applying a filter of size $F \times F$ to an $W \times H$ image with stride 1 reduces the spatial dimensions by $(F-1)$. For a 3×3 filter, the output spatial size is $(W-2) \times (H-2)$.

Zero Padding To preserve spatial dimensions, we can pad the borders of the input with zeros (or replicate edge values). Zero padding helps preserve edge information that would otherwise be "shrunk away."

Stride The **stride** S controls how many pixels the filter moves between successive applications. Stride 1 means the filter shifts by one pixel at a time; stride 2 means it skips every other position, halving the output spatial size. Strides of 3 or more are uncommon.

Output Size Formula

Given input volume of size $W_1 \times H_1 \times D_1$, with K filters each of spatial extent $F \times F$, stride S, and zero padding P, the output volume has size:

$$\boxed{W_2 = \frac{W_1 - F + 2P}{S} + 1, \qquad H_2 = \frac{H_1 - F + 2P}{S} + 1, \qquad D_2 = K}$$

The fractions $(W_1 - F + 2P)/S$ and $(H_1 - F + 2P)/S$ must be integers. For example, $W_1 = 10$, $F = 3$, $P = 0$, $S = 2$ is invalid because $(10-3)/2 = 3.5$ is not an integer.

Parameter and FLOP Count

- **Number of parameters**: Each filter has $F \cdot F \cdot D_1 + 1$ weights (the $+1$ is bias). With K filters: $(FFD_1 + 1)K$.
- **Number of FLOPs**: Each output element requires $F \cdot F \cdot D_1$ multiplications and $(FFD_1 - 1)$ additions, plus 1 bias addition. Total: approximately $2FFD_1W_2H_2D_2$.
- **Compare to fully connected**: An FNN layer with the same output spatial dimensions would require $(W_1H_1D_1 + 1)W_2H_2D_2$ parameters — which can be orders of magnitude larger. Parameter sharing is what makes CNNs practical.

2.3.4 Pooling Layer

Pooling

A pooling layer reduces the spatial size of feature maps by aggregating each patch into a single value. It has no learnable parameters. The two most common types:

- **Max pooling**: output the maximum value in the patch. Found empirically to work best.
- **Average pooling**: output the mean value in the patch.

Pooling serves two purposes:

1. **Reduces computation**: smaller feature maps mean fewer operations in subsequent layers.
2. **Induces shift invariance**: small translations of the input result in the same pooled output if the maximum (or average) value remains in the same pooling window. This is an inductive bias for tasks where we care about whether a feature is present, not exactly where.

Max pooling introduces a third inductive bias in CNNs: **shift invariance** (complementing locality and translation equivariance). Modern architectures (ResNet and later) sometimes use strided convolutions instead of pooling for downsampling, since this lets the network learn the downsampling operation.

2.3.5 Example CNN Architectures

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) has driven the evolution of CNN architectures since 2010.

LeNet-5 (LeCun et al., 1998) The pioneering CNN architecture, designed for handwritten digit recognition (MNIST). Consists of alternating convolutional and subsampling (pooling) layers followed by fully connected layers. Established the basic pattern: Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC $\rightarrow$ FC.

AlexNet (Krizhevsky et al., 2012) Won ILSVRC 2012 by a massive margin (top-5 error 15.3% vs. 26.2% for the second-best non-DL method), igniting the deep learning revolution.

- 5 convolutional layers + 3 fully connected layers.
- Approximately 60M parameters (most in the FC layers).
- **First to use ReLU** in a large CNN.
- Heavy data augmentation (random crops, horizontal flips, color jitter).
- Dropout with $p = 0.5$ on FC layers.
- SGD with momentum 0.9, batch size 128, L2 weight decay.
- Learning rate 0.01, manually reduced by factor 10 when validation plateaued.
- Trained on two GPUs in parallel (split across feature maps).

VGG (Simonyan & Zisserman, 2015) Runner-up in ILSVRC 2014. Demonstrated that **depth** is a critical component of good CNN performance.

- 16 (or 19) weight layers.
- Only 3×3 convolutions throughout. Two stacked 3×3 conv layers have the same receptive field as one 5×5 layer but with fewer parameters and more nonlinearity.
- 138M parameters (mostly still in FC layers).
- Simple, uniform architecture that became a standard feature extractor.

ResNet (He et al., 2016) Won ILSVRC 2015 by a huge margin. Allowed training of networks with hundreds of layers (the 152-layer variant).

The Degradation Problem

Simply stacking more layers does not always help. A 56-layer plain network has higher training error than a 20-layer plain network. This is not overfitting (which would mean low training error, high test error); it is a training difficulty: very deep plain networks have trouble propagating gradients to lower layers.

Residual Connections (Skip Connections)

ResNet addresses the degradation problem with **identity skip connections**. A residual block computes:

$$\boxed{\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l, \theta_l)}$$

where F is a stack of convolutional layers (the "residual" mapping), and the $+\mathbf{h}_l$ term is the identity shortcut connection. The network learns to approximate $F(\mathbf{h}_l, \theta_l) = H(\mathbf{h}_l) - \mathbf{h}_l$, the residual between the desired mapping $H(\mathbf{h}_l)$ and the identity.

Why it works: During backpropagation, the gradient $\partial L/\partial \mathbf{h}_l$ receives a path of the form:

$$\frac{\partial L}{\partial \mathbf{h}_l} = \frac{\partial L}{\partial \mathbf{h}_{l+1}} + \frac{\partial L}{\partial \mathbf{h}_{l+1}} \cdot \frac{\partial F}{\partial \mathbf{h}_l}$$

The first term is the unmodified gradient from the skip connection, which never vanishes. This solves the vanishing gradient problem for arbitrarily deep networks.

A ResNet with more layers should always be at least as good as a shallower one: the deeper network can learn $F(\mathbf{h}) \equiv 0$ for extra layers, reducing to the shallower architecture. In practice, ResNets show lower training and test errors with more layers.

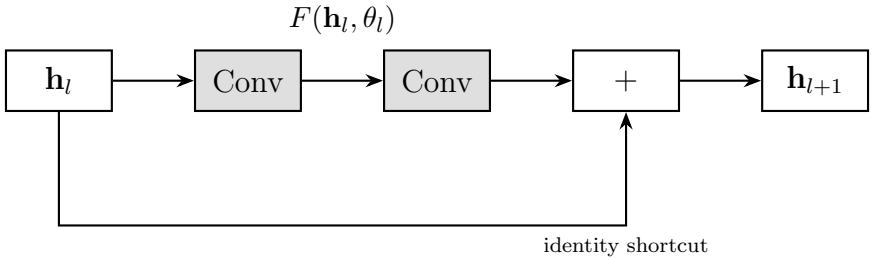


Figure 6: A residual block: the output is $\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l, \theta_l)$. The skip connection lets gradients flow unmodified.

DenseNet (Huang et al., 2017) DenseNets extend skip connections by connecting every layer to all previous layers via concatenation (not summation as in ResNet). Letting $\mathbf{x}_\ell$ denote the feature map of layer ℓ:

$$\text{ResNet}: \quad \mathbf{x}_\ell = F_\ell(\mathbf{x}_{\ell-1}) + \mathbf{x}_{\ell-1}$$

$$\text{DenseNet}: \quad \mathbf{x}_\ell = F_\ell\big([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{\ell-1}]\big)$$

where $[\cdot]$ denotes concatenation along the channel dimension. DenseNet advantages:

- Direct connections from all preceding layers make gradient flow very easy.
- Layer ℓ has direct access to all preceding feature maps, improving information flow.
- Fewer parameters ("slim" layers with e.g. 12 filters each) because there is no need to re-learn redundant feature maps.

2.3.6 Data Augmentation

Overfitting can be reduced by getting more training data, but labeled data is often scarce. **Data augmentation** generates synthetic training examples from existing ones, implicitly telling the model which transformations are allowed invariances:

- Translation, rotation, scaling by various amounts
- Random cropping (then resizing to fixed size)
- Horizontal flipping (not suitable for all tasks, e.g., digit recognition)
- Color jitter (contrast, brightness, saturation, hue)
- Mixup: convex combination of two training examples

Augmentation is done on the fly during training (to save storage) and is a standard regularization technique in vision.

2.4 Recurrent Neural Networks

2.4.1 Why Sequential Models?

Feedforward and convolutional networks map a fixed-size input $\mathbf{x}$ to an output $\mathbf{y}$. Many real-world tasks involve sequential data, where each example is a variable-length sequence of inputs and outputs:

$$\big(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(\tau)}\big) \longrightarrow \big(\mathbf{y}^{(1)}, \mathbf{y}^{(2)}, \ldots, \mathbf{y}^{(\tau)}\big)$$

The current output $\mathbf{y}^{(t)}$ may depend on all previous inputs $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(t)}$. Important applications:

- **Language modeling**: predict the next word: $P(w_k \mid w_1, \ldots, w_{k-1})$. A sentence's probability factorizes as:

$$P(w_1, \ldots, w_n) = P(w_1)P(w_2 \mid w_1)P(w_3 \mid w_1, w_2) \cdots P(w_n \mid w_1, \ldots, w_{n-1})$$

- **Machine translation**: map an input sequence (source language) to an output sequence (target language).
- **Speech recognition**: map audio frames to text transcript.
- **Video analysis**: action recognition from frame sequences.
- **Time-series forecasting**: stock prices, weather, sensor data.

2.4.2 Vanilla Recurrent Neural Network

RNN Architecture

A Recurrent Neural Network (RNN) processes sequential data by maintaining a hidden state $\mathbf{h}^{(t)}$ that carries information about all previous inputs up to time t. The computation at each time step is:

$$\boxed{\mathbf{a}^{(t)} = \mathbf{b} + W_{\text{hh}}\,\mathbf{h}^{(t-1)} + W_{\text{ih}}\,\mathbf{x}^{(t)}, \qquad \mathbf{h}^{(t)} = \tanh\big(\mathbf{a}^{(t)}\big)}$$

where $\mathbf{x}^{(t)}$ is the input at time t, $\mathbf{h}^{(t)}$ is the hidden state, and the parameters $\mathbf{b}$, W_{hh}, W_{ih} are shared across all time steps. The hidden state $\mathbf{h}^{(t)}$ acts as a memory: $\mathbf{h}^{(t-1)}$ summarizes everything about the sequence before time t.

An RNN can be represented compactly as a recurrent graph (with a loop) or unrolled through time (a deep feedforward network with shared weights). The length of the unrolled graph equals the sequence length.

Output and Loss The output at each time step is typically:

$$\mathbf{o}^{(t)} = \mathbf{c} + W_{\text{ho}}\,\mathbf{h}^{(t)}, \qquad \hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{o}^{(t)})$$

For one training pair $(\{\mathbf{x}^{(t)}\}, \{\mathbf{y}^{(t)}\})$, the loss is the sum of per-step cross-entropy losses:

$$L = -\sum_{t=1}^{\tau} \log P_{\text{model}}\big(y^{(t)} \mid \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(t)}\big)$$

The probability $P_{\text{model}}(y^{(t)} \mid \cdots)$ is obtained by reading the entry for $y^{(t)}$ from the softmax output $\hat{\mathbf{y}}^{(t)}$.

Backpropagation Through Time (BPTT) RNNs are trained using **Backpropagation Through Time (BPTT)**, which is regular backpropagation applied to the unrolled computational graph. Because weights are shared across time, gradients must be summed over all time steps. BPTT is implemented automatically in all modern frameworks.

2.4.3 Vanishing and Exploding Gradients

RNNs can be seen as very deep feedforward networks (one layer per time step) with shared weights W_{hh}. When backpropagating through many steps:

$$\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-k)}} = \prod_{j=1}^{k} \big(W_{\text{hh}}^{\top} \cdot \text{diag}\big(1 - (\mathbf{h}^{(t-j)})^2\big)\big)$$

- If the spectral radius (largest singular value) of W_{hh} is less than 1, the product of matrices causes gradients to **vanish** exponentially.
- If the spectral radius is greater than 1, gradients **explode** exponentially.
- The tanh derivative further amplifies vanishing gradients (max derivative = 1, often much less).

As a result, vanilla RNNs struggle to learn long-range dependencies (relationships between distant time steps, e.g., subject–verb agreement in long sentences).

Gradient Clipping for Exploding Gradients Exploding gradients are relatively easy to fix:

$$\mathbf{g}' = \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\|_2 \le \delta \\ \dfrac{\delta}{\|\mathbf{g}\|_2}\,\mathbf{g} & \text{otherwise} \end{cases}$$

This preserves the gradient direction while capping its magnitude. The threshold δ is typically around 5.

Truncated BPTT To make long sequences tractable, BPTT is often truncated: the RNN is unrolled only over a fixed number of steps (e.g., 35), gradients computed for that window, then the process repeats. This is an approximation but works well in practice.

2.4.4 Long Short-Term Memory (LSTM)

The LSTM (Hochreiter & Schmidhuber, 1997) was specifically designed to address the vanishing gradient problem in RNNs and learn long-range dependencies.

Key Idea: Cell State with Additive Interactions

The LSTM introduces a cell state $\mathbf{C}^{(t)}$ that runs along the top of the LSTM block, updated by additive interactions rather than multiplicative ones. This provides a pathway where gradients can flow unmodified (no nonlinearity, no weight matrix applied), preventing them from vanishing. Three gates control information flow: forget gate, input gate, and output gate.

LSTM Cell: Full Equations

An LSTM cell at time step t computes the following:
Forget gate — decides what to discard from the old cell state:

$$\mathbf{f}^{(t)} = \sigma\big(W_f\big[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}\big] + \mathbf{b}_f\big)$$

Input gate — decides what new information to store in the cell state:

$$\mathbf{i}^{(t)} = \sigma\big(W_i\big[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}\big] + \mathbf{b}_i\big)$$

Cell candidate — new candidate values for the cell state:

$$\tilde{\mathbf{C}}^{(t)} = \tanh\big(W_C\big[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}\big] + \mathbf{b}_C\big)$$

Cell state update — the critical additive update:

$$\boxed{\mathbf{C}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{C}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{C}}^{(t)}}$$

Output gate — decides what part of the cell state to output:

$$\mathbf{o}^{(t)} = \sigma\big(W_o\big[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}\big] + \mathbf{b}_o\big)$$

Hidden state output:

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh\big(\mathbf{C}^{(t)}\big)$$

Here $[\mathbf{h}; \mathbf{x}]$ denotes concatenation, $\odot$ denotes elementwise (Hadamard) product, σ is the sigmoid function, and tanh is applied elementwise.

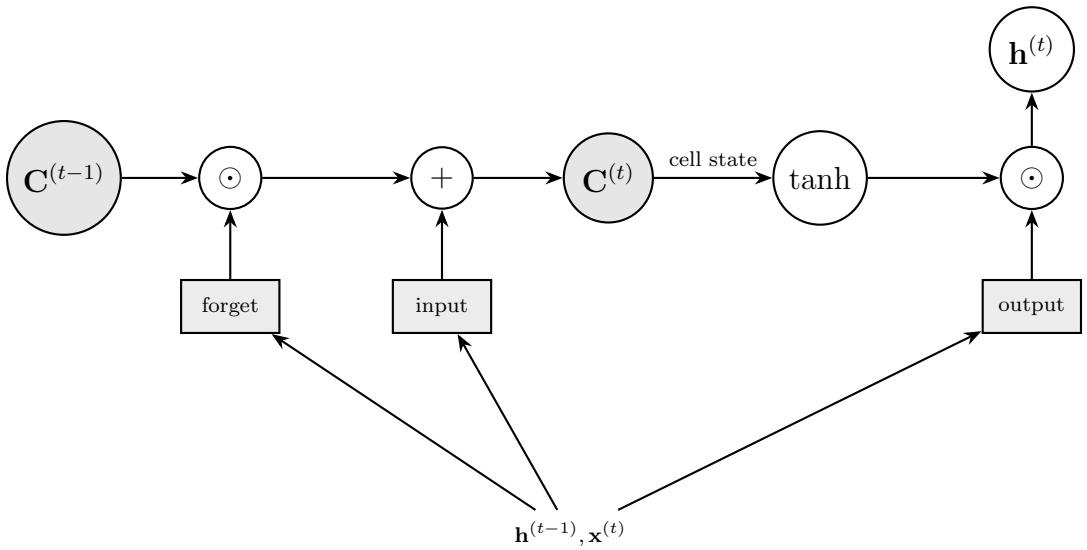


Figure 7: An LSTM cell. The cell state $\mathbf{C}^{(t)}$ flows along the top row with additive updates (forget gates the old cell state, input gates the new candidate). The right branch applies tanh to $\mathbf{C}^{(t)}$ and gates it with the output gate to produce the hidden state $\mathbf{h}^{(t)}$.

Why the Cell State Solves Vanishing Gradients Consider the gradient of the loss with respect to the cell state at an earlier time step:

$$\frac{\partial L}{\partial \mathbf{C}^{(t-k)}} = \frac{\partial L}{\partial \mathbf{C}^{(t)}} \cdot \prod_{j=1}^{k} \mathbf{f}^{(t-j)}$$

Crucially, this product is of gate values $\mathbf{f}$ (between 0 and 1), not of a fixed weight matrix W_{hh}. The network can learn to set $\mathbf{f} \approx \mathbf{1}$ whenever it wants information to flow through long distances, preserving the gradient magnitude. When $\mathbf{f} = \mathbf{1}$ exactly, $\partial L/\partial \mathbf{C}^{(t-k)} = \partial L/\partial \mathbf{C}^{(t)}$ (perfect gradient preservation). This is fundamentally different from vanilla RNNs, where the gradient product is an unlearnable sequence of weight matrices.

Gate Activations Gates use sigmoid so their values are near 0 or 1 (binary-like decisions). The cell candidate and final hidden state use tanh to keep values in a reasonable range with strong gradients during backprop.

2.4.5 Gated Recurrent Unit (GRU)

The GRU (Cho et al., 2014) is a simplified version of the LSTM with fewer parameters (no separate cell state and no output gate). Performance is similar to LSTM on most tasks, but GRU tends to perform slightly better on small datasets.

GRU Cell: Full Equations

Update gate — controls the mix of old and new state:

$$\mathbf{z}^{(t)} = \sigma\big(W_z\big[\mathbf{h}^{(t-1)};\, \mathbf{x}^{(t)}\big] + \mathbf{b}_z\big)$$

Reset gate — controls how much of the previous state to use when computing the candidate:

$$\mathbf{r}^{(t)} = \sigma\big(W_r\big[\mathbf{h}^{(t-1)};\, \mathbf{x}^{(t)}\big] + \mathbf{b}_r\big)$$

Candidate hidden state:

$$\tilde{\mathbf{h}}^{(t)} = \tanh\big(W_h\big[\mathbf{r}^{(t)} \odot \mathbf{h}^{(t-1)};\, \mathbf{x}^{(t)}\big] + \mathbf{b}_h\big)$$

Final hidden state update (the update gate simultaneously acts as forget and input gate):

$$\boxed{\mathbf{h}^{(t)} = \big(1 - \mathbf{z}^{(t)}\big) \odot \mathbf{h}^{(t-1)} + \mathbf{z}^{(t)} \odot \tilde{\mathbf{h}}^{(t)}}$$

The GRU combines the LSTM's forget and input gates into a single update gate $\mathbf{z}$, and merges the cell state into the hidden state. It has approximately 2/3 as many parameters as an LSTM.

2.4.6 RNN Architectures and Attention

Bidirectional RNNs A unidirectional RNN's hidden state $\mathbf{h}^{(t)}$ captures information only from the past $(\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(t)})$. For some tasks (e.g., speech recognition, handwriting recognition, named entity recognition), information from the future is also useful. A bidirectional RNN runs two RNNs in parallel: one forward and one backward, then concatenates their hidden states.

Encoder–Decoder (Seq2Seq) For tasks where the input and output sequences have different lengths (machine translation, dialogue):

- **Encoder**: an RNN that processes the input sequence $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n_x)}$ and produces a context vector $\mathbf{c}$ (typically the final hidden state).
- **Decoder**: an RNN conditioned on $\mathbf{c}$ that generates the output sequence one token at a time: $P(\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(n_y)} \mid \mathbf{c}) = \prod_{t=1}^{n_y} P(\mathbf{y}^{(t)} \mid \mathbf{y}^{(<t)}, \mathbf{c})$.

This architecture has a fundamental limitation: the context vector $\mathbf{c}$ is fixed-length and must compress all information about the input sequence into a single vector. For long input sequences, this bottleneck causes information loss.

Attention (Bahdanau et al., 2015)

Attention Mechanism

Instead of a single fixed context $\mathbf{c}$, attention computes a dynamic context vector $\mathbf{c}^{(t)}$ for each decoder time step:

$$\mathbf{c}^{(t)} = \sum_{j=1}^{n_x} \alpha_{t,j} \cdot \mathbf{h}_j$$

where $\mathbf{h}_j$ is the encoder hidden state at input position j, and $\alpha_{t,j}$ is an attention weight indicating how much input position j should be attended to when generating output token t.
The weights $\alpha_{t,j}$ are computed by comparing the query (previous decoder state $\mathbf{h}_{t-1}$) with each key (encoder state $\mathbf{h}_j$) via a compatibility function, then applying softmax:

$$e_{t,j} = \mathbf{h}_j^\top \mathbf{h}_{t-1}, \qquad \alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^{n_x} \exp(e_{t,k})}$$

This is **dot-product attention**. The value retrieved at each position is $\mathbf{h}_j$ (same as the key in this simple version).

Attention allows the decoder to "look back" at the input sequence at every time step, removing the fixed-length bottleneck. Unlike standard neural network weights (static), attention weights $\alpha_{t,j}$ are dynamic — they depend on both the input sequence and the current decoder state. This mechanism later evolved into the **Transformer** architecture, which replaces RNNs entirely with self-attention.

Teacher Forcing During seq2seq training, the decoder is typically trained with **teacher forcing**: after predicting $\hat{\mathbf{y}}^{(t)}$, the ground-truth token $\mathbf{y}^{(t)}$ is fed as input for the next step (not the model's own prediction $\hat{\mathbf{y}}^{(t)}$). This accelerates training and prevents error accumulation in early steps. However, it creates a train–test mismatch (exposure bias): at inference time the model must use its own predictions, which may not match the training distribution.

3 Attention-Based Architectures

3.1 Self-Attention and the Transformer

The Transformer (Vaswani et al., 2017, "Attention is All You Need") is a sequence-to-sequence model based solely on attention, dispensing entirely with recurrence. This allows significantly more parallelization during training, resulting in faster training and better accuracy than RNN-based Seq2Seq models.

3.1.1 Self-Attention Layer

Motivation Consider the sentence: "The animal didn't cross the street because it was too tired." What does "it" refer to — the street or the animal? Self-attention enables the model to associate "it" with "animal" when obtaining a new representation for "it." It improves the representation of each token by combining information from all other tokens in the sequence.

Queries, Keys, and Values Let $\mathbf{x}_1, \ldots, \mathbf{x}_n$ be the current token representations (row vectors of dimension d_m, typically $d_m = 512$). We introduce three learnable projection matrices:

- W^Q: a $d_m \times d_k$ matrix ($d_k = 64$)
- W^K: a $d_m \times d_k$ matrix
- W^V: a $d_m \times d_v$ matrix ($d_v = 64$)

For each token i, we project to obtain:

$$\mathbf{q}_i = \mathbf{x}_i W^Q \quad \text{(query)}, \qquad \mathbf{k}_j = \mathbf{x}_j W^K \quad \text{(key)}, \qquad \mathbf{v}_j = \mathbf{x}_j W^V \quad \text{(value)}$$

Scaled Dot-Product Attention

The new representation $\mathbf{z}_i$ for token i is computed in three steps:

1. Compute raw attention scores (dot-product): $\alpha_{i,j} \leftarrow \mathbf{q}_i \mathbf{k}_j^\top$
2. Scale: $\alpha_{i,j} \leftarrow \dfrac{\alpha_{i,j}}{\sqrt{d_k}}$
3. Apply softmax normalization: $\alpha_{i,j} \leftarrow \dfrac{e^{\alpha_{i,j}}}{\sum_{j'} e^{\alpha_{i,j'}}}$
4. Weighted sum of values: $\mathbf{z}_i = \sum_j \alpha_{i,j}\, \mathbf{v}_j$

In matrix notation, let X be the matrix with $\mathbf{x}_j$ as row vectors. Then:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

and the full output is:

$$\boxed{Z = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V}$$

The softmax is applied row-wise.

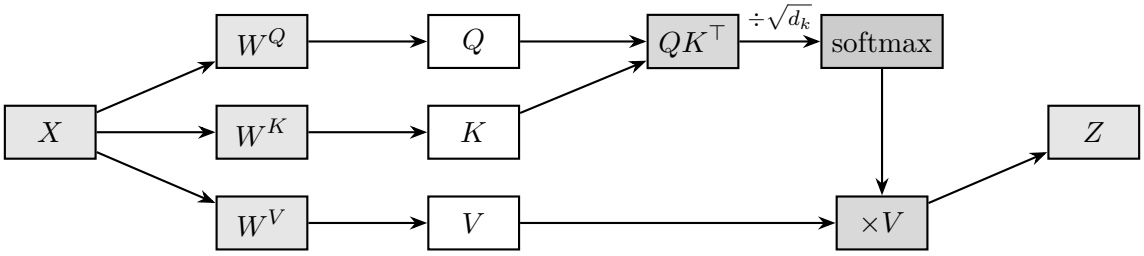


Figure 8: Scaled dot-product attention: input X is projected into queries Q, keys K, and values V. The attention weights $\text{softmax}(QK^\top/\sqrt{d_k})$ are applied to V to produce the output Z.

Why the $\sqrt{d_k}$ Scaling? The dot product $\mathbf{q}_i \mathbf{k}_j^\top$ has variance proportional to d_k. As d_k grows, the dot-product values become very large, pushing the softmax function into

regions where it has extremely small gradients. Scaling by $1/\sqrt{d_k}$ keeps the variance of the arguments to softmax near 1, ensuring stable gradients regardless of the embedding dimension.

Variance Analysis of Dot Product

Let $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{d_k}$ have i.i.d. entries with $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$ and $\mathrm{Var}(q_i) = \mathrm{Var}(k_i) = \sigma^2$. Then:

$$\mathbb{E}[\mathbf{q}^\top \mathbf{k}] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i] = \sum_{i=1}^{d_k} \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$$

For the variance:

$$\mathrm{Var}(\mathbf{q}^\top \mathbf{k}) = \sum_{i=1}^{d_k} \mathrm{Var}(q_i k_i) = d_k \cdot \mathrm{Var}(q_i k_i)$$

Since $\mathrm{Var}(XY) = \mathrm{Var}(X)\mathrm{Var}(Y) + \mathrm{Var}(X)\mathbb{E}[Y]^2 + \mathbb{E}[X]^2\mathrm{Var}(Y) = \sigma^4$ under zero mean:

$$\mathrm{Var}(\mathbf{q}^\top \mathbf{k}) = d_k \sigma^4$$

So the dot product variance is proportional to d_k. Scaling by $1/\sqrt{d_k}$ makes the variance independent of the embedding dimension, keeping softmax inputs in a numerically stable regime regardless of d_k.

3.1.2 Multi-Head Attention

A single attention head combines information from other tokens in one way. Multi-head attention uses h parallel heads to let the model attend to information from different representation subspaces simultaneously.

For each head $i = 1, \ldots, h$, we have its own set of projection matrices W_i^Q, W_i^K, W_i^V (each $d_m \times d_k$, with $d_k = d_m/h$):

$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V, \qquad Z_i = \mathrm{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i$$

The outputs from all heads are concatenated and projected back to dimension d_m:

$$\boxed{Z = \mathrm{Concat}(Z_1, Z_2, \ldots, Z_h)\, W^O}$$

where W^O is a learnable $(hd_v) \times d_m$ matrix.

In the original Transformer: $d_m = 512$, $h = 8$, $d_k = d_v = 64$.

3.1.3 Positional Encoding

The Transformer contains no recurrence or convolutions, so it has no inherent notion of token order. To inject positional information, we add a positional encoding vector to each input embedding.

Sinusoidal Positional Encoding

For a token at position pos, its PE is a vector of dimension d_m:

$$\boxed{\mathrm{PE}(pos, 2i) = \sin\left(pos/10000^{2i/d_m}\right), \quad \mathrm{PE}(pos, 2i+1) = \cos\left(pos/10000^{2i/d_m}\right)}$$

for $0 \le i < d_m/2$. Different dimensions of the PE correspond to sinusoids of different wavelengths (from 2π to $10000 \cdot 2\pi$). This allows the model to learn relative positions from absolute positions via linear transformations — a property that helps generalize

to sequence lengths longer than those seen during training.
The positional encodings are added to the input embeddings at the bottom of the encoder/decoder stacks.

3.1.4 Encoder

The encoder is a stack of $N = 6$ identical encoder blocks (no weight sharing across blocks). Each block has two sub-layers:

1. Multi-Head Self-Attention
2. Position-wise Feed-Forward Network (FFN)

A **residual connection** wraps around each sub-layer, followed by **Layer Normalization**:

$$\boxed{\mathbf{h}' = \text{LayerNorm}(\mathbf{h} + \text{Sublayer}(\mathbf{h}))}$$

- **LayerNorm** normalizes across features within a single training example (not across a batch), unlike Batch Norm. It is preferred in NLP because of variable-length sequences, small effective batch sizes, and the need for consistent behavior at inference time.
- **Pre-Norm vs. Post-Norm**: The original Transformer used post-norm (normalize after adding the residual). For deeper models, pre-norm (normalize before the heavy computation, leaving the residual path untouched) is more stable and avoids the need for learning-rate warm-up. Almost all modern LLMs (GPT-2/3, LLaMA, Falcon) use Pre-Norm.
- The FFN sub-layer applies the same two-layer MLP (with ReLU/GELU activation) to each position independently — no weight sharing across positions, but the same parameters are used for every position in the sequence.

3.1.5 Decoder

The decoder is a stack of $N = 6$ identical decoder blocks. Each decoder block has three sub-layers (each with residual connections + LayerNorm):

1. **Masked (Causal) Self-Attention**: each position may attend only to itself and preceding positions, never to future positions. Implementation: for $j > i$, set the raw attention score $\alpha_{i,j} = -\infty$ before applying softmax, forcing $\alpha_{i,j} = 0$.
2. **Cross-Attention** (or Decoder-Encoder Attention): queries come from the previous decoder layer, while keys and values come from the output of the encoder. This lets each decoder position attend to all positions in the input sequence.
3. Position-wise FFN.

The decoder generates one token at a time in an autoregressive fashion. Its output is fed through a linear layer and softmax to produce a probability distribution over the vocabulary for the next token.

3.1.6 BERT: Transformer Encoder for Representation Learning

BERT Architecture

BERT (Bidirectional Encoder Representations from Transformers, Devlin et al., 2018) is a language representation model consisting of a stack of Transformer encoder blocks only.

- **BERT-base**: $N = 12$ layers, $d_m = 768$, $h = 12$ heads, 110M parameters
- **BERT-large**: $N = 24$ layers, $d_m = 1024$, $h = 16$ heads, 340M parameters

BERT is "bidirectional" because the self-attention layer lets each token attend to all other tokens in both directions.

BERT Input BERT takes a pair of sentences (A, B) as input. The sequence is:

$$\texttt{CLS} \text{ tokens of } A \texttt{ SEP} \text{ tokens of } B \texttt{ SEP}$$

where `CLS` is the classification token (prepended to the sequence) and `SEP` is the separator token. Three embeddings are summed to form the input to BERT:

1. Token embedding (learned word embeddings)
2. Segment (token-type) embedding: E_A (all 0s) for sentence A, E_B (all 1s) for sentence B
3. Positional embedding (learned, unlike the sinusoidal PE in the original Transformer)

BERT Pre-Training Tasks BERT is pre-trained on unlabeled text (BooksCorpus + Wikipedia, 3.3 billion words) using two unsupervised tasks:

1. **Masked Language Model (MLM)**:
   - 15% of tokens in each sequence are selected for prediction
   - For each selected token: with 80% chance replaced by `MASK`, with 10% chance replaced by a random token, with 10% chance kept unchanged
   - The model must predict the original token using a cross-entropy loss
   - The 10% random and 10% unchanged replacements mitigate the pre-train/fine-tune mismatch: during fine-tuning, no `MASK` tokens appear
2. **Next Sentence Prediction (NSP)**:
   - 50% of the time, sentence B actually follows sentence A (label: IsNext)
   - 50% of the time, B is a random sentence from the corpus (label: NotNext)
   - The output representation C of the `CLS` token is fed to a classifier to predict this binary label

The total training loss is the sum of the MLM loss (over all masked positions) and the NSP loss.

BERT Fine-Tuning After pre-training, BERT can be fine-tuned for downstream tasks by adding task-specific output layers and fine-tuning all BERT parameters:

- **Sentence-level classification**: feed the `CLS` representation C to a one-layer classifier (e.g., MNLI entailment, sentiment analysis)
- **Token-level tasks**: feed each token's representation to an output layer (e.g., Named Entity Recognition, POS tagging)
- **Question Answering**: predict start/end positions of the answer span

Fine-tuning is relatively inexpensive; the original paper reports that all results can be replicated in under an hour on a single Cloud TPU.

3.2 Decoder-Only Transformers and Large Language Models

3.2.1 Decoder-Only Transformer Models

Generative LLMs are based on decoder-only Transformer architectures, which excel at autoregressive text generation. Notable examples include GPT-4 (OpenAI, 2023), Llama 2 (Meta, 2023), Mistral (Mistral AI, 2023), Gemini (Google, 2023), Claude 3 (Anthropic, 2024), and DeepSeek-v3 (DeepSeek AI, 2025).

Autoregressive Text Generation

A decoder-only Transformer models the probability of a sequence autoregressively:

$$p(\mathbf{x}) = \prod_{i=1}^{T} p(x_i \mid x_1, x_2, \ldots, x_{i-1})$$

At generation time:
1. The user provides a prompt (e.g., "The weather today is")
2. The model computes probabilities for all possible next tokens (e.g., "sunny": 0.6, "rainy": 0.3, "cloudy": 0.1)
3. One token is selected (via greedy search, beam search, or temperature sampling)
4. The process repeats until an end-of-sequence (EOS) token is generated

3.2.2 Three-Stage LLM Training Pipeline

The training pipeline for LLMs involves three key stages:

Stage 1: Pre-training

Train the model on massive, diverse text corpora (books, websites, articles) containing billions to trillions of tokens, using the **next-token prediction loss**:

$$\mathcal{L}_{\text{pretrain}} = -\sum_{i} \log p_\theta(x_i \mid x_{i-k}, \ldots, x_{i-1})$$

where k is the context-window size. The model learns general-purpose language representations. This stage is computationally dominant.

Stage 2: Supervised Fine-Tuning (SFT)

The pre-trained model is fine-tuned on labeled demonstration data $D = \{(x, y)\}$ where x is a prompt and y is a desired response:

$$\mathcal{L}_{\text{SFT}} = -\sum_{(x,y)\in D} \log p_\theta(y \mid x)$$

This adapts the model to specific tasks (summarization, question answering, instruction following). The result is called an **SFT model**.

Stage 3: Reinforcement Learning from Human Feedback (RLHF)

RLHF aligns the model's outputs with human preferences and values. It has two sub-steps:
(3a) Train a Reward Model. Collect human preference data $D = \{(x, y_w, y_l)\}$, where y_w is the preferred ("winner") response and y_l is the dispreferred ("loser") response to prompt x. Train a reward function $r(x, y)$ using the **Bradley–Terry model**:

$$P(y_w \succ y_l \mid x) = \frac{\exp(r(x, y_w))}{\exp(r(x, y_w)) + \exp(r(x, y_l))}$$

The reward model loss is:

$$\mathcal{L}(r) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\left[\log \frac{\exp(r(x, y_w))}{\exp(r(x, y_w)) + \exp(r(x, y_l))}\right]$$

(3b) PPO Fine-Tuning. Optimize the language model policy $\pi_\theta(y \mid x)$ against the reward model using Proximal Policy Optimization (PPO). The reinforcement learning objective is:

$$\mathcal{L}_{\text{RL}} = \mathbb{E}_{x \sim D}[\log \pi_\theta(y \mid x) \cdot r(x, y)]$$

This encourages the model to generate responses that the reward model (and hence humans) prefer.

Alternative: **Direct Preference Optimization (DPO)** (Rafailov et al.), which eliminates the separate reward model by directly optimizing the language model using preference data in a classification-style loss.

3.2.3 Using LLMs

- **In-Context Learning**: LLMs can perform tasks dynamically based on the context provided in the prompt, without updating any parameters. This is the basis for prompt engineering.
- **Zero-Shot / Few-Shot Prompting**: Provide no examples (zero-shot) or a few examples (few-shot) in the prompt to guide the output.
- **Chain-of-Thought (CoT) Prompting**: Encourage step-by-step reasoning by showing reasoning chains in the examples.
- **Retrieval-Augmented Generation (RAG)**: First retrieve relevant documents from an external knowledge base, then let the LLM generate responses grounded in those documents. Improves accuracy and reduces hallucination.
- **Hallucination**: LLMs may generate plausible-sounding but false information. Always verify outputs.
- **Jailbreaking**: Malicious users may craft prompts to bypass safety safeguards. Use robust moderation.

3.2.4 Scaling LLMs

- **Scaling Laws**: Model performance follows a power law with model size (number of parameters N) and training data size D (Kaplan et al., 2020):

$$\text{Performance} \propto N^\alpha D^\beta$$

- **Mixture of Experts (MoE)**: Replace dense FFNs with sparse expert networks. Only top-k experts are activated per input, keeping compute cost roughly constant while dramatically increasing total parameter count. A gating function $G(x) = \text{softmax}(W_g \cdot x + b_g)$ selects which experts to use:

$$\boxed{y = \sum_{i \in \text{Top-}k(G(x))} G_i(x) \cdot \text{FFN}_i(x)}$$

  Used by Switch Transformer, Mixtral 8x7B, DeepSeek-V3, etc.

3.3 Vision Transformer (ViT) and CLIP

3.3.1 Vision Transformer (ViT)

The Vision Transformer (Dosovitskiy et al., 2021, "An Image is Worth 16x16 Words") adapts the standard Transformer encoder to image classification. The core idea is remarkably simple: split an image into fixed-size patches, treat each patch as a "token," and feed the resulting sequence into a pure Transformer encoder.

ViT Pipeline

Given an input image $x \in \mathbb{R}^{H \times W \times C}$:

1. **Patchify**: Split the image into $N = HW/P^2$ non-overlapping patches of resolution $P \times P$ (default $P = 16$). Each patch $x_p \in \mathbb{R}^{P \times P \times C}$ is flattened into a vector of length P^2C.
2. **Patch Embedding**: Each flattened patch vector is projected to dimension d (the hidden size of the Transformer) through a learnable linear projection matrix $E \in \mathbb{R}^{(P^2C) \times d}$:
$$\mathbf{z}_p^{(0)} = x_p E, \qquad p = 1, \ldots, N$$
3. **Class Token**: Prepend a single learnable `CLS` token $\mathbf{z}_{\text{class}}^{(0)} \in \mathbb{R}^d$ to the patch sequence:
$$\boxed{\mathbf{z}^{(0)} = [\mathbf{z}_{\text{class}}^{(0)}; \mathbf{z}_1^{(0)}; \mathbf{z}_2^{(0)}; \ldots; \mathbf{z}_N^{(0)}]}$$
4. **Positional Embedding**: Add learnable (or sinusoidal) positional embeddings $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times d}$ to inject spatial information:
$$\mathbf{z}^{(0)} \leftarrow \mathbf{z}^{(0)} + E_{\text{pos}}$$
5. **Transformer Encoder**: Feed the sequence through L Transformer encoder blocks (Multi-Head Self-Attention + MLP with GELU, each with residual connections and LayerNorm).
6. **Classification Head**: Take the final `CLS` representation $\mathbf{z}_{\text{class}}^{(L)}$ and pass it through a linear classifier (head) producing logits over the label space.

All of E, E_{pos}, and $\mathbf{z}_{\text{class}}^{(0)}$ are learnable parameters.

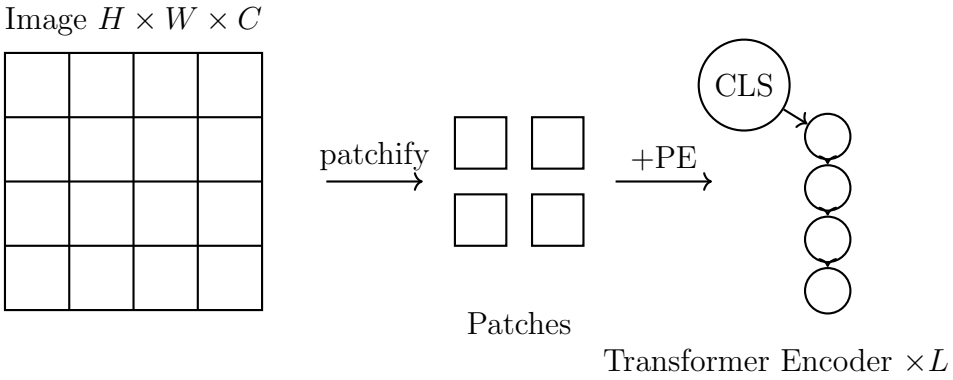


Figure 9: Vision Transformer pipeline: image is split into patches, each linearly projected and positioned, with a learnable CLS token prepended.

ViT vs. CNNs: Inductive Bias

- **CNNs** have strong built-in inductive bias: locality, 2-D neighborhood structure, and translation equivariance are baked into every layer.
- **ViTs** have much weaker inductive bias. Self-attention layers are global (any token attends to any other), and only the MLP sub-layers are local and translationally equivariant.
- **Consequence**: ViTs generally need more data to outperform CNNs. On small pre-training datasets, ViTs can perform worse than comparable CNNs. But on large datasets, ViTs surpass CNNs because their flexibility lets them learn richer patterns.

ViT Feature Maps Patch embeddings at any self-attention layer can be rearranged into a 3-D tensor $(H/P, W/P, d)$. A frontal slice (fixed d index) upsampled back to image size yields a heatmap called a ViT feature map. These maps are generally more semantically meaningful than ResNet feature maps.

Model Sizes

- ViT-Base (ViT-B/16): $L = 12$, $d = 768$, $h = 12$, ∼86M params, patch size 16×16
- ViT-Large (ViT-L/16): $L = 24$, $d = 1024$, $h = 16$, ∼307M params
- ViT-Huge (ViT-H/14): $L = 32$, $d = 1280$, $h = 16$, ∼632M params, patch size 14×14

Robustness ViTs have been shown to be more robust than CNNs to adversarial perturbations and out-of-distribution inputs (Naseer et al., 2021; Paul & Chen, 2022).

3.3.2 CLIP: Contrastive Language–Image Pre-training

CLIP (Radford et al., 2021, ICML) is a vision–language model that learns a joint embedding space where images and their paired text captions are close together. It is trained on 400 million image–text pairs scraped from the web.

CLIP Architecture: Dual-Tower Encoders

CLIP has two encoders that produce embeddings in a shared d-dimensional space:

1. **Image Encoder**: A Vision Transformer (or ResNet) that maps an image I to a vector $\mathbf{e}_I \in \mathbb{R}^d$
2. **Text Encoder**: A Transformer language model that maps a text caption T (e.g., "A photo of a cat.") to a vector $\mathbf{e}_T \in \mathbb{R}^d$

Both encoders end with a linear projection to ensure their outputs live in the same joint embedding space. The cosine similarity between an image embedding and a text embedding measures compatibility.

Contrastive Loss Given a batch of N image–text pairs $\{(I_i, T_i)\}$, the CLIP contrastive loss has two parts (symmetric):

1. **Image-to-Text**: for each image I_i, the paired text T_i should have the highest similarity among all texts in the batch.
2. **Text-to-Image**: for each text T_j, the paired image I_j should have the highest similarity among all images in the batch.

CLIP Contrastive Loss

Let $\text{sim}(I, T) = \mathbf{e}_I^\top \mathbf{e}_T / \tau$ be the temperature-scaled cosine similarity, where τ is a learnable temperature parameter. The full loss is:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(\text{sim}(I_i, T_i))}{\sum_{j=1}^{N} \exp(\text{sim}(I_i, T_j))} - \frac{1}{N}\sum_{j=1}^{N} \log \frac{\exp(\text{sim}(I_j, T_j))}{\sum_{i=1}^{N} \exp(\text{sim}(I_i, T_j))}$$

In practice, this is computed in matrix form: the similarity matrix $S \in \mathbb{R}^{N \times N}$ has diagonal entries $\text{sim}(I_i, T_i)$ and off-diagonal entries $\text{sim}(I_i, T_j)$ for $i \neq j$. All off-diagonal entries are treated as negative pairs for both rows (image-to-text) and columns (text-to-image).

Why the Symmetric Two-Way Loss

The two loss terms (image-to-text and text-to-image) come from treating CLIP as learning a joint embedding space. If we only trained the image-to-text direction, a text embedding could collapse to a constant and still minimize the loss (all images map to the same text prototype). The symmetric second term prevents this by forcing each text to be close to its paired image from all images. In InfoNCE form, the joint loss maximizes $\text{sim}(I_i, T_i)$ while minimizing $\text{sim}(I_i, T_j)$ and $\text{sim}(I_j, T_i)$ for $i \neq j$, yielding a well-structured joint space.

Zero-Shot Image Classification CLIP's most striking capability is zero-shot transfer to new classification tasks without any gradient updates. For a target dataset with classes $\{c_1, c_2, \ldots, c_K\}$:

1. Design a text prompt for each class: $T_c =$ "A photo of a c."
2. Encode each prompt to obtain $\mathbf{e}_{T_c}$ for each class c
3. Given a new image I, encode it to $\mathbf{e}_I$ and compute class probabilities via softmax over cosine similarities:

$$P(c \mid I) = \frac{\exp(\mathbf{e}_I^\top \mathbf{e}_{T_c})}{\sum_{c'=1}^{K} \exp(\mathbf{e}_I^\top \mathbf{e}_{T_{c'}})}$$

Why Zero-Shot Works

The text prompts T_c act as soft classifiers — their embeddings $\mathbf{e}_{T_c}$ serve as prototype vectors for each class. An image's class is simply its nearest text prototype in the joint embedding space. Since this space was trained to align visual and semantic concepts from 400M diverse web pairs, it generalizes to unseen class names.

Performance and Robustness

- On ImageNet, zero-shot CLIP is competitive with a fully supervised linear probe trained on ImageNet itself — without ever seeing any ImageNet labels.
- A linear probe on CLIP image features outperforms the best ImageNet models.
- Zero-shot CLIP is much more robust to natural distribution shift than standard ImageNet models, shrinking the robustness gap by up to 75%.

Scaling CLIP

- ALIGN (Jia et al., 2020): 1.8B noisy image–text pairs
- BASIC (Pham et al., 2023): 6.6B image–text pairs with bigger model sizes
- Larger data and model sizes consistently improve zero-shot transfer performance

4 Classical Machine Learning Methods

4.1 Support Vector Machines

Motivation

Support Vector Machine (SVM) is an optimization method for classification. It builds mainly on three ideas: **large margin classification**, **regularization** (for data not linearly separable), and **feature transformation and kernels** (to go beyond linear classifiers). SVM was a popular classification method widely used before the rise of deep learning.

This set of notes is based on Ethem Alpaydin (2020). Introduction to Machine Learning. (Chapter 14)

4.1.1 Problem Statement

Training set: $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$, where $y_i \in \{-1, +1\}$.

Learn a linear classifier:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

where $\mathbf{x} = (x_1, \ldots, x_D)^\top$ and $\mathbf{w} = (w_1, \ldots, w_D)^\top$.

- First, deal with linearly separable data: **Hard-Margin Support Vector Machine**
- Then, data that are not linearly separable: **Soft-Margin Support Vector Machine**

4.1.2 Hard-Margin Support Vector Machine

Multiple Solutions For a linearly separable dataset, there are many hyperplanes that correctly separate the two classes. Which one should we choose?

Optimal Separating Hyperplane The **margin** of a separating hyperplane is the distance to the hyperplane from the data point closest to it. From Structural Risk Minimization (SRM) theory, the separating hyperplane with the largest margin generalizes best (i.e., has the smallest expected generalization error).

Optimal Margin Classifier

The **Optimal Margin Classifier** maximizes the minimum distance between any training point and the decision boundary. This is also called the linear support vector machine.
Support Vectors are those data points that the margin planes push up against. Only these points define the optimal $\mathbf{w}$; all other points could be moved around (without crossing the margin) without changing the solution.

Canonical Separating Hyperplanes Decision boundary:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

The **plus-plane** touches some positive examples and is parallel to the decision boundary: $\mathbf{w}^\top \mathbf{x} + b = c$ for some constant c.

The **minus-plane** touches some negative examples. Because the decision boundary is halfway between the two margin planes, it has equation $\mathbf{w}^\top \mathbf{x} + b = -c$.

Dividing both equations by c, we get the **canonical** form (renaming $\mathbf{w}/c$ as $\mathbf{w}$ and b/c as b):

$$\mathbf{w}^\top \mathbf{x} + b = 0 \quad \text{(decision boundary)} \tag{5}$$
$$\mathbf{w}^\top \mathbf{x} + b = 1 \quad \text{(plus-plane)} \tag{6}$$
$$\mathbf{w}^\top \mathbf{x} + b = -1 \quad \text{(minus-plane)} \tag{7}$$

The vector $\mathbf{w}$ is perpendicular to all three planes, because for any two points $\mathbf{u}$ and $\mathbf{v}$ on the same plane, $\mathbf{w}^\top(\mathbf{u} - \mathbf{v}) = 0$.

Constraints Satisfied by Training Examples For each training example $(\mathbf{x}_i, y_i)$:

$$\mathbf{w}^\top \mathbf{x}_i + b \geq 1 \quad \text{if } y_i = +1, \qquad \mathbf{w}^\top \mathbf{x}_i + b \leq -1 \quad \text{if } y_i = -1$$

Or equivalently:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, N$$

What Is the Margin? Let $\mathbf{x}^-$ be a point on the minus-plane and $\mathbf{x}^+$ be the point on the plus-plane closest to $\mathbf{x}^-$. Since the line connecting them is perpendicular to all three planes:

$$\mathbf{x}^+ - \mathbf{x}^- = \lambda \mathbf{w} \quad \text{for some constant } \lambda$$

Using the plane equations (6)–(7):

$$\mathbf{w}^\top \mathbf{x}^+ + b = 1, \qquad \mathbf{w}^\top \mathbf{x}^- + b = -1$$

Subtracting: $\mathbf{w}^\top(\mathbf{x}^+ - \mathbf{x}^-) = 2 \Rightarrow \lambda \mathbf{w}^\top \mathbf{w} = 2 \Rightarrow \lambda = \dfrac{2}{\mathbf{w}^\top \mathbf{w}}$.

Therefore the margin is:

$$M = \|\mathbf{x}^+ - \mathbf{x}^-\| = \|\lambda \mathbf{w}\| = \lambda \|\mathbf{w}\| = \frac{2}{\|\mathbf{w}\|}$$

Primal Optimization Problem Maximizing the margin $\dfrac{2}{\|\mathbf{w}\|}$ is equivalent to minimizing $\|\mathbf{w}\|^2$.

Hard-Margin SVM Primal Problem

Find $\mathbf{w}$ and b to:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i$$

This is a **convex quadratic programming (QP)** problem. There is one linear constraint per training point.

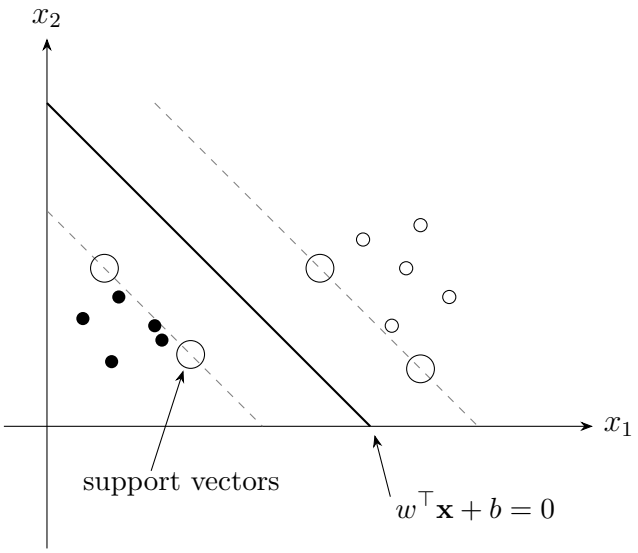


Figure 10: Hard-margin SVM: the thick black line is the maximum-margin hyperplane, and the two dashed lines are parallel margins at equal distance from it. Filled circles = class +1, empty = class −1; the ringed points are the support vectors, which lie exactly on the margins.

The Lagrangian and Duality The Lagrangian of the primal problem is:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{N} \alpha_i \big(y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1\big)$$

where $\alpha_i \geq 0$ are the Lagrange multipliers (one per constraint).

The **primal** problem is equivalent to:

$$\min_{\mathbf{w},b} \max_{\boldsymbol{\alpha}:\alpha_i \geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$$

The **dual** problem swaps the order:

$$\max_{\boldsymbol{\alpha}:\alpha_i \geq 0} \min_{\mathbf{w},b} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$$

Why the Dual Matters

The primal QP has complexity that depends on the dimensionality D of the input space. The dual QP has complexity $O(N^3)$ that depends on the sample size N. For high-dimensional data, the dual is much more efficient. Moreover, the dual formulation reveals the support vector structure and allows the **kernel trick** to be applied.

Karush–Kuhn–Tucker (KKT) Conditions When the training data are linearly separable, strong duality holds: $d^* = p^*$, and the optimal solution $(\mathbf{w}^*, b^*, \boldsymbol{\alpha}^*)$ satisfies the KKT conditions:

$$\frac{\partial \mathcal{L}}{\partial w_j} = 0, \qquad \frac{\partial \mathcal{L}}{\partial b} = 0,$$

$$\alpha_i \big[y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) - 1\big] = 0, \qquad y_i(\mathbf{w}^{*\top} \mathbf{x}_i + b^*) - 1 \geq 0, \qquad \alpha_i \geq 0$$

The third condition is the **complementarity condition**: either $\alpha_i = 0$ (constraint is inactive, the point is not a support vector) or $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$ (constraint is active, the point is a support vector lying exactly on a margin plane).

Support Vectors

Support Vectors

At the optimal solution, the KKT complementarity condition $\alpha_i[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1] = 0$ gives:

- If $y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 1$ (point is inside its class margin): then $\alpha_i = 0$. This point does not affect the solution.
- If $\alpha_i > 0$: then necessarily $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$. This point lies exactly on a margin plane and is called a **support vector**.

The optimal $\mathbf{w}$ is determined solely by the support vectors. Removing non-support vectors from the training set would yield the same solution.

Solving the Dual Problem For fixed $\boldsymbol{\alpha}$, minimizing $\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$ over $\mathbf{w}$ and b gives the closed-form solution:

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i, \qquad \sum_{i=1}^{N} \alpha_i y_i = 0$$

Substituting back into the Lagrangian, the dual problem becomes:

Hard-Margin SVM Dual Problem

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{subject to} \quad \alpha_i \geq 0, \quad \sum_{i=1}^{N} \alpha_i y_i = 0$$

This is also a QP problem, with complexity $O(N^3)$ depending on sample size N rather than input dimensionality D.

Finding the Bias b From the KKT condition $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$ for support vectors, multiplying both sides by y_i:

$$\mathbf{w}^\top \mathbf{x}_i + b = y_i \implies b = y_i - \mathbf{w}^\top \mathbf{x}_i$$

For numerical stability, average over all support vectors:

$$b = \frac{1}{N_S} \sum_{i=1}^{N_S} \big(y_i - \mathbf{w}^\top \mathbf{x}_i\big)$$

where N_S is the number of support vectors.

Prediction For a new example $\mathbf{x}$, the classification rule is:

$$\hat{y} = \text{sign}\left(\mathbf{w}^\top \mathbf{x} + b\right) = \text{sign}\left(\sum_{i=1}^{N_S} \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b\right)$$

Interpretation:

- $\mathbf{x}_i^\top \mathbf{x}$ measures the similarity between $\mathbf{x}$ and support vector $\mathbf{x}_i$ (inner product).
- α_i controls the influence of support vector $\mathbf{x}_i$.
- $y_i \in \{-1, +1\}$ casts a positive or negative vote.

Multi-class Extension One common approach handles K classes by defining K two-class problems, each separating one class from all others combined. Let $g_i(\mathbf{x}) = \mathbf{w}_i^\top \mathbf{x} + b_i$ be the discriminant function for class i. The classification rule is:

$$\hat{y} = \arg\max_i g_i(\mathbf{x})$$

4.1.3 Soft-Margin Support Vector Machine

When data are not linearly separable, the hard-margin constraint $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ is violated for at least one example. We relax the constraints by introducing **slack variables**.

Slack Variables Introduce positive slack variables $\xi_i \geq 0$:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$$

- $\xi_i = 0$: point satisfies the original constraint, lies on or beyond its margin plane.
- $0 < \xi_i \leq 1$: point lies inside the margin but is still classified correctly.
- $\xi_i > 1$: point is misclassified.

The sum $\sum_i \xi_i$ is an upper bound on the number of training errors. We penalize it in the objective.

Soft-Margin SVM Primal Problem

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

The constant $C > 0$ controls the trade-off:

- **Small** C: wider margin allowed, many misclassified or margin-violating points tolerated. Simpler model, possible underfitting.
- **Large** C: hard margin (narrow margin enforced), few misclassifications tolerated. Complex model, possible overfitting.

Lagrangian and Dual The Lagrangian for the soft-margin problem introduces additional multipliers $r_i \geq 0$ for the $\xi_i \geq 0$ constraints:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \mathbf{r}) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i - \sum_{i=1}^{N}\alpha_i\big(y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1 + \xi_i\big) - \sum_{i=1}^{N} r_i\xi_i$$

Minimizing over $\mathbf{w}$, b, $\boldsymbol{\xi}$ gives the same $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ and $\sum_i \alpha_i y_i = 0$, plus the new condition:

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \implies C - \alpha_i - r_i = 0 \implies \alpha_i \leq C$$

Since $r_i \geq 0$, this gives the upper bound $\alpha_i \leq C$ on the dual variables.

Soft-Margin SVM Dual Problem

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{N}\alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{subject to} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{N}\alpha_i y_i = 0$$

Compare with the hard-margin dual: the only difference is the upper bound $\alpha_i \leq C$.

Types of Support Vectors With the soft-margin, there are three types of points at optimality:

- $\alpha_i = 0$: constraint inactive, point lies strictly outside its margin plane. Irrelevant to the solution.
- $0 < \alpha_i < C$: constraint active, $\xi_i = 0$, point lies exactly on its margin plane. These are the canonical support vectors that define the margin.
- $\alpha_i = C$: constraint active, $\xi_i > 0$, point is either inside the margin or misclassified. These points "pull" the margin boundary.

4.1.4 Relationship with the Hinge Loss

The soft-margin primal is:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i, \quad \text{subject to} \quad \xi_i \geq 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b), \quad \xi_i \geq 0$$

Since the objective minimizes ξ_i, at optimality each ξ_i takes its smallest feasible value:

$$\xi_i = \max\big(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big)$$

SVM as Regularized Hinge Loss

The SVM objective can be rewritten without constraints as:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \underbrace{\max\big(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big)}_{\text{hinge loss at } (\mathbf{x}_i, y_i)}$$

So SVM is **linear classification with L2 regularization and hinge loss**. The term $\frac{1}{2}\|\mathbf{w}\|^2$ is the L2 penalty that keeps the margin large, and C controls the regularization strength.

4.1.5 Kernel Extension for Nonlinear Models

Linear SVMs can only learn linear decision boundaries. For more complex patterns, we need a way to produce nonlinear decision boundaries while keeping the optimization tractable.

Feature Transformation Idea: map each input $\mathbf{x} \in \mathbb{R}^D$ to a higher-dimensional feature space via a function $\phi : \mathbb{R}^D \to \mathcal{H}$, then run a linear SVM in that feature space:

$$(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N) \implies (\phi(\mathbf{x}_1), y_1), \ldots, (\phi(\mathbf{x}_N), y_N)$$

Linear separation in the feature space corresponds to nonlinear separation in the original input space. The higher the dimensionality of $\mathcal{H}$, the more likely the data become linearly separable.

Example: XOR Problem Data (2D): $(-1,-1;-1)$, $(-1,1;+1)$, $(1,-1;+1)$, $(1,1;-1)$ — not linearly separable.

Transform via $\phi(x_1, x_2) = (x_1, x_2, x_1x_2)$:

$(-1,-1) \mapsto (-1,-1,1)$ with label -1, $\quad (-1,1) \mapsto (-1,1,-1)$ with label $+1$,
$(1,-1) \mapsto (1,-1,-1)$ with label $+1$, $\quad (1,1) \mapsto (1,1,1)$ with label -1.

Now linearly separable: the plane $x_1x_2 = 0$ separates the classes.

Linear SVM in Feature Space The dual problem in feature space $\mathcal{H}$ is:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} \alpha_i\alpha_j y_i y_j \, \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) \quad \text{subject to} \quad 0 \le \alpha_i \le C, \quad \sum_{i=1}^{N} \alpha_i y_i = 0$$

The prediction rule becomes:

$$\hat{y} = \operatorname{sign}\left(\sum_{i=1}^{N_S} \alpha_i y_i \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}) + b\right)$$

The Kernel Trick

Kernel Function and Kernel Trick

Define a **kernel function** as the inner product in feature space:

$$K(\mathbf{x}_i, \mathbf{x}_j) \;=\; \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

Then any occurrence of $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ in the SVM dual or prediction rule can be replaced

by $K(\mathbf{x}_i, \mathbf{x}_j)$ — without ever computing ϕ explicitly!

Why This Matters

We can specify a kernel $K(\mathbf{x}_i, \mathbf{x}_j)$ that corresponds to an extremely high-dimensional (even infinite-dimensional) feature space, yet compute the kernel itself cheaply. The kernel trick lets SVMs learn nonlinear decision boundaries with essentially the same computational cost as the linear case.

Dual Problem in Terms of Kernel

Kernel SVM Dual Problem

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \, K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{subject to} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{N} \alpha_i y_i = 0$$

Decision boundary:

$$\boxed{f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^{N_S} \alpha_i y_i \, K(\mathbf{x}_i, \mathbf{x}) + b \right)}$$

Common Kernels

Kernel Functions

- **Linear kernel**:
$$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$$
Corresponds to $\phi(\mathbf{x}) = \mathbf{x}$. This is just the original linear SVM.
- **Polynomial kernel**:
$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^d$$
For $d = 2$ and $c = 0$, the corresponding feature map expands all pair-wise products of input features. For example, $\phi(x_1, x_2) = (x_1^2, x_1 x_2, x_2^2)$ when $d = 2$.
- **Radial Basis Function (RBF) / Gaussian kernel**:
$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2} \right) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{z}\|^2 \right), \quad \gamma = \frac{1}{2\sigma^2}$$
The RBF kernel corresponds to an infinite-dimensional feature space. Each support vector acts as a "center" and $K(\mathbf{x}_i, \mathbf{x})$ is the "similarity" between $\mathbf{x}$ and $\mathbf{x}_i$. Small γ (large σ): smooth, simple decision boundary (low variance, high bias). Large γ (small σ): wiggly, complex boundary (high variance, low bias, overfitting risk).

Choosing C and γ

Both C and γ are critical hyperparameters:
- **Small** C: wide margin, allows more misclassification, simpler model.
- **Large** C: narrow margin, penalizes misclassification heavily, more complex model.
- **Small** γ: wide Gaussian bell curve, each support vector affects a large neighborhood, smooth decision boundary.
- **Large** γ: narrow Gaussian bell curve, each support vector affects only nearby points, complex decision boundary.

These are commonly selected via cross-validation over a grid (e.g., $C \in \{10^{-3}, \ldots, 10^3\}$ and $\gamma \in \{10^{-3}, \ldots, 10^3\}$).

Classification with Kernel SVM The full pipeline for kernel SVM classification:

1. Choose a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ (implicitly defines ϕ).
2. Solve the dual QP to get optimal $\boldsymbol{\alpha}$.
3. Compute b from support vectors: $b = \frac{1}{N_S} \sum_{i=1}^{N_S} (y_i - \sum_{j=1}^{N_S} \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i))$.
4. Classify new examples using $f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^{N_S} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \right)$.

When to Use SVM Instead of Deep Learning?

- **Small datasets**: SVM performs well with limited samples; deep learning needs large data.
- **When training time must be short**: SVM trains faster than deep networks for small/medium data.
- **When interpretability matters**: SVMs are simpler and easier to analyze than deep nets.
- **Complex decision boundary but not huge data**: kernel tricks give SVM strong nonlinear power.
- **High overfitting risk**: SVM's margin maximization naturally controls overfitting better on small data.

4.2 Decision Trees and Random Forests

Motivation

Decision trees are one of the earliest machine learning algorithms, designed for tabular data. They are nonparametric (no assumption on data distribution), highly interpretable (clear if-then rules), and work well with small to medium-sized datasets. Random Forests are ensemble methods built on decision trees that dramatically improve generalization by reducing variance.
This set of notes is based on Ethem Alpaydin (2020). Introduction to Machine Learning. (Chapter 9)

4.2.1 Decision Trees

Basic Structure A decision tree consists of:

- **Internal nodes**: each tests an attribute (feature). Branches correspond to attribute values.
- **Leaf nodes**: each assigns a class label.
- A path from the root to a leaf is a conjunction of attribute tests — an if-then decision rule.

Decision trees can be used for both classification and regression; here we focus on classification. For continuous attributes, a threshold is specified to produce a binary split.

Example: Play-Tennis Dataset A classic illustrative dataset: decide whether to play tennis given weather attributes. The tree produces rules like:

- If outlook is `Sunny` AND humidity is `Normal` ⇒ Play
- Else if outlook is `Overcast` ⇒ Play
- Else if outlook is `Rain` AND wind is `Weak` ⇒ Play
- Else ⇒ Don't play

4.2.2 Decision Tree Learning Algorithm

The goal of tree learning is to recursively partition the feature space into regions that are as "pure" as possible with respect to the target label.

Attribute Selection: Maximum Reduction in Impurity At each node, we need to select which attribute to split on. The best attribute is the one that leads to the **maximum reduction in impurity**. Many impurity measures exist; we introduce the two most common.

Entropy

Entropy

Entropy is a common impurity or randomness measure. Let a node have K classes and let p_i denote the fraction of examples belonging to class i, with $\sum_{i=1}^{K} p_i = 1$.

The **entropy** of the node is:

$$H(S) = -\sum_{i=1}^{K} p_i \log_2 p_i$$

where we adopt the convention $0 \cdot \log_2 0 = 0$. Entropy is measured in bits.

- When all examples belong to one class: $H(S) = 0$ (perfectly pure, no uncertainty).
- When classes are uniformly distributed ($p_i = 1/K$): $H(S) = \log_2 K$ (maximum uncertainty).

For binary classification ($K = 2$) with fraction p of the positive class:

$$H(S) = -p \log_2 p - (1-p) \log_2 (1-p)$$

Information Gain After splitting the data S on attribute A into V subsets $\{S_v\}_{v=1}^{V}$, the **information gain** (expected reduction in impurity) of that split is:

Information Gain

$$\text{Gain}(S, A) = H(S) - \sum_{v=1}^{V} \frac{|S_v|}{|S|} H(S_v)$$

The term $\sum_v \frac{|S_v|}{|S|} H(S_v)$ is the weighted average entropy of the child nodes, weighted by their relative sizes. The information gain is always non-negative.
Rule: at each node, split on the attribute with the maximum information gain.

Derivation from First Principles

Information gain is mutual information between attribute A and the class label Y:

$$\text{IG}(S, A) = I(A; Y) = H(Y) - H(Y|A)$$

Here $H(Y) = H(S)$ is the entropy of the current node S before splitting. The conditional entropy $H(Y|A)$ is the expected entropy after splitting on the values of A:

$$H(Y|A) = \sum_v P(A = v)\, H(Y|A = v) = \sum_v \frac{|S_v|}{|S|} H(S_v)$$

where S_v is the subset of examples in S for which attribute A takes value v. Substituting into the mutual information:

$$\boxed{\text{Gain}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)}$$

Why Gini Index? The Gini index $G(S) = 1 - \sum_{i=1}^{K} p_i^2$ follows from misclassification probability under randomized classification: pick a class with probability p_i, then the probability of misclassifying an example from class i is $\sum_{j \neq i} p_j$. The expected misclassification probability:

$$\sum_{i=1}^{K} p_i \sum_{j \neq i} p_j = \sum_{i=1}^{K} p_i(1 - p_i) = \sum_{i=1}^{K} (p_i - p_i^2) = 1 - \sum_{i=1}^{K} p_i^2 = G(S)$$

Gini measures the expected error of a randomized classifier proportional to class frequencies, and it requires no logarithms to compute.

Information Gain Example

Consider a node with 100 examples, split on an attribute into three child nodes with counts 30, 50, 20 (total still 100). Information gain is:

$$H(S) - \left(\frac{30}{100} H(S_1) + \frac{50}{100} H(S_2) + \frac{20}{100} H(S_3) \right)$$

The attribute providing the largest positive value of this quantity is chosen.

For the Play-Tennis dataset, the information gains are:

Humidity: 0.151, Wind: 0.048, Temperature: 0.029, Outlook: 0.246

So `Outlook` is selected as the root split.

Gini Index An alternative to entropy is the **Gini index**, which measures the expected error if we randomly assign examples to classes based on class probabilities:

Gini Index

$$\boxed{G(S) = 1 - \sum_{i=1}^{K} p_i^2}$$

Gini is minimized (zero) when a node is perfectly pure ($p_i = 1$ for one class), and maximized when all classes are equally frequent. Gini index is faster to compute than entropy (no log required) and is used by CART (Classification and Regression Trees).

The Gini-based information gain is defined the same way:

$$\text{Gain}_G(S, A) = G(S) - \sum_{v=1}^{V} \frac{|S_v|}{|S|} G(S_v)$$

Information Gain vs. Gini vs. Gain Ratio

Information gain prefers attributes with many values (more splits ⇒ more gain). ID3 uses information gain; CART uses Gini index; C4.5 (successor to ID3) uses Gain Ratio, which normalizes by the attribute's intrinsic information to correct this bias toward high-cardinality attributes.

CART Algorithm CART (Classification and Regression Trees, Breiman et al. 1984) is the most influential decision tree algorithm. Key characteristics:

- **Binary splits** at every node, even for multi-valued categorical attributes (all-splits-subset approach).
- Uses **Gini index** for classification, **squared error** for regression.
- Continuous attributes are discretized by trying candidate thresholds at midpoints between consecutive sorted values.
- **Recursive partitioning**: repeat the splitting process on each child node.

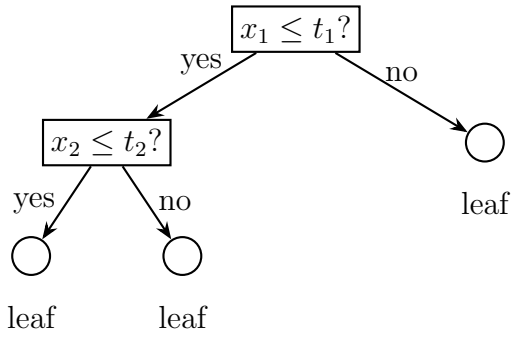


Figure 11: Decision tree: each internal node splits on one attribute; leaves make predictions.

Stopping Criteria Recursive splitting must stop to avoid overfitting. Common stopping criteria:

1. **Pure node**: all examples at the node belong to the same class (entropy = 0, gain = 0).
2. **Minimum samples**: fewer than $n_{\min}$ examples remain at a node; stop and assign the majority class.
3. **Maximum depth**: tree has reached a user-specified depth limit.
4. **No positive gain**: no attribute gives any positive information gain; further splits would not reduce impurity.

Overfitting and Pruning A fully grown tree perfectly classifies all training examples but often generalizes poorly. As the tree gets deeper, fewer examples occupy each node, leading to high variance. **Pruning** removes subtrees to improve generalization:

- **Pre-pruning** (early stopping): halt splitting before the tree is fully grown using one of the stopping criteria above.
- **Post-pruning**: grow the tree fully, then remove subtrees one by one, choosing the removal that maximizes validation-set accuracy.

4.2.3 Decision Tree Ensembles and Random Forests

Ensembles of Trees A single decision tree has high variance (different splits give different trees on different training sets). **Ensemble methods** combine multiple trees to produce better predictions by averaging out this variance:

- **Classification**: majority vote among all trees.
- **Regression**: average of all tree predictions.

For an ensemble to be effective, the individual learners should be independent of each other. Decision trees in bagging are dependent because they tend to pick the same powerful attributes at the root. Random forests solve this.

Bagging (Bootstrap Aggregation)

Bagging

Bagging (Breiman, 1996) creates B independent bootstrapped datasets by sampling the original training set with replacement: each bootstrap sample contains roughly 63% unique examples from the original set (the remaining 37% appear multiple times). A separate decision tree is grown on each bootstrap sample. The final prediction is the majority vote (classification) or average (regression) across all B trees.
Bagging reduces variance by averaging. Each tree sees slightly different data, producing slightly different models; averaging cancels out their individual fluctuations. However, trees in bagging are still highly correlated because they tend to pick the same attributes.

Random Forests

Random Forests (Breiman, 2001)

Random Forests add one critical modification on top of bagging: **attribute bagging** (or random subspace method). At each node of every tree, instead of evaluating all attributes for the best split, we evaluate only a random subset of m attributes.
A random forest is random in two senses:
1. **Sample bagging**: bootstrapped samples for each tree (same as bagging).
2. **Attribute bagging**: random subset of features is considered at every split node.

Default choices for m:
- **Classification**: $m = \lfloor \sqrt{D} \rfloor$, where D is total number of features.
- **Regression**: $m = \lfloor D/3 \rfloor$.

Why Random Forests Work

Attribute bagging ensures that trees cannot always choose the same highly predictive attribute. This decorrelates the individual trees: some trees will split on a different attribute first, leading to very different tree structures. The variance reduction from averaging is much greater when the trees are less correlated.
Intuitively: if there is one dominant attribute that is the best split almost everywhere, bagging trees will all pick it and remain highly correlated. Attribute bagging forces some trees to try second-choice attributes, producing a richer set of models whose predictions cancel each other's noise.

Random Forest Algorithm

1. Set number of trees B (e.g., 100, 500, 1000).
2. For $b = 1, 2, \ldots, B$:
   (a) Bootstrap: sample a training set S_b of size N with replacement from the original training set.
   (b) Grow a decision tree T_b on S_b with these modifications:
   - At each split node, randomly select m features (a hyperparameter, typically $\sqrt{D}$ for classification).
   - Evaluate only those m features for the best split (by Gini or information gain).

- Grow trees fully (no pruning; let each tree be overfitting — the ensemble averages out the variance).

3. Prediction:
   - **Classification**: $\hat{y} = \text{majority_vote}\big(\{T_b(\mathbf{x})\}_{b=1}^{B}\big)$.
   - **Regression**: $\hat{y} = \dfrac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})$.

Properties of Random Forests

- **High accuracy**: often among the best "off-the-shelf" supervised learning methods, comparable to gradient boosting without the tuning complexity.
- **Robust to overfitting**: averaging many trees controls variance, even though each individual tree is heavily overfitted.
- **Natural measure of feature importance**: the Gini impurity (or information gain) summed over all trees gives a reliable ranking of feature usefulness.
- **Out-of-Bag (OOB) error estimate**: for each example, average predictions only from trees whose bootstrap sample did not include it. This is essentially a built-in cross-validation estimate requiring no extra computation.
- **Handles mixed attribute types** natively, with no need for normalization or one-hot encoding of categorical features.
- **Bias is not reduced**: each tree in a forest is typically deeper than a single well-pruned tree, so forests can still have high bias if the underlying model class is too simple (e.g., very small m).

Boosting (Mention) Decision tree ensembles can also be created via **boosting**, which works differently from bagging/random forests:

- **AdaBoost**: sequentially trains trees, each time focusing more on examples that previous trees misclassified.
- **Gradient Boosting / XGBoost / LightGBM**: fit trees to the gradient of the residual of the current model, producing powerful state-of-the-art results on tabular data.

4.2.4 Continuous-Valued Attributes

Decision trees can also handle continuous features. The idea is to turn each continuous attribute into a sequence of threshold-based binary splits.

Threshold Selection Algorithm Given a continuous attribute A with observed values $\{v_1, v_2, \ldots, v_m\}$ sorted in increasing order:

1. Sort the attribute values: $v_1 < v_2 < \cdots < v_m$.
2. Generate $m-1$ candidate thresholds at midpoints between consecutive values: $c_k = \dfrac{v_k + v_{k+1}}{2}$ for $k = 1, \ldots, m-1$.
3. For each candidate threshold c, evaluate $A_c = \mathbf{1}[A \le c]$ as a boolean attribute using information gain or Gini index.
4. Choose the threshold c^* that maximizes the information gain.
5. Optionally, set the actual split to the largest observed value not exceeding c^*.

Example Attribute `Temperature`: $40, 48, 60, 72, 80, 90$ with class labels `No, No, Yes, Yes, Yes, No`.

Sorted values: $\{40, 48, 60, 72, 80, 90\}$. Candidate thresholds (midpoints): $44, 54, 66, 76, 85$.

For each threshold, compute information gain and pick the best. Note that the two child branches can still contain multiple temperature values, so further splits on `Temperature` with different thresholds may be applied deeper in the tree.

5 Generative Models

5.1 Variational Autoencoder

5.1.1 Motivation: Unsupervised Generative Modeling

Given an unlabeled dataset $\{\mathbf{x}_i\}_{i=1}^N$ where each $\mathbf{x}_i \in \mathbb{R}^D$ represents an image, we want to learn the underlying data distribution $p(\mathbf{x})$ so that we can generate new images similar to (but different from) those in the dataset. We introduce a low-dimensional **latent variable** $\mathbf{z} \in \mathbb{R}^J$ that explains the observed data:

$$\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I), \qquad \mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z})$$

where I is the $J \times J$ identity matrix, and $p_\theta(\mathbf{x} \mid \mathbf{z})$ is a deep neural network (the "decoder") with parameters θ. The latent dimension J is much smaller than D (e.g., $J = 20$ vs. $D = 784$ for MNIST).

VAE Generative Model

The decoder is parameterized as a Gaussian with mean and (diagonal) covariance determined by a neural network:

$$\boxed{p_\theta(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}\big(\mathbf{x} \mid \boldsymbol{\mu}_x(\mathbf{z}, \theta), \operatorname{diag}\big(\boldsymbol{\sigma}_x^2(\mathbf{z}, \theta)\big)\big)}$$

where $\boldsymbol{\mu}_x(\mathbf{z}, \theta)$ and $\boldsymbol{\sigma}_x(\mathbf{z}, \theta)$ are the outputs of the decoder network. This defines the conditional distribution of data given a latent code.
The joint likelihood of observed data:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x}, \mathbf{z})\, d\mathbf{z} = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$$

This integral is intractable for high-dimensional $\mathbf{x}$, so direct maximum-likelihood training is impossible.

5.1.2 Encoder and Decoder

To make learning tractable, we introduce an auxiliary model $q_\phi(\mathbf{z} \mid \mathbf{x})$ called the **encoder** (or recognition model), which approximates the true posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$.

Encoder Parameterization

The encoder is also parameterized as a Gaussian:

$$\boxed{q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\mathbf{z} \mid \boldsymbol{\mu}_z(\mathbf{x}, \phi), \operatorname{diag}\big(\boldsymbol{\sigma}_z^2(\mathbf{x}, \phi)\big)\big)}$$

where $\boldsymbol{\mu}_z(\mathbf{x}, \phi)$ and $\boldsymbol{\sigma}_z(\mathbf{x}, \phi)$ are outputs of the encoder network with parameters ϕ. Together, the encoder and decoder form an autoencoder-like architecture, but with stochastic connections.

5.1.3 Objective Function: ELBO

We maximize a lower bound on the per-sample log-likelihood $\log p_\theta(\mathbf{x})$ — the **Evidence Lower Bound (ELBO)**.

Variational Lower Bound Derivation

Starting from the non-negativity of KL divergence:

$$0 \leq D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x}))$$

Expanding the KL divergence:

$$\begin{aligned} D_{\mathrm{KL}}(q_\phi \,\|\, p_\theta(\mathbf{z} \mid \mathbf{x})) &= \int q_\phi(\mathbf{z} \mid \mathbf{x}) \log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z} \mid \mathbf{x})} \, d\mathbf{z} \\ &= \int q_\phi(\mathbf{z} \mid \mathbf{x}) \log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})\, p_\theta(\mathbf{x})}{p_\theta(\mathbf{x}, \mathbf{z})} \, d\mathbf{z} \\ &= \int q_\phi(\mathbf{z} \mid \mathbf{x}) \log p_\theta(\mathbf{x}) \, d\mathbf{z} + \int q_\phi(\mathbf{z} \mid \mathbf{x}) \log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p_\theta(\mathbf{z}, \mathbf{x})} \, d\mathbf{z} \\ &= \log p_\theta(\mathbf{x}) - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] + D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})) \end{aligned}$$

Rearranging and using $D_{\mathrm{KL}} \geq 0$:

$$\boxed{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] - D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})) \;\leq\; \log p_\theta(\mathbf{x})}$$

This is the ELBO. Maximizing it maximizes a lower bound on the log-likelihood. Equality holds when $q_\phi(\mathbf{z} \mid \mathbf{x}) = p_\theta(\mathbf{z} \mid \mathbf{x})$ exactly, i.e., when the encoder perfectly recovers the true posterior.

ELBO Interpretation

The ELBO has two terms with clear meanings:

1. **Reconstruction Term**: $\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]$ measures how well the decoder can reconstruct $\mathbf{x}$ from a latent code sampled from the encoder. This is the "reconstruction error" in log-probability space.
2. **KL Regularization Term**: $-D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z}))$ penalizes diverging too far from the prior $\mathcal{N}(\mathbf{0}, I)$. This keeps the latent space continuous and well-behaved for generation.

5.1.4 The Reparameterization Trick

Why Reparameterization Is Needed

Computing the reconstruction term requires sampling from the encoder:

$$\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] \approx \frac{1}{K} \sum_{k=1}^{K} \log p_\theta(\mathbf{x} \mid \mathbf{z}^{(k)}), \qquad \mathbf{z}^{(k)} \sim q_\phi(\mathbf{z} \mid \mathbf{x})$$

The problem: the sampling operation is stochastic and breaks the gradient ∇_ϕ. While the left-hand side depends on the encoder parameters ϕ (through the sampling distribution), the right-hand side uses concrete samples that no longer depend on ϕ at all. Standard backpropagation cannot flow gradients through a random sampling step.

The Reparameterization Trick

For a Gaussian encoder, we can rewrite the sampling step in a deterministic form:

$$\boxed{\mathbf{z} = \boldsymbol{\mu}_z(\mathbf{x}, \phi) \;+\; \boldsymbol{\sigma}_z(\mathbf{x}, \phi) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)}$$

where $\odot$ denotes element-wise multiplication. This is equivalent to sampling from $\mathcal{N}(\boldsymbol{\mu}_z, \mathrm{diag}(\boldsymbol{\sigma}_z^2))$, but now:

- $\boldsymbol{\mu}_z$ and $\boldsymbol{\sigma}_z$ are deterministic functions of ϕ (outputs of the encoder network)
- $\boldsymbol{\epsilon}$ is stochastic, but it acts as an input noise that does not depend on ϕ
- For any fixed $\boldsymbol{\epsilon}$, the mapping from ϕ to $\mathbf{z}$ is deterministic

This means gradients can now flow through the computation graph, enabling end-to-end optimization of both θ and ϕ.

5.1.5 Closed-Form KL Divergence for Two Gaussians

Both $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$ and $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I)$ are Gaussians, so their KL divergence has a closed-form solution:

KL Divergence: $\mathcal{N}(\boldsymbol{\mu}, \Sigma) \,\|\, \mathcal{N}(\mathbf{0}, I)$

For the encoder with diagonal covariance, with $\boldsymbol{\mu}_z = (\mu_1, \ldots, \mu_J)$ and $\boldsymbol{\sigma}_z^2 = (\sigma_1^2, \ldots, \sigma_J^2)$ (the j-th component for input $\mathbf{x}_i$ is μ_{ij} and σ_{ij}):

$$\boxed{D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}_i) \,\|\, p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^{J} \left(1 + \log(\sigma_{ij}^2) - \mu_{ij}^2 - \sigma_{ij}^2\right)}$$

This follows from the general KL divergence formula for two multivariate Gaussians:

$$D_{\mathrm{KL}}\big(\mathcal{N}(\boldsymbol{\mu}_1, \Sigma_1) \,\|\, \mathcal{N}(\boldsymbol{\mu}_2, \Sigma_2)\big) = \tfrac{1}{2}\left[\mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \Sigma_2^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - J + \log \tfrac{|\Sigma_2|}{|\Sigma_1|}\right]$$

Setting $\boldsymbol{\mu}_2 = \mathbf{0}$, $\Sigma_2 = I$, and $\Sigma_1 = \mathrm{diag}(\boldsymbol{\sigma}_z^2)$ gives the result above.

5.1.6 Final VAE Objective

Practical Training Objective

Putting everything together, the per-sample objective function maximized by gradient ascent is:

$$\boxed{\begin{aligned}\mathcal{L}(\mathbf{x}_i, \theta, \phi) \approx \frac{1}{K} \sum_{k=1}^{K} \log p_\theta(\mathbf{x}_i \mid \mathbf{z}^{(k)}) \\ + \frac{1}{2} \sum_{j=1}^{J} \left(1 + \log(\sigma_{ij}^2) - \mu_{ij}^2 - \sigma_{ij}^2\right)\end{aligned}}$$

where $\mathbf{z}^{(k)} = \boldsymbol{\mu}_z(\mathbf{x}_i, \phi) + \boldsymbol{\sigma}_z(\mathbf{x}_i, \phi) \odot \boldsymbol{\epsilon}^{(k)}$ and $\boldsymbol{\epsilon}^{(k)} \sim \mathcal{N}(\mathbf{0}, I)$.
In practice, $K = 1$ (a single Monte Carlo sample per data point) is almost always sufficient, because the stochasticity of $\boldsymbol{\epsilon}$ combined with minibatch SGD provides effective exploration. Both encoder parameters ϕ and decoder parameters θ are optimized jointly using stochastic gradient ascent on $\mathcal{L}$.

5.1.7 Generating Examples with a Trained VAE

After training, only the **decoder** is needed to generate new samples:

$$\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I), \qquad \mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z})$$

We sample a latent code from the prior and pass it through the trained decoder. The prior $\mathcal{N}(\mathbf{0}, I)$ is chosen specifically so that the encoder is regularized to map training data

into a roughly standard-normal region of latent space, making the prior a good sampling distribution.

Alternatively, one can manually pick $\mathbf{z}$ values to explore the latent space structure. For example, in a 2-D latent space learned from Frey Faces, the x-axis encodes head pose and the y-axis encodes smile intensity.

5.1.8 VAEs in Modern Deep Generative Modeling

- **Latent Diffusion** (e.g., Stable Diffusion): Instead of denoising in the pixel space, train the generative process in a compact VAE latent space. A pre-trained VAE encodes $\mathbf{x} \mapsto \mathbf{z}$; the diffusion model denoises in $\mathbf{z}$; the VAE decoder maps back to pixels. This is dramatically more efficient.
- **Flow / Flow-Matching**: Similarly learn an ODE or velocity field in the VAE latent domain for efficient generation, then decode to pixels.
- VAEs remain highly useful as latent compressors (dimensionality reduction), even though diffusion models and flow models achieve higher fidelity for direct image generation.

5.2 Generative Adversarial Networks

Generative Adversarial Networks (GANs, Goodfellow et al., 2014, "Generative Adversarial Nets") learn a generator $g_{\theta_g}(\mathbf{z})$ that maps a latent vector $\mathbf{z}$ to a synthetic data point $\mathbf{x}$, using adversarial training against a discriminator $D_{\theta_d}(\mathbf{x})$ that tries to tell real data apart from generated data.

GAN Setup

The generator $g_{\theta_g}(\mathbf{z}) : \mathbb{R}^J \to \mathbb{R}^D$ is a deep neural network with parameters θ_g. It maps a latent vector $\mathbf{z}$ sampled from a simple prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I)$ to a vector $\mathbf{x}$ in data space. The generator defines an implicit distribution $p_g(\mathbf{x})$ — the distribution of $\mathbf{x} = g_{\theta_g}(\mathbf{z})$ when $\mathbf{z} \sim p(\mathbf{z})$. Unlike a VAE, $p_g(\mathbf{x})$ has no closed form.
The discriminator $D_{\theta_d}(\mathbf{x}) : \mathbb{R}^D \to [0, 1]$ is another deep neural network with parameters θ_d. It maps a data point $\mathbf{x}$ to the probability that $\mathbf{x}$ is a real sample from the true data distribution $p_r(\mathbf{x})$. Its input can be either a real training example or a fake example generated by the generator.

5.2.1 The Minimax Objective

GAN Minimax Objective

The GAN training objective is a minimax game between generator and discriminator:

$$\boxed{\theta_g^* = \arg\min_{\theta_g} \max_{\theta_d} V(\theta_g, \theta_d)}$$

where the value function V is defined as:

$$\boxed{\begin{aligned} V(\theta_g, \theta_d) &= \mathbb{E}_{\mathbf{x}\sim p_r(\mathbf{x})}[\log D_{\theta_d}(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\big[\log\big(1 - D_{\theta_d}\big(g_{\theta_g}(\mathbf{z})\big)\big)\big] \\ &= \mathbb{E}_{\mathbf{x}\sim p_r(\mathbf{x})}[\log D_{\theta_d}(\mathbf{x})] + \mathbb{E}_{\mathbf{x}\sim p_g(\mathbf{x})}[\log(1 - D_{\theta_d}(\mathbf{x}))] \end{aligned}}$$

- The **discriminator** wants to maximize V: label real examples with $D \approx 1$ (first term large) and fake examples with $D \approx 0$ (second term large). This is exactly the cross-entropy loss for binary classification with true label 1 for real and 0 for fake.
- The **generator** wants to minimize V: produce fakes such that the discriminator assigns high probability $D \approx 1$ to them, making the second term small.

(a) Optimal Discriminator Derivation

Start from $V(G, D) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{x}\sim p_g}[\log(1 - D(\mathbf{x}))]$. Write the pointwise version:

$$V(D) = \int p_r(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x}))\, d\mathbf{x}$$

For each fixed $\mathbf{x}$, we maximize the integrand $f(D) = p_r \log D + p_g \log(1 - D)$ over $D \in (0, 1)$. Take derivative and set to zero:

$$\frac{df}{dD} = \frac{p_r}{D} - \frac{p_g}{1 - D} = 0 \implies p_r(1 - D) = p_g D \implies p_r = D(p_r + p_g)$$

Solving for D:

$$\boxed{D^*(\mathbf{x}) = \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_g(\mathbf{x})}}$$

Verify second derivative for maximality:

$$\frac{d^2 f}{dD^2} = -\frac{p_r}{D^2} - \frac{p_g}{(1 - D)^2} < 0$$

This is strictly negative for all valid densities ($p_r > 0$, $p_g > 0$), confirming D^* is the unique global maximum.

(b) Minimax Objective Minimizes JS Divergence

Substitute D^* into $V(G, D^*)$:

$$V(G) = \mathbb{E}_{p_r}\left[\log \frac{p_r}{p_r + p_g}\right] + \mathbb{E}_{p_g}\left[\log \frac{p_g}{p_r + p_g}\right]$$

Note that we can rewrite the log-ratios using the mixture $M = (p_r + p_g)/2$:

$$\frac{p_r}{p_r + p_g} = \frac{p_r/M}{(p_r + p_g)/M} = \frac{2p_r/M}{2} = \frac{1}{2}\frac{p_r}{M}$$

Similarly:

$$\frac{p_g}{p_r + p_g} = \frac{1}{2}\frac{p_g}{M}$$

Substituting back into $V(G)$:

$$\begin{aligned} V(G) &= \mathbb{E}_{p_r}\left[\log \tfrac{1}{2} + \log \tfrac{p_r}{M}\right] + \mathbb{E}_{p_g}\left[\log \tfrac{1}{2} + \log \tfrac{p_g}{M}\right] \\ &= \log \tfrac{1}{2} + \mathbb{E}_{p_r}\left[\log \tfrac{p_r}{M}\right] + \log \tfrac{1}{2} + \mathbb{E}_{p_g}\left[\log \tfrac{p_g}{M}\right] \\ &= 2 \cdot \log \tfrac{1}{2} + \mathrm{KL}(p_r \| M) + \mathrm{KL}(p_g \| M) \\ &= -\log 4 + \mathrm{KL}(p_r \| M) + \mathrm{KL}(p_g \| M) \end{aligned}$$

Recognizing $M = (p_r + p_g)/2$ as the mixture distribution, the Jensen–Shannon divergence is:

$$D_{\mathrm{JS}}(p_r, p_g) = \frac{1}{2}\mathrm{KL}(p_r \| M) + \frac{1}{2}\mathrm{KL}(p_g \| M)$$

Thus:

$$\boxed{V(G) = -\log 4 + 2\, D_{\mathrm{JS}}(p_r, p_g)}$$

Therefore, minimizing $V(G)$ w.r.t. G minimizes the JS divergence between the generator distribution p_g and the real data distribution p_r.

5.2.2 Training Procedure

Each training iteration alternates between improving the discriminator and improving the generator:

1. **Step 1 — Improve the Discriminator (run k times in practice, often $k = 1$)**:
   (a) Sample m real examples $\mathbf{x}_1, \ldots, \mathbf{x}_m$ from the training set
   (b) Sample m latent vectors $\mathbf{z}_1, \ldots, \mathbf{z}_m$ from $p(\mathbf{z})$
   (c) Generate m fake examples $\hat{\mathbf{x}}_i = g_{\theta_g}(\mathbf{z}_i)$
   (d) Update θ_d by ascending one gradient step of V:

$$\theta_d \leftarrow \theta_d + \eta \nabla_{\theta_d} V(\theta_g, \theta_d)$$

2. **Step 2 — Improve the Generator**:
   (a) Sample another m latent vectors $\mathbf{z}_1, \ldots, \mathbf{z}_m$ from $p(\mathbf{z})$
   (b) Update θ_g by descending one gradient step of V (or its alternative; see below):

$$\theta_g \leftarrow \theta_g - \eta \nabla_{\theta_g} V(\theta_g, \theta_d)$$

Crucially, neither network is trained to full convergence within one iteration. Each receives only a single gradient step. Training to convergence at each step would make the optimization intractable and destabilize training.

5.2.3 Theoretical Analysis

Optimal Discriminator for a Fixed Generator

Theorem (Goodfellow et al., 2014)

For a fixed generator with sufficient capacity, the optimal discriminator that maximizes $V(G, D)$ over all possible functions $D : \mathbb{R}^D \to [0, 1]$ is:

$$\boxed{D^*(\mathbf{x}) = \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_g(\mathbf{x})}}$$

GAN Minimizes Jensen–Shannon Divergence

GAN and Jensen–Shannon Divergence

Substituting the optimal discriminator D^* back into V gives:

$$\begin{aligned} V(G) = \max_D V(G, D) &= \mathbb{E}_{\mathbf{x} \sim p_r}\left[\log \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_g(\mathbf{x})}\right] + \mathbb{E}_{\mathbf{x} \sim p_g}\left[\log \frac{p_g(\mathbf{x})}{p_r(\mathbf{x}) + p_g(\mathbf{x})}\right] \\ &= -\log 4 + 2\, JSD(p_r \,\|\, p_g) \end{aligned}$$

where $JSD(p_r \,\|\, p_g)$ is the **Jensen–Shannon divergence** between the true data distribution and the generator distribution. Therefore:

$$\textbf{GAN training} = \textbf{minimizing the JS divergence } JSD(p_r \,\|\, p_g)$$

The discriminator is simply a device for approximating this intractable divergence.

Recall that VAE training minimizes KL divergence $KL(p_r \,\|\, p_\theta)$. The choice of divergence has important consequences for generated samples.

5.2.4 Two Generator Cost Functions and Vanishing Gradients

Vanishing Gradient Problem

The original generator objective minimizes $\mathbb{E}_{\mathbf{x}\sim p_g}[\log(1 - D(\mathbf{x}))]$. Early in training, the generator is poor and the discriminator quickly learns to assign $D(\mathbf{x}) \approx 0$ to all fake samples. Then:

$$\nabla_{\theta_g}\mathbb{E}_{\mathbf{x}\sim p_g}[\log(1 - D(\mathbf{x}))] \approx \nabla_{\theta_g}\mathbb{E}_{\mathbf{x}\sim p_g}[\log(1 - 0)] = 0$$

The gradient vanishes! The generator receives no signal on how to improve at precisely the moment it most needs it. Empirically, gradient norms can decay by 5 orders of magnitude within a few thousand discriminator iterations (Arjovsky et al., ICLR 2017).

Alternative Generator Cost

In practice, the generator is trained to maximize $\mathbb{E}_{\mathbf{x}\sim p_g}[\log D(\mathbf{x})]$ instead of minimizing $\mathbb{E}_{\mathbf{x}\sim p_g}[\log(1 - D(\mathbf{x}))]$. Equivalently, the generator cost is:

$$\mathcal{L}_{\text{gen}}(\theta_g) = -\mathbb{E}_{\mathbf{x}\sim p_g(\mathbf{x})}[\log D_{\theta_d}(\mathbf{x})]$$

This avoids vanishing gradients early on, because when $D(\mathbf{x}) \approx 0$ for fakes, $\log D(\mathbf{x}) \to -\infty$ and the gradient explodes (very large gradient pointing upward). However, when the discriminator becomes good ($D(\mathbf{x}) \approx 0$ for fakes), this alternative cost itself leads to unstable training because the gradients become extremely large.
Conclusion: Both generator costs have pathologies. In practice, neither network is trained to optimum — each gets only a single gradient step per iteration — to keep both costs in a regime with meaningful, manageable gradients.

5.2.5 Mode Collapse and Training Instability

Key Challenges with GANs

- **Mode Collapse**: The generator may produce samples from only a small subset of the true data distribution ("modes"). If the generator finds a small set of fakes that consistently fool the discriminator, it has no incentive to diversify. GANs have no built-in mechanism to encourage full coverage of the data distribution.
- **Training Instability**: The minimax objective has no single optimum; convergence requires careful balancing. Asymmetric update rates, learning-rate tuning, and architectural choices (e.g., BatchNorm, spectral normalization) help but do not eliminate the problem.
- **No Inference**: Unlike VAEs, standard GANs have no encoder, so there is no straightforward way to map a real image to its latent code $\mathbf{z}$ or to use the latent space for downstream tasks.

5.2.6 GAN vs. VAE

GAN vs. VAE: Tradeoffs

VAE	GAN
Minimizes KL divergence $KL(p_r \,\|\, p_\theta)$	Minimizes JS divergence $JSD(p_r \,\|\, p_g)$
Training is stable	Training is unstable
No mode collapse (maximizes likelihood, which covers all modes)	Prone to mode collapse
Generated samples are often blurrier (reconstruction loss averages over plausible outputs)	Generated samples are sharper and more realistic
Has an encoder for latent inference	No encoder; no latent inference
Latent space is continuous and well-behaved	Latent space structure is less interpretable
Closed-form ELBO; tractable lower bound	No likelihood lower bound

GANs generate sharper images because the discriminator pushes the generator toward producing highly detailed, pixel-perfect fakes. VAEs produce blurrier outputs because the reconstruction loss $\mathbb{E}[\log p(\mathbf{x} \mid \mathbf{z})]$ averages over all plausible reconstructions of $\mathbf{x}$, blurring fine details.

5.2.7 Fréchet Inception Distance (FID)

The **Fréchet Inception Distance (FID)** (Heusel et al., 2017) is the standard metric for evaluating generative image models. It compares the distributions of features extracted from the deepest layer of a pre-trained Inception-v3 network:

$$\boxed{\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|_2^2 + \text{Tr}\Big(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{1/2}\Big)}$$

where $\boldsymbol{\mu}_r, \Sigma_r$ are the mean and covariance of features from real images, and $\boldsymbol{\mu}_g, \Sigma_g$ are from generated images. Lower FID means generated images better match real images in terms of high-level semantic features (as captured by Inception-v3) and exhibit greater diversity.

5.2.8 Legacy and Influence

While GANs are no longer the dominant approach for state-of-the-art image generation (diffusion models now rule that space), the adversarial training paradigm GANs introduced remains highly influential. Variants like CycleGAN (unpaired image-to-image translation), Pix2Pix, StyleGAN, and BigGAN have addressed specialized problems. The core idea of pitting a generator against a discriminator has been adapted for text generation, speech synthesis, video generation, and even adversarial augmentation for self-supervised representation learning.

5.3 Diffusion Models

Motivation

Diffusion models are a class of deep generative models that learn to generate data by reversing a step-by-step denoising process. They have become the dominant paradigm for high-quality image generation (e.g., Stable Diffusion, DALL-E, Midjourney) be-

cause they produce high-quality samples with good mode coverage (unlike GANs, which suffer from mode collapse). The trade-off is slower generation speed, an active area of research.
Reference: Ho et al. (2020). Denoising Diffusion Probabilistic Models, NeurIPS.

Position of Diffusion in the Generative Landscape

- **GANs**: generate high-quality samples rapidly, but have poor mode coverage and are hard to train.
- **VAEs**: cover data modes faithfully, but often suffer from blurry or low-quality samples.
- **Diffusion models**: generate high-quality images (beat GANs) and have good mode coverage, but are slow in generation (a weakness being addressed by DDIM, LDM, etc.).

5.3.1 Denoising Diffusion Probabilistic Models (DDPM): The Forward Process

The Core Idea

The forward process (also called the **diffusion process**) takes a data sample $\mathbf{x}_0 \sim q(\mathbf{x})$ and applies T steps of Gaussian noise injection, producing a sequence of increasingly noisy samples $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$. After enough steps, $\mathbf{x}_T$ is essentially pure Gaussian noise. The reverse process learns to undo the noise, one step at a time, transforming pure noise back into realistic data.

Forward Process as a Markov Chain The forward process is defined as a **Markov chain** with fixed (non-learned) Gaussian transitions:

Forward Process Definition

$$\boxed{q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \;=\; \mathcal{N}\Big(\mathbf{x}_t;\; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\; \beta_t \mathbf{I}\Big)}$$

where $\beta_t \in [0, 1]$ is the **noise schedule** at step t, and $\mathbf{I}$ is the identity covariance matrix.
Equivalently, we can write the iterative update:

$$\mathbf{x}_t = \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

At each step, the signal in $\mathbf{x}_{t-1}$ is scaled down by $\sqrt{1-\beta_t}$ and Gaussian noise of variance β_t is added.

Typical noise schedule: linearly increase β_t from $\beta_1 = 10^{-4}$ to $\beta_T = 10^{-2}$, where T is typically 1000.

Closed-Form Transition Distribution A key property of the forward process is that we can compute $q(\mathbf{x}_t \mid \mathbf{x}_0)$ in closed form, without iterating through all intermediate steps. Define:

$$\alpha_t \;=\; 1 - \beta_t, \qquad \bar{\alpha}_t \;=\; \prod_{s=1}^{t} \alpha_s$$

Closed-Form Derivation of $q(\mathbf{x}_t|\mathbf{x}_0)$

The forward Markov chain is:

$$\mathbf{x}_s = \sqrt{\alpha_s}\,\mathbf{x}_{s-1} + \sqrt{1-\alpha_s}\,\boldsymbol{\epsilon}_s, \qquad \boldsymbol{\epsilon}_s \sim \mathcal{N}(\mathbf{0}, I) \text{ i.i.d.}$$

We unroll the recursion. After 1 step:

$$\mathbf{x}_1 = \sqrt{\alpha_1}\mathbf{x}_0 + \sqrt{1-\alpha_1}\boldsymbol{\epsilon}_1$$

After 2 steps, substitute $\mathbf{x}_1$:

$$\mathbf{x}_2 = \sqrt{\alpha_2}\mathbf{x}_1 + \sqrt{1-\alpha_2}\boldsymbol{\epsilon}_2 = \sqrt{\alpha_1\alpha_2}\mathbf{x}_0 + \sqrt{\alpha_2(1-\alpha_1)}\boldsymbol{\epsilon}_1 + \sqrt{1-\alpha_2}\boldsymbol{\epsilon}_2$$

After t steps, the coefficient of $\mathbf{x}_0$ is $\sqrt{\alpha_1\alpha_2\cdots\alpha_t} = \sqrt{\bar{\alpha}_t}$. The noise term is a sum of t independent Gaussian variables, each with its own coefficient c_s:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sum_{s=1}^{t} c_s\boldsymbol{\epsilon}_s$$

where:

$$c_s^2 = \alpha_{s+1}\alpha_{s+2}\cdots\alpha_t\cdot(1-\alpha_s) \quad \text{for } s < t, \qquad c_t^2 = 1-\alpha_t$$

We verify that the total noise variance sums to $1-\bar{\alpha}_t$ by induction. Base case $t=1$: $c_1^2 = 1-\alpha_1 = 1-\bar{\alpha}_1$, which holds. Assume $\sum_{s=1}^{t-1} c_s^{(t-1)2} = 1-\bar{\alpha}_{t-1}$ at step $t-1$. At step t, each previous noise term gets scaled by $\sqrt{\alpha_t}$, so $\sum_{s=1}^{t-1} c_s^{(t)2} = \alpha_t \sum_{s=1}^{t-1} c_s^{(t-1)2} = \alpha_t(1-\bar{\alpha}_{t-1})$.

Adding the new term $c_t^2 = 1-\alpha_t$:

$$\sum_{s=1}^{t} c_s^2 = \alpha_t(1-\bar{\alpha}_{t-1}) + (1-\alpha_t) = \alpha_t - \alpha_t\bar{\alpha}_{t-1} + 1 - \alpha_t = 1-\bar{\alpha}_t$$

Since $\boldsymbol{\epsilon}_1, \ldots, \boldsymbol{\epsilon}_t$ are independent $\mathcal{N}(\mathbf{0}, I)$ variables scaled by coefficients c_s, their sum is Gaussian with variance $\sum_s c_s^2 I = (1-\bar{\alpha}_t)I$. We can absorb all noise into a single sample:

$$\sum_{s=1}^{t} c_s\boldsymbol{\epsilon}_s = \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}, \qquad \bar{\boldsymbol{\epsilon}} \sim \mathcal{N}(\mathbf{0}, I)$$

Thus:

$$\boxed{\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}, \quad \bar{\boldsymbol{\epsilon}} \sim \mathcal{N}(\mathbf{0}, I)}$$

which is equivalent to $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t|\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)I)$.

Closed-Form $q(\mathbf{x}_t \mid \mathbf{x}_0)$

$$\boxed{q(\mathbf{x}_t \mid \mathbf{x}_0) \;=\; \mathcal{N}\big(\mathbf{x}_t;\, \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\; (1-\bar{\alpha}_t)\,\mathbf{I}\big)}$$

Equivalently, we can sample $\mathbf{x}_t$ directly from $\mathbf{x}_0$:

$$\mathbf{x}_t \;=\; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}, \quad \bar{\boldsymbol{\epsilon}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Here $\bar{\boldsymbol{\epsilon}}$ is the compound noise (the sum of all noise added over steps 1 through t), which is itself Gaussian.

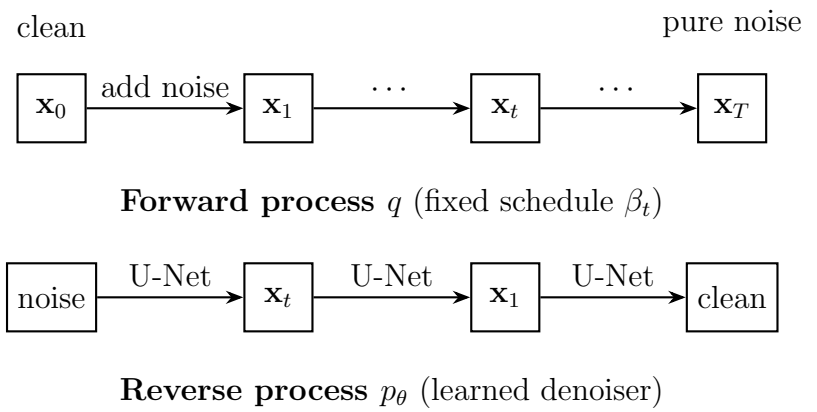


Figure 12: Diffusion model: the forward process (top) gradually adds Gaussian noise to a clean image $\mathbf{x}_0$ over T steps, producing pure noise $\mathbf{x}_T$. The learned reverse process (bottom) trains a U-Net to iteratively denoise, recovering a clean image from pure noise.

Interpretation

- $\sqrt{\bar{\alpha}_t}$ is the total signal retained from the original $\mathbf{x}_0$ after t steps. It decreases monotonically: $\bar{\alpha}_1 > \bar{\alpha}_2 > \cdots > \bar{\alpha}_T \approx 0$.
- $\sqrt{1-\bar{\alpha}_t}$ is the total noise added up to step t. It increases monotonically.
- The noise added in step t (not cumulative) is $\sqrt{\beta_t} = \sqrt{1-\alpha_t}$.

The Final Step The noise schedule is designed so that $\bar{\alpha}_T \approx 0$, which means:

$$q(\mathbf{x}_T \mid \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

That is, $\mathbf{x}_T$ is essentially standard Gaussian noise independent of $\mathbf{x}_0$. This gives us a simple starting point for the reverse process.

Joint Distribution and Ancestral Sampling The joint distribution over the entire forward trajectory (given $\mathbf{x}_0$) factors via the Markov property:

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) \;=\; \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

Marginal $q(\mathbf{x}_t)$ can be obtained via **ancestral sampling**: sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, then $\mathbf{x}_t \sim q(\mathbf{x}_t \mid \mathbf{x}_0)$. This is important for training because it lets us sample arbitrary-step noisy versions of any training image.

5.3.2 The Reverse Process: Learning to Denoise

The forward process $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ is fixed. Our goal is to learn the reverse process, i.e., the transition from $\mathbf{x}_t$ back to $\mathbf{x}_{t-1}$:

Reverse Process Definition

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) \;=\; \mathcal{N}\big(\mathbf{x}_{t-1};\, \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\, \sigma_t^2 \mathbf{I}\big)$$

This is a Gaussian with learned mean $\boldsymbol{\mu}_\theta$ and fixed variance σ_t^2 (often set to β_t). Starting from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, we iteratively denoise back to $\mathbf{x}_0$.

Noise Predictor Formulation Rather than directly parameterizing the mean $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$, DDPM predicts the noise $\bar{\boldsymbol{\epsilon}}$ that was added to obtain $\mathbf{x}_t$ from $\mathbf{x}_0$. From the closed-form:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}$$

Solving for $\bar{\boldsymbol{\epsilon}}$:

$$\bar{\boldsymbol{\epsilon}} = \frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0}{\sqrt{1-\bar{\alpha}_t}}$$

Noise Predictor

A neural network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is trained to predict the compound noise $\bar{\boldsymbol{\epsilon}}$ given the noisy image $\mathbf{x}_t$ and the time step t:

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \approx \bar{\boldsymbol{\epsilon}}$$

The time step t is usually injected into the network via a sinusoidal position embedding, similar to Transformers.

Network Architecture: U-Net

U-Net for Noise Prediction

The standard architecture for the noise predictor is a **U-Net**:

- Input and output have the same spatial dimensions (e.g., 64×64).
- An encoder path progressively downsamples (extracts increasingly abstract features).
- A bottleneck processes the most abstract representation.
- A decoder path progressively upsamples.
- **Skip connections** directly copy feature maps from the encoder to the decoder, re-introducing fine spatial details lost during downsampling.

5.3.3 Training Objective

The DDPM training loss is derived from maximizing the log-likelihood via a variational lower bound. In practice, it is simplified to the mean-squared-error between the true noise and the predicted noise.

DDPM Training Objective

$$\boxed{\mathcal{L} \;=\; \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0),\, t \sim \mathcal{U}(\{1,\ldots,T\}),\, \bar{\boldsymbol{\epsilon}} \sim \mathcal{N}(\mathbf{0},\mathbf{I})}\left[\left\|\bar{\boldsymbol{\epsilon}} \;-\; \boldsymbol{\epsilon}_\theta(\mathbf{x}_t,\, t)\right\|^2\right]}$$

where $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}$.
Intuition: the loss is 0 when the network perfectly predicts the compound noise that was added. Minimizing this loss forces the network to learn the distribution of noise at every noise level.

Why This Loss?

The full variational lower bound involves KL divergences between Gaussians, which simplifies to a weighted sum of squared differences. DDPM drops the position-dependent weight factors and trains with the uniform MSE loss above. This simplification was empirically found to produce better samples than the theoretically more precise bound.

Sampling (Image Generation) Once trained, generating an image is the reverse process:

DDPM Sampling Algorithm

1. Start from pure Gaussian noise: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. For $t = T, T-1, \ldots, 1$:
   (a) Predict noise: $\hat{\boldsymbol{\epsilon}}_t = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$.
   (b) Compute posterior mean estimate:
$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) \;=\; \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\hat{\boldsymbol{\epsilon}}_t\right)$$
   (c) Sample $\mathbf{x}_{t-1} \sim \mathcal{N}\big(\boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\, \sigma_t^2 \mathbf{I}\big)$, where typically $\sigma_t^2 = \beta_t$.
3. Return $\mathbf{x}_0$ as the generated sample.

Why Is Generation Slow?

DDPM sampling evaluates the network $\boldsymbol{\epsilon}_\theta$ a full T times (e.g., 1000 evaluations per image). This is much slower than GAN or VAE generation (a single forward pass). Methods like DDIM and latent diffusion models address this bottleneck.

5.3.4 Theory: Variational Derivation

This section provides a high-level sketch of the theoretical justification for DDPM.

Joint Distributions The forward process defines a joint distribution over $\mathbf{x}_0$ and latent variables $\mathbf{x}_{1:T}$:
$$q(\mathbf{x}_{0:T}) = q(\mathbf{x}_0) \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

The reverse process defines:
$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t), \quad p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

Training as Maximizing Likelihood Our goal is to maximize $\mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0)}[\log p_\theta(\mathbf{x}_0)]$. This is intractable directly, so we optimize the variational lower bound (ELBO):
$$\mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0)}[\log p_\theta(\mathbf{x}_0)] \;\geq\; \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0),\, \mathbf{x}_{1:T} \sim q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)}\left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)}\right]$$

Minimizing the reverse KL divergence gives the loss:
$$\mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0),\, \mathbf{x}_{1:T} \sim q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)}\left[\log \frac{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})}\right]$$

Forward-Backward Decomposition Using the Markov property, we decompose $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)$ as:
$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) \;=\; \prod_{t=2}^{T} q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \;\cdot\; q(\mathbf{x}_T \mid \mathbf{x}_0)$$
This allows us to pair each forward transition with a corresponding reverse transition in the KL divergence.

Posterior of the Reverse Step

Lemma: Exact Reverse Posterior

The exact forward process posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ is Gaussian:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \;=\; \mathcal{N}\Big(\mathbf{x}_{t-1};\, \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0),\, \tilde{\beta}_t \mathbf{I}\Big)$$

with:

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) \;=\; \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t \;-\; \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\bar{\boldsymbol{\epsilon}}\right), \qquad \tilde{\beta}_t \;=\; \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t$$

Key Insight

The posterior mean $\tilde{\boldsymbol{\mu}}_t$ depends on $\bar{\boldsymbol{\epsilon}}$, which we know (since we control the forward process). If our learned $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ matches this posterior, the KL divergence for that step becomes zero. This is exactly why training a noise predictor works — the optimal $\boldsymbol{\epsilon}_\theta$ is the true noise $\bar{\boldsymbol{\epsilon}}$, and substituting it into the mean formula recovers $\tilde{\boldsymbol{\mu}}_t$.

Parameterization of the Reverse Mean We parameterize the reverse mean using the noise predictor:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) \;=\; \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t \;-\; \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right)$$

Setting $\sigma_t^2 = \beta_t$ (a fixed, non-learned variance), each term in the variational lower bound reduces to:

$$\mathbb{E}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\,\|\bar{\boldsymbol{\epsilon}} \;-\; \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2\right]$$

Dropping the position-dependent weight $\dfrac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}$ and summing over all t gives the simplified DDPM training objective we described earlier.

5.3.5 Making Diffusion Models More Efficient

A major drawback of DDPM is its slow sampling (T network evaluations per image). Several methods address this.

DDIM (Denoising Diffusion Implicit Models) DDIM (Song et al., ICLR 2021) samples at a subset of the T time steps. It defines a new non-Markovian forward process q' that still has the same closed-form $q'(\mathbf{x}_t \mid \mathbf{x}_0)$. At each step, DDIM takes a larger jump in the reverse direction:

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_s-\sigma_t^2}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

where $s < t$, and:

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}}$$

When $\sigma_t = 0$, DDIM is **deterministic**: given the same starting noise, it always produces the same image. This enables sampling with as few as 10–50 steps instead of 1000.

Latent Diffusion Models (LDM) / Stable Diffusion Latent Diffusion Models (Rombach et al., CVPR 2022) run the diffusion process in a lower-dimensional latent space, dramatically reducing computational cost:

1. **Phase 1**: Pre-train a VAE that maps pixels $\mathbf{x}$ to latent $\mathbf{z} = E(\mathbf{x})$ and back to pixels $\hat{\mathbf{x}} = D(\mathbf{z})$. For example, $3 \times 512 \times 512$ images $\Rightarrow 6 \times 64 \times 64$ latents.
2. **Phase 2**: Train a diffusion model on latents:

$$\mathcal{L}_{\text{LDM}} \;=\; \mathbb{E}_{\mathbf{x},\,t,\,\bar{\boldsymbol{\epsilon}}}\big[\|\bar{\boldsymbol{\epsilon}} \;-\; \boldsymbol{\epsilon}_\theta(\mathbf{z}_t,\,t)\|^2\big]\,, \quad \mathbf{z}_t = \sqrt{\bar{\alpha}_t}\,E(\mathbf{x}) + \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}$$

After diffusion generates a latent $\hat{\mathbf{z}}_0$, we decode with the VAE decoder: $\hat{\mathbf{x}} = D(\hat{\mathbf{z}}_0)$. Stable Diffusion is built on this approach.

5.3.6 Three Representations of the Reverse Process

The DDPM framework allows three equivalent ways to parameterize the reverse process, all recoverable from the closed-form forward distribution.

Noise Predictor (Default DDPM) The network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ predicts the compound noise $\bar{\boldsymbol{\epsilon}}$ that was added to create $\mathbf{x}_t$ from $\mathbf{x}_0$:

$$\bar{\boldsymbol{\epsilon}} = \frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0}{\sqrt{1-\bar{\alpha}_t}}$$

Training loss:

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0,\,t,\,\bar{\boldsymbol{\epsilon}}}\big[\|\bar{\boldsymbol{\epsilon}} \;-\; \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2\big]$$

Data Predictor Alternatively, we can predict the original clean data $\mathbf{x}_0$ directly from $\mathbf{x}_t$:

$$\bar{\boldsymbol{\epsilon}} = \frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0}{\sqrt{1-\bar{\alpha}_t}} \quad\Longrightarrow\quad \mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}}$$

Training loss for data prediction:

$$\mathcal{L}_{\text{data}} = \mathbb{E}_{\mathbf{x}_0,\,t,\,\bar{\boldsymbol{\epsilon}}}\big[\|\mathbf{x}_0 \;-\; D_\theta(\mathbf{x}_t, t)\|^2\big]$$

Data prediction allows direct regularization on generated outputs and is used in AlphaFold3 for protein structure prediction.

Score Predictor The **score** of a distribution $q(\mathbf{x})$ is $\nabla_{\mathbf{x}} \log q(\mathbf{x})$. From the closed-form Gaussian:

$$\log q(\mathbf{x}_t \mid \mathbf{x}_0) \propto -\frac{\|\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0\|^2}{2(1-\bar{\alpha}_t)} + \text{constant}$$

Taking the gradient:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid \mathbf{x}_0) \;=\; -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0}{1-\bar{\alpha}_t} \;=\; -\frac{\bar{\boldsymbol{\epsilon}}}{\sqrt{1-\bar{\alpha}_t}}$$

Score Predictor

A **score predictor** $s_\theta(\mathbf{x}_t, t)$ is trained to estimate $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid \mathbf{x}_0)$. It is directly related to the noise predictor:

$$s_\theta(\mathbf{x}_t, t) \;\approx\; -\frac{\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{1-\bar{\alpha}_t}}$$

The noise predictor loss can be rewritten as score matching:

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0,\,t,\,\bar{\boldsymbol{\epsilon}}}\big[(1-\bar{\alpha}_t)\,\|s_\theta(\mathbf{x}_t, t) \;-\; \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid \mathbf{x}_0)\|^2\big]$$

Score-based generative modeling (Song et al., 2021) generalizes DDPM using stochastic differential equations (SDEs), where diffusion is viewed as continuous-noise evolution.

5.3.7 Guided Generation

Standard DDPM sampling produces images unconditionally — the user has no control over the content. **Guided generation** methods let us steer the generation toward a desired class label or text prompt.

Intuition: Sampling as Gradient Updates Recall the approximate DDPM sampling rule derived from score matching:

$$\mathbf{x}_{t-1} \approx \frac{1}{\sqrt{\alpha_t}} \left[\mathbf{x}_t + (1-\alpha_t)\, \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)\right] + \sigma_t \mathbf{z}$$

At each step, we move uphill in $\log q(\mathbf{x}_t)$, increasing the probability of the sample. For class-conditional generation, we replace $\log q(\mathbf{x}_t)$ with $\log q(\mathbf{x}_t \mid y)$:

$$\mathbf{x}_{t-1} \approx \frac{1}{\sqrt{\alpha_t}} \left[\mathbf{x}_t + (1-\alpha_t)\, \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid y)\right] + \sigma_t \mathbf{z}$$

Classifier Guidance

Classifier Guidance (Dhariwal & Nichol, 2021)

Using Bayes' rule, $q(\mathbf{x}_t \mid y) = \dfrac{q(\mathbf{x}_t)\, q(y \mid \mathbf{x}_t)}{q(y)}$, so:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid y) \;=\; \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) \;+\; \nabla_{\mathbf{x}_t} \log q(y \mid \mathbf{x}_t)$$

If we train a separate **classifier** $p_\phi(y \mid \mathbf{x}_t)$ on noisy images, we can estimate $\nabla_{\mathbf{x}_t} \log p_\phi(y \mid \mathbf{x}_t)$ and add it to the sampling rule with a guidance strength hyperparameter w:

$$\mathbf{x}_{t-1} \;=\; \frac{1}{\sqrt{\alpha_t}} \left[\mathbf{x}_t \;+\; (1-\alpha_t)\left(\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) \;+\; w\, \nabla_{\mathbf{x}_t} \log p_\phi(y \mid \mathbf{x}_t)\right)\right] \;+\; \sigma_t \mathbf{z}$$

Rewriting in terms of the noise predictor:

$$\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, y) \;=\; \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \;-\; w \cdot \sqrt{1-\bar{\alpha}_t}\, \nabla_{\mathbf{x}_t} \log p_\phi(y \mid \mathbf{x}_t)$$

Larger w produces samples more strongly conditioned on the class label.

Classifier-Free Guidance

Classifier-Free Guidance (Ho & Salimans, 2022)

Instead of training a separate classifier, we train a single conditional noise predictor $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y)$ using labeled data $\{\mathbf{x}_0, y\}$:

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, y, t, \bar{\boldsymbol{\epsilon}}}\left[\|\bar{\boldsymbol{\epsilon}} \;-\; \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y)\|^2\right]$$

During training, we randomly set the condition y to a special "empty" token $\varnothing$ with some probability (e.g., 10%), so that $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing)$ approximates the unconditional noise predictor.
At inference time, we compute both the unconditional and conditional predictions:

$$\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, y) \;=\; \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing) \;+\; w \cdot (\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y) \;-\; \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing))$$

- $w = 1$: only the conditional predictor is used.

- $w > 1$: amplifies the difference between conditional and unconditional predictions, making samples more class-specific. Common choices are $w \in [5, 15]$, with 7.5 and 10 being standard.

Why "Classifier-Free"?

The gradient $\nabla_{\mathbf{x}_t} \log q(y \mid \mathbf{x}_t)$ that classifier guidance estimates via a separate classifier is directly approximated by $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y) - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing)$ in classifier-free guidance. No separate classifier is trained — the same noise predictor serves both roles. This is the standard conditioning approach in Stable Diffusion and DALL-E 2.

Text Guidance in Latent Diffusion / Stable Diffusion For text-to-image generation, the conditioning variable y is a text prompt, encoded into an embedding vector $\tau(y)$ by a pre-trained text encoder (e.g., CLIP). The training objective becomes:

$$\mathcal{L} \;=\; \mathbb{E}_{\mathbf{x},\, y,\, t,\, \bar{\boldsymbol{\epsilon}}}\big[\|\bar{\boldsymbol{\epsilon}} \;-\; \boldsymbol{\epsilon}_\theta(\mathbf{z}_t,\, t,\, \tau(y))\|^2\big]\,,$$

where $\mathbf{z}_t$ is the noisy latent. The text embedding $\tau(y)$ is injected into the noise predictor via **cross-attention** layers. During training, $\tau(y)$ is randomly replaced by $\varnothing$ to enable classifier-free guidance.

At each denoising step, the text embedding acts as a "reminder" to the model of the kind of image we want to generate.